

University of Denver

Digital Commons @ DU

Electronic Theses and Dissertations

Graduate Studies

Fall 11-22-2024

Model-Based Navigation and Control of Multirotor UAVs: A Machine Learning Approach

Serhat Sönmez
University of Denver

Follow this and additional works at: <https://digitalcommons.du.edu/etd>



Part of the [Controls and Control Theory Commons](#), and the [Robotics Commons](#)

Recommended Citation

Sönmez, Serhat, "Model-Based Navigation and Control of Multirotor UAVs: A Machine Learning Approach" (2024). *Electronic Theses and Dissertations*. 2521.

<https://digitalcommons.du.edu/etd/2521>

DOI

<https://doi.org/10.56902/ETDCRP.2024.39>



All Rights Reserved.

This Dissertation is brought to you for free and open access by the Graduate Studies at Digital Commons @ DU. It has been accepted for inclusion in Electronic Theses and Dissertations by an authorized administrator of Digital Commons @ DU. For more information, please contact jennifer.cox@du.edu, dig-commons@du.edu.

Model-Based Navigation and Control of Multirotor UAVs: A Machine Learning Approach

Abstract

In recent decades, unmanned systems, particularly Unmanned Aerial Vehicles (UAVs), have seen significant advancement and unprecedented growth in military, civilian and public domain applications. Scientists have focused on enhancing UAV navigation and control through cutting-edge technologies and support tools. UAVs find applications in many fields, except military, such as agriculture, infrastructure inspection, wildlife monitoring, search and rescue, emergency response, border protection, to name but a few relevant civilian applications. Given the faster-than-exponential increase of available computational power, learning-based algorithms have emerged as a prominent tool for (real-time) multirotor UAV navigation and control. This dissertation centers around the fusion of conventional control methods with learning algorithms to fine-tune and/or identify and estimate multirotor UAV aerodynamic parameters (model parameters), as well as controller parameters and gains, also tackling uncertainties. The combined multilayer navigation control architecture, based on obtained results, shows promise as it is real-time applicable and implementable.

Document Type

Dissertation

Degree Name

Ph.D. in Electrical Engineering

First Advisor

Kimon P. Valavanis

Second Advisor

Matthew J. Rutherford

Third Advisor

Alvaro Arias

Keywords

Adaptive control, Deep deterministic policy gradient (DDPG), Multirotor unmanned aerial vehicles (UAVs), Optimal control, Reinforcement learning

Subject Categories

Computer Engineering | Controls and Control Theory | Electrical and Computer Engineering | Engineering | Robotics

Publication Statement

Copyright is held by the author. User is responsible for all copyright compliance.

MODEL-BASED NAVIGATION AND CONTROL OF MULTIROTOR UAVS: A
MACHINE LEARNING APPROACH

A Dissertation

Presented to

the Faculty of the Daniel Felix Ritchie School of Engineering and Computer Science
University of Denver

In Partial Fulfillment

of the Requirements for the Degree

Doctor of Philosophy

by

Serhat Sönmez

March 2025

Advisor: Kimon P. Valavanis, Ph.D. and Matthew J. Rutherford, Ph.D.

Author: Serhat Sönmez

Title: Model-Based Navigation and Control of Multirotor UAVs: A Machine Learning Approach

Advisor: Kimon P. Valavanis, Ph.D. and Matthew J. Rutherford, Ph.D.

Degree Date: March 2025

ABSTRACT

In recent decades, unmanned systems, particularly Unmanned Aerial Vehicles (UAVs), have seen significant advancement and unprecedented growth in military, civilian and public domain applications. Scientists have focused on enhancing UAV navigation and control through cutting-edge technologies and support tools. UAVs find applications in many fields, except military, such as agriculture, infrastructure inspection, wildlife monitoring, search and rescue, emergency response, border protection, to name but a few relevant civilian applications. Given the faster-than-exponential increase of available computational power, learning-based algorithms have emerged as a prominent tool for (real-time) multirotor UAV navigation and control. This dissertation centers around the fusion of conventional control methods with learning algorithms to fine-tune and/or identify and estimate multirotor UAV aerodynamic parameters (model parameters), as well as controller parameters and gains, also tackling uncertainties. The combined multilayer navigation control architecture, based on obtained results, shows promise as it is real-time applicable and implementable.

Keywords — Reinforcement Learning, Multirotor UAVs, Optimal Control, Adaptive Control, DDPG

ACKNOWLEDGEMENTS

Dedicated to Mesut Atay and Ömer Mert.

I would like to deeply thank my advisors, Kimon P. Valavanis and Matthew J. Rutherford, for their guidance, patience and support toward the completion of this dissertation. Kimon did not only improve my technical background but also my humanitarian perspective and academic ethics. He always provided me with academic freedom to confidently express my ideas and opinions. He became more than just an advisor to me, he is a role model for my carrier. Dr. Rutherford opened a huge space for me in the lab and advised me on how to manage my PhD-life with his book suggestions and talks. I would like to thank my committee members, Alvaro Arias, Haluk Ogmen, and Rui Fan, for their support and crucial opinions that helped to improve my dissertations. I would also like to thank my dear colleagues, Simone and Laura, who accompanied me throughout this academic journey and improved me in every sense. I would like to thank my lovely country, the Republic of Türkiye, and the Ministry of National Education of the Republic of Türkiye for giving me an education opportunity abroad. I would also like to thank my parents and sister for their love and support. Last but the most important, I would like to thank my lovely wife, Sedef Sönmez, for her patience and support throughout my academic journal and hardest time of my life. Life is amazing with her.

This research has been partially supported by the Ministry of National Education of the Republic of Türkiye on behalf of the Istanbul Medeniyet University, Istanbul, Türkiye, and the D. F. Ritchie School of Engineering and Computer Science, University of Denver, Denver, CO 80208.

TABLE OF CONTENTS

ABSTRACT	ii
ACKNOWLEDGEMENTS	iii
TABLE OF CONTENTS	iv
LIST OF TABLES	vi
LIST OF FIGURES	vii
ABBREVIATIONS	x
CHAPTER 1 : INTRODUCTION	1
1.1 Motivation and Rationale	1
1.2 Problem Statement	2
1.3 Proposed Solution	3
1.4 Summary of Contributions	5
1.5 Dissertation Outline	6
CHAPTER 2 : LITERATURE REVIEW	7
2.1 Introduction	7
2.2 Definitions:	9
2.3 Offline Learning	15
2.4 Online Learning	47
2.5 Advantages and Limitations	59
2.6 Remarks	60
CHAPTER 3 : QUADROTOR MATHEMATICAL MODEL DERIVATION	61
3.1 Comprehensive N-E Based Quadrotor Mathematical Model	61
3.2 Remarks	71
CHAPTER 4 : PROPOSED NAVIGATION CONTROL ARCHITECTURE	72
4.1 Introduction	72
4.2 Conventional Controllers	73
4.3 Learning-Based Parameter Estimation Approaches	79
4.4 Estimation of Aerodynamic Uncertainties	93
CHAPTER 5 : RESULTS	94
5.1 Introduction	94
5.2 Proposed Controller Frameworks Results	95
5.3 Discussion of Results	112
5.4 Remarks	114
CHAPTER 6 : DISCUSSION AND CONCLUSION	115
6.1 Introduction	115

6.2	Discussion and Conclusions	115
6.3	Future Work	118
REFERENCES		120

LIST OF TABLES

TABLE 2.1	Offline Learning Papers	16
TABLE 2.1	Offline Learning Papers (Continued).	17
TABLE 2.1	Offline Learning Papers (Continued).	18
TABLE 2.2	Classification of Reinforcement Learning (Offline)	25
TABLE 2.3	Comparison of value function-based and policy search-based methods	36
TABLE 2.4	Online Learning Papers	47
TABLE 3.1	Equations of Motion of a quadrotor UAV based on N-E formulation	69
TABLE 4.1	UAV and Controller Parameters	72
TABLE 4.2	Training Parameters	84
TABLE 4.3	Comparison of hyperparameters for PID controller parameter es- timation task.	91
TABLE 5.1	RMSEs of Attitude and Position	100
TABLE 5.2	Attitude and Position RMSEs of Controller Frameworks	111
TABLE 5.3	Attitude and Position RMSEs of Controller Frameworks after $10^{th}second$	112

LIST OF FIGURES

FIGURE 1.1	UAV control strategy with learning-based estimation algorithm.	4
FIGURE 2.1	Publications of online and offline learning algorithms for control of multirotor UAVs since 2015 based on Google Scholar search.	8
FIGURE 2.2	Taxonomy of RL algorithms [13].	9
FIGURE 2.3	Taxonomy of DRL algorithms for UAV tasks [13].	10
FIGURE 2.4	Offline reinforcement learning block diagram illustration [16]. .	10
FIGURE 2.5	Online reinforcement learning block diagram illustration [16]. .	11
FIGURE 2.6	Off-policy RL algorithm configuration [16].	12
FIGURE 2.7	Block diagram and interaction between agent and environment [15].	25
FIGURE 2.8	Online learning controller scheme of [100].	49
FIGURE 2.9	Flow diagram of the racing parameter optimization during online training phase [98].	50
FIGURE 2.10	Block diagram of the online learning-based control developed by Sarabakha and Kayacan [95].	51
FIGURE 2.11	Block diagram of Neural-Fly algorithm offered by O’Connell et al. [101].	52
FIGURE 2.12	Block diagram of Fuzzy-RBF NN adaptive PID controller [103].	54
FIGURE 2.13	Block diagram of NN-based control architecture in [99].	55
FIGURE 2.14	Block diagram of the online training in StateRate proposed by [96].	58
FIGURE 3.1	Initial and body-fixed frames and forces and torques applied on the quadrotor [107].	62
FIGURE 4.1	Hierarchical controller structure.	74
FIGURE 4.2	The block diagram of DDPG algorithm [123].	80
FIGURE 4.3	Block diagram of the PID parameter tuning based on RL. . . .	82
FIGURE 4.4	Structure of the actor neural network used for PID parameter tuning.	83
FIGURE 4.5	Structure of the critic neural network used for PID parameter tuning.	84
FIGURE 4.6	Block diagram of PID parameter estimation based on RL. . . .	84
FIGURE 4.7	Structure of the actor neural network used in PID parameter Estimation.	86
FIGURE 4.8	Structure of the critic neural network in PID parameter estimation.	86
FIGURE 4.9	Change of the rotor speeds without motor dynamics.	87
FIGURE 4.10	Change of the rotor speeds with the motor dynamics.	88
FIGURE 4.11	(a) Wind velocities on x, y, and z axes in time, and (b) The magnitude of wind in time.	89

FIGURE 4.12	Block diagram of PID parameter estimation based on RL. . . .	90
FIGURE 4.13	Block diagram of LQR parameter estimation based on RL. . .	92
FIGURE 4.14	Block diagram of MRAC parameter estimation based on RL. .	93
FIGURE 5.1	The desired helix trajectory in 3-D.	94
FIGURE 5.2	The desired helix trajectory.	95
FIGURE 5.3	Manual-tuned PID position error in x , y , and z axes.	96
FIGURE 5.4	Manual-tuned PID attitude error in roll, pitch, and yaw angles.	97
FIGURE 5.5	The tuned parameters based on RL in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ	97
FIGURE 5.6	The estimated parameters based on RL in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ	98
FIGURE 5.7	Attitude errors: (a) roll angle ϕ , (b) pitch angle θ , (c) yaw angle ψ	99
FIGURE 5.8	Position errors: (a) x -axis, (b) y -axis, (c) z -axis.	99
FIGURE 5.9	The attitude error norms of the manually tuned, RL-based tuned, and RL-based estimated controllers.	100
FIGURE 5.10	The position error norms of the manually tuned, RL-based tuned, and RL-based estimated controllers.	101
FIGURE 5.11	The RL-based estimated PID parameters in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ	102
FIGURE 5.12	Position errors in x , y , and z axes for RL-based PID parameter estimation.	103
FIGURE 5.13	The position error norms of RL-based PID controller under the comprehensive scenario	104
FIGURE 5.14	Attitude errors in the roll, pitch, and yaw angles for RL-based PID parameter estimation.	104
FIGURE 5.15	The attitude error norms of RL-based estimated controllers for the comprehensive scenario.	105
FIGURE 5.16	The estimated parameters based on RL in time: (a) Q matrix, (b) R matrix	105
FIGURE 5.17	Position errors in x , y , and z axes for RL-based LQR parameter estimation.	106
FIGURE 5.18	The position error norms of RL-based LQR controller.	106
FIGURE 5.19	Attitude errors in the roll, pitch, and yaw angles for RL-based LQR controller.	107
FIGURE 5.20	The attitude error norms of RL-based LQR controllers.	108
FIGURE 5.21	The estimated MRAC parameters based on RL in time.	109
FIGURE 5.22	Position errors in x , y , and z axes for RL-based MRAC parameter estimation.	109

FIGURE 5.23	The position error norms of RL-based MRAC.	110
FIGURE 5.24	The attitude errors in the roll, pitch, and yaw angles for RL- based MRAC.	110
FIGURE 5.25	The attitude error norms of RL-based MRAC.	111

ABBREVIATIONS

The following abbreviations are used in the dissertation:

ADP	Approximate Dynamic Programming
AI	Artificial Intelligence
AMLQ	Adaptive Multi-Level Quantization
ANN	Artificial Neural Network
AR	Augmented Reality
ARE	Algebraic Riccati Equation
BS	Base Station
CAWS	Channel-Aware Waypoint Selection
CdRL	Consciousness-Driven Reinforcement Learning
CLSTM	Convolutional 1D Layers cascaded with LSTM layers
CNN	Convolutional Neural Network
DAIML	Domain Adversarially Invariant Meta-Learning
DDPG	Deep Deterministic Policy Gradient
DDPG-MC	DDPG-based Movement Control
DDQN	Double DQN
DeFRA	DRL-based Flight Resource Allocation framework
DGN	Graph Convolutional Reinforcement Learning
DL	Deep Learning
DNN	Deep Neural Network
DP	Dynamic Programming
DPG-IC	Deterministic Policy Gradient-Integral Compensator
DQN	Deep Q-Network
DQN-FRA	DQNs-based Flight Resource Allocation Policy
DRGN	Deep Recurrent Graph Network
DRL	Deep Reinforcement Learning
DSO	Direct Sparse odometry
DTW	Dynamic Time Warping
EM	Environmental Disturbance
FANAT	Flying Ad-hoc Network
FLS	Fuzzy Logic System
FPID	Fuzzy-PID

GAT	Graph Attention Network
GAT-FANET	GAT-based FANET
GCS	Ground Control Station
GRU	Gated Recurrent Unit
HAPPO	Heterogeneous-Agent PPO
HJB	Hamilton-Jacobian-Belmann equation
HMM	Hidden Markov Model
IBVS	Image-based Visual Servoing
LOS	Line-Of-Sight
LQR	Linear-Quadratic Regulator
LSPI	Least Square Policy Iteration
LSTM	Long Short-Term Memory
LSTMCNN	LSTM Layers interleaved with convolutional 1D layers
MAAC	Multi-Actor Attention Critic
MADDPG	Multi-Agent DDPG
MAE	Mean Absolute Error
MAPPO	Multi-Agent PPO
MAV	Micro Aerial Vehicle
MC	Monte Carlo
MD	Motor Dynamics
MDP	Markov Decision Process
meta-TD3	Meta Twin Delay Deep Deterministic policy gradient
ML	Machine Learning
MPC	Model Predictive Control
MRAC	Model-Reference Adaptive Control
NLGL	Nonlinear Guidance Law
NN	Neural Network
PBVS	Position-based Visual Servoing
PF	Particle Filter
PID	Proportional-Integral-Derivative
PILCO	Probabilistic Inference for Learning Control
PLATO	Policy Learning using Adaptive Trajectory Optimization
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
PPO-IC	Proximal Policy Optimization-Integral Compensator

PR	Policy Relief
PTRS	Planned Trajectory Random Scheduling
PWM	pulse-Width Modulation
RBF	Radial Basis Function
RBILC	Real-time Brain-inspired Learning Control
RDPG	Recurrent Deterministic Policy Gradient algorithm
RGB	Red-Blue-Green
RL	Reinforcement Learning
RMLP	Recurrent Multilayer Perceptron
RRT	Rapidly-exploring Random Tree
RSS	Received Signal Strength
SAC	Soft Action-Critic
SARSA	State-Action-Reward-State-Action
SDRGN	Soft Deep Recurrent Graph Network
SOL	Structured Online Learning-based algorithm
SSD	Single-Shot-Detection
SW	Significance Weighting
TD	Temporal-Difference
TD3	Twin-Delay Deep Deterministic Gradient
TLD	Tracking-Learning-Detection
TRPO	Trust Region Policy Optimization
TRPO-gae	TRPO with a generalized advantage estimator
UAV	Unmanned Aerial Vehicle
UGV	Unmanned Ground Vechile
UKF	Unscented Kalman Filter
VIO	Visual-Inertial Odometry

CHAPTER 1: INTRODUCTION

1.1. Motivation and Rationale

Alan M. Turing's technical report on "Intelligent Machinery", written in 1948 for the National Physical Laboratory, is considered as the first manifesto of Artificial Intelligence (AI). This first paper has been followed by Turing's 1950 paper with title "Can machines think?" [1], [2], and it may be argued that both papers are main contributors to creating new research directions in intelligent machines, machine learning, and learning-based control. These areas were further facilitated by advances in, and pioneering contributions of, Control System Theory to study complex systems, and, from the late 1960's, to study robotics and automation. Over the years, Control Theory and AI, because of the advent of digital computers and the exponential increase in computational power, have been proven to be effective, efficient, and reliable 'tools' used in robotics, and in robotic systems.

During the last two decades, Unmanned Aerial Vehicles (UAVs), or Unmanned Aircraft Systems (UAS), have witnessed unprecedented levels of growth, with civilian and public domain applications spanning power line inspection [3], mining area monitoring, [4], wildlife monitoring and conservation [5], border protection [6], building inspection [7], and precision agriculture [8], to name but a few applications. Although different UAV types and configurations have been utilized for such applications, multirotor UAVs, particularly quadrotors, have been the most commonly and widely used, due to their perceived advantages, effectiveness, hovering capabilities, and efficiency during flight.

This dissertation research is motivated by the challenge to establish a solid foundational framework and methodology, applicable and implementable in real-time, for effective and efficient modeling, navigation and control of multirotor UAVs that operate in dynamic and uncertain environments. For modeling purposes, the conventional Newton-Euler (N-E) formulation is used to provide the baseline multirotor model. But the N-E baseline model is further enhanced to account for gyroscopic and drag effects. The resulting complete model is adopted for controller design, implementation, testing, and performance evaluation. Included in this process is identification, estimation and learning of the underlying model parameters (aerodynamic parameters) and controller gains, which allow for the multirotor UAV to fly and operate under nominal and detrimental conditions in diverse environments. From this perspective, modeling, navigation and controller design reflect a multilayer learning-based architecture.

Multirotor UAVs belong to the family of nonlinear and underactuated systems with (mostly) time-varying parameters and unstructured uncertainties. For this type of systems, several model (aerodynamic) parameters (i.e., different drag types like blade, induced, and parasitic drag) cannot be calculated mathematically, or analytically. The same holds (may hold) true for controller gains, particularly when a family of admissible multirotor models is used, rather than one, nominal model. Under these assumptions, the alternative is to use offline, online, or a combination of offline-online learning techniques to 'learn' and/or 'estimate' these parameters, instead of (only) hand-tuning them, or follow trial-and-error techniques to determine their values. Viewed from this perspective, multirotor modeling, navigation and control reflect a combined and integrated model- and learning-based approach that is applicable and implementable in real-time.

1.2. Problem Statement

The central objective of this dissertation is to develop an integrated framework that combines conventional modeling and controller design techniques with state-of-

the-art (offline and online) learning methods and algorithms, to address and solve challenges related to multirotor UAV modeling, navigation and control, where model parameters and controller parameters and gains need to be identified and/or estimated. As such, model parameters and controller gains are adaptively estimated based on (measured in real-time) attitude and position errors, thus, improving trajectory tracking. Parameter estimation is based on reinforcement learning. For example, the agent learns to estimate the controller parameters or the aerodynamic uncertainties.

Consequently, focusing on the learning module of the overall system under consideration, it is important to provide answers to the following questions (addressed and answered in this dissertation):

- What does an agent learn?
- Why does an agent (need to) learn?
- How does an agent learn?
- How fast does an agent learn?
- How is the learning process implemented?

The end-result of the research, when answering the previous questions, offers a validated and verified methodology that is applicable to all types of UAV configurations. For proof-of-concept validation and verification (V&V) and for demonstration purposes, the quadrotor configuration is adopted as a testbed.

1.3. Proposed Solution

A hierarchical navigation and control architecture is proposed to effectively govern the behavior of multirotor UAVs in dynamic and uncertain environments. This

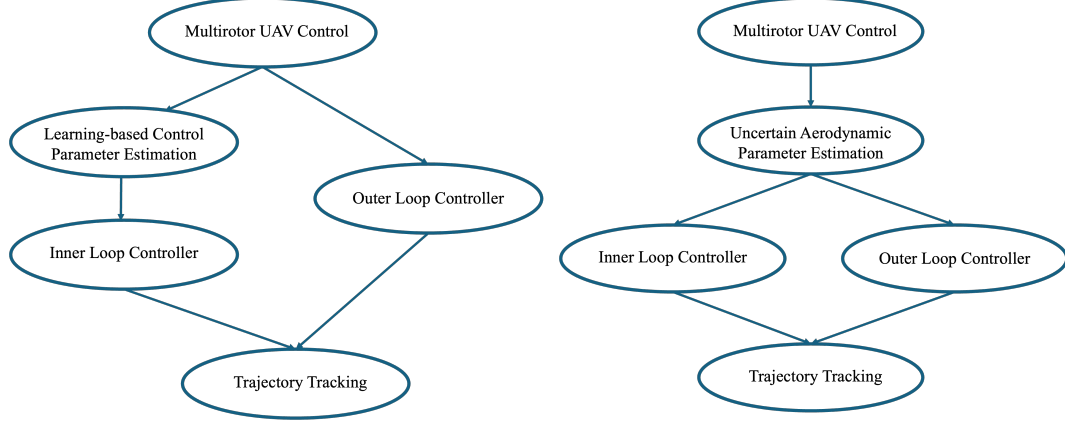


Figure 1.1: UAV control strategy with learning-based estimation algorithm.

architecture comprises an inner and outer loop structure, each serving distinct control objectives.

The outer loop is tasked with regulating thrust and desired Euler angles. A control mechanism is implemented to maintain stable and precise positioning of the UAV, thereby furnishing essential inputs to the inner loop. The inner loop is responsible to provide the UAV with desired torques. A control mechanism is employed to facilitate precise manipulation of the UAV's attitude.

Controller parameters are related to both inner (attitude) and outer (position) loops, and an estimation approach is applied to the inner loop to estimate controller gains. The controller gains are estimated using data provided from the UAV model and the underlying controller framework via a feedback technique. The model parameters, which are the drag effects, cause additional forces and torques, estimation of the model parameters improves the performance of the entire navigation control framework, and it is applied to the inner and outer loops. A new artificial neural network is designed and reward functions are used to facilitate the learning process. Figure 1.1 gives an overview of the adopted approach.

To address challenges posed by uncertainties in both controller gains and aerodynamic characteristics, an advanced learning approach is incorporated into the control framework. Specifically, the Deep Deterministic Policy Gradient (DDPG) algorithm is implemented to dynamically adjust and estimate these parameters in real-time. Furthermore, to ensure adaptability and robustness of the overall control system, an online learning algorithm is developed to continuously train the agent responsible for parameter estimation. This learning process enables the controller to autonomously adapt to changing environmental conditions and evolving system dynamics, enhancing the UAV's overall performance and reliability.

1.4. Summary of Contributions

This dissertation details a comprehensive and integrated model- and learning-based framework for modeling, navigation and control of multirotor UAVs. Specific contributions are listed next.

- A detailed mathematical model for quadrotor UAVs is derived following the N-E formulation, which includes gyroscopic and drag effects (induced, translational, profile, and parasitic drag), as well as motor dynamics.
- As opposed to already published research, the derived comprehensive and complete UAV mathematical model (not the simplified one) is used in the training phase of the adopted learning algorithms, and for controller design.
- A combination of offline and online learning is used and implemented in real-time that allows for continuous estimation of parameters and gains under different operational profiles.
- V&V of the proposed methodology is detailed via extensive simulations and simulated experiments.

1.5. Dissertation Outline

This dissertation is organized as follows: Chapter 2 provides a comprehensive literature review of offline and online learning-based algorithms for multirotor UAVs, along with their advantages and limitations, as well as information on what is being learned and how it is learned. Chapter 3 derives analytically the comprehensive quadrotor model and equations of motion using as baseline the conventional N-E formulation. Chapter 4 details the proposed navigation control architecture with a focus on the learning-based component. Comprehensive results are included in Chapter 5. Finally, discussion, conclusion, contribution, and future work are included in Chapter 6.

CHAPTER 2: LITERATURE REVIEW

2.1. Introduction

This Chapter reviews in detail articles on learning-based algorithms for multi-rotor UAVs. The review includes findings that may be found in [9], [10], [11], [12], and [13], and references therein.

Learning-based approaches for multirotor UAV navigation and control are generally divided into offline and online learning algorithms. A Google Scholar Search since 2015 returns the paper distribution shown in Figure 2.1, which depicts published papers in offline and online learning that deal with multirotor UAVs. Published research reflecting learning approaches is classified as machine learning (ML), deep learning (DL), and reinforcement learning (RL).

Literature review reveals that there exists considerable published research addressing learning algorithms for UAV control. The emphasis of already published surveys (5 review articles) is on developing and adopting machine learning, deep learning, or reinforcement learning algorithms.

The survey in [10] centers around deep learning methods applied to UAVs along with limitations and performance evaluation. Work in [11] emphasizes model-based reinforcement learning techniques applied to robotics, but also to UAVs. Applications are categorized into hovering and maneuvering tasks. Model-based and model-free reinforcement learning methods are discussed in terms of their advantages and limitations. Research in [12] explores machine learning algorithms for UAV autonomous control. The historical background and taxonomy of machine learning is presented

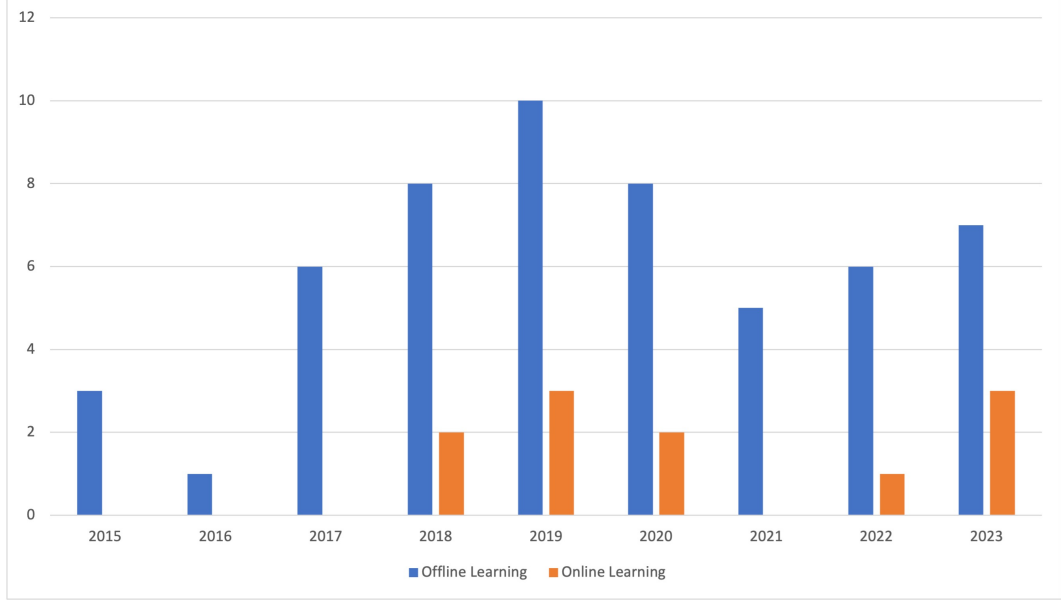


Figure 2.1: Publications of online and offline learning algorithms for control of multirotor UAVs since 2015 based on Google Scholar search.

alongside techniques for autonomous flight. The review in [13] investigates deep reinforcement learning (DRL) algorithms as applied to drones. DRL techniques are described, and limitations are explained. Two taxonomies are illustrated related to RL algorithms based on model-based and model-free approaches and DRL for UAV tasks. These taxonomies are depicted in Figs. 2.2 and 2.3, respectively. The survey in [14] presents learning algorithms with applications in robotics and multirotor UAVs. Safe decision-making under uncertainties is studied comprehensively combining control theory and RL algorithms.

Next, a set of (adopted) definitions is provided, which helps clarify concepts and classify learning algorithms as offline and online learning.

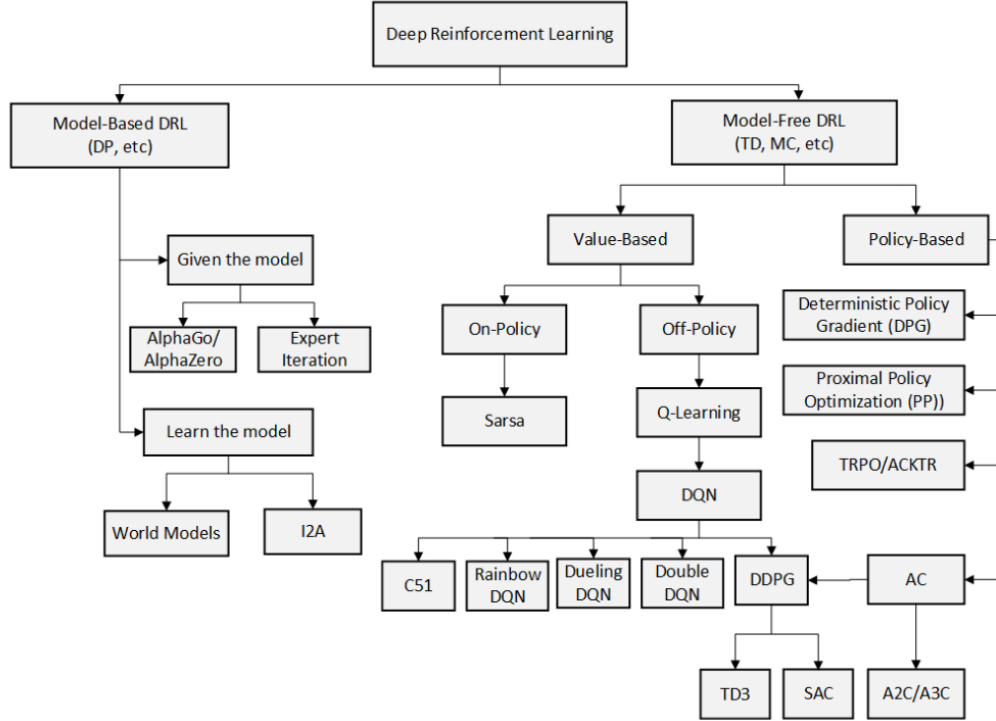


Figure 2.2: Taxonomy of RL algorithms [13].

2.2. Definitions:

The following definitions are adopted throughout this dissertation.

Reinforcement Learning: Reinforcement Learning, RL, is a machine learning technique in which an agent communicates with the environment to find the best action using the state space and a reward function. RL includes four main components; a policy, a reward signal, a value function, and optionally, an environment model [10], [15]. RL may be either online or offline. The general configuration of the offline RL and the online RL approaches are shown in Figures 2.4 and 2.5.

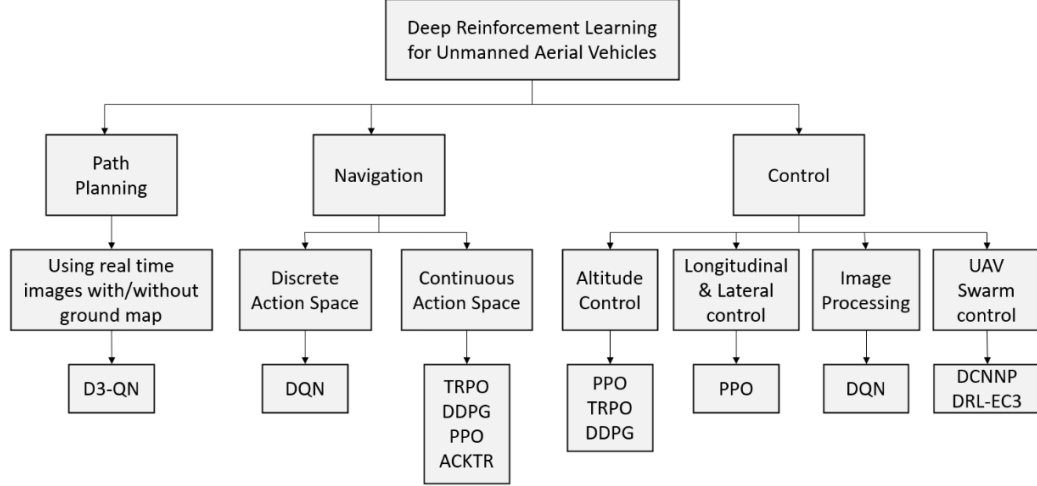


Figure 2.3: Taxonomy of DRL algorithms for UAV tasks [13].

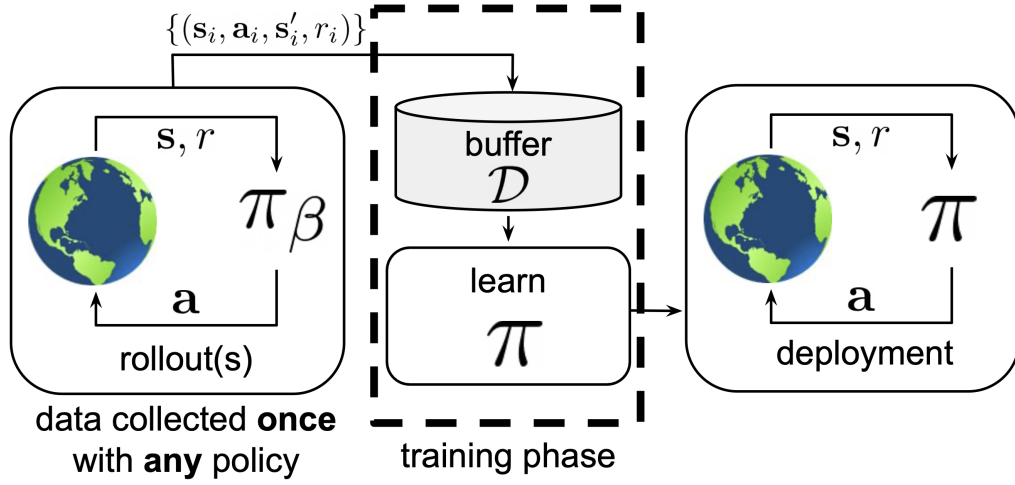


Figure 2.4: Offline reinforcement learning block diagram illustration [16].

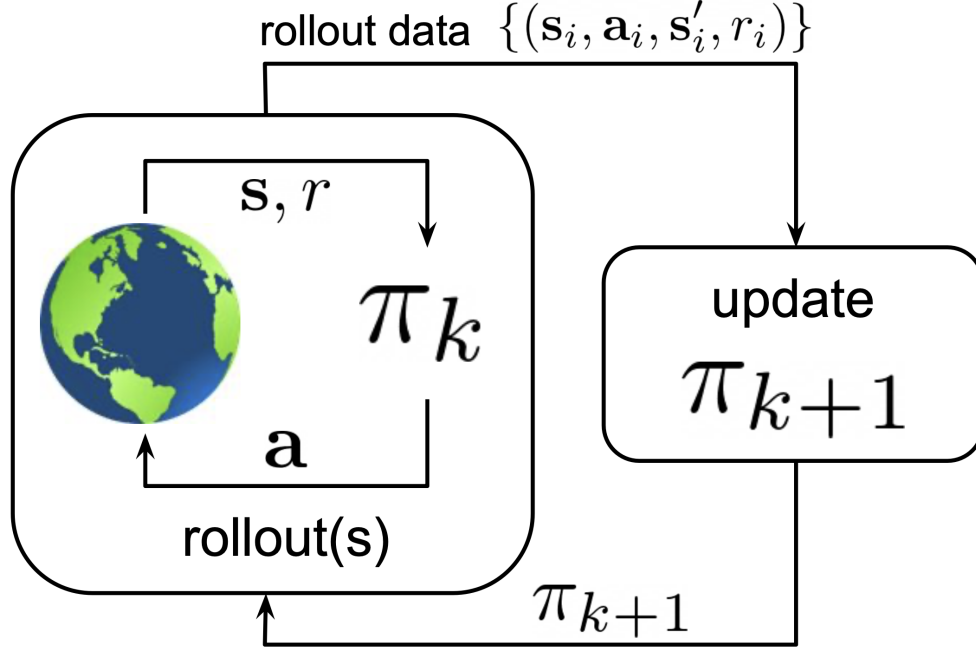


Figure 2.5: Online reinforcement learning block diagram illustration [16].

Policy: In the context of learning algorithms, a policy determines how the learning agent behaves at a given moment. A policy maps the discerned states of the environment to the actions that the agent should take when in those states [15]. The policy may reflect either on-policy or off-policy options.

On-policy: On-policy techniques relate to a policy that is used to make decisions. Policy Iteration, Value Iteration, Monte Carlo for On-Policy, and State-Action-Reward-State-Action (SARSA) algorithms are representative examples of on-policy algorithms [15].

Off-policy: Off-policy methods refer to a policy that is dissimilar to that used to produce data. Q-learning and Deep Deterministic Policy Gradient (DDPG) are examples of off-policy algorithms [15], see Figure 2.6 for the configuration of the off-policy RL algorithm. The agent experience is the input to a data buffer $\mathcal{D}$, which is

also called a replay buffer. Each new policy π_{k+1} is trained by utilizing the samples of all previous policies $\pi_0, \pi_1, \dots, \pi_k$ that are stored in $\mathcal{D}$.

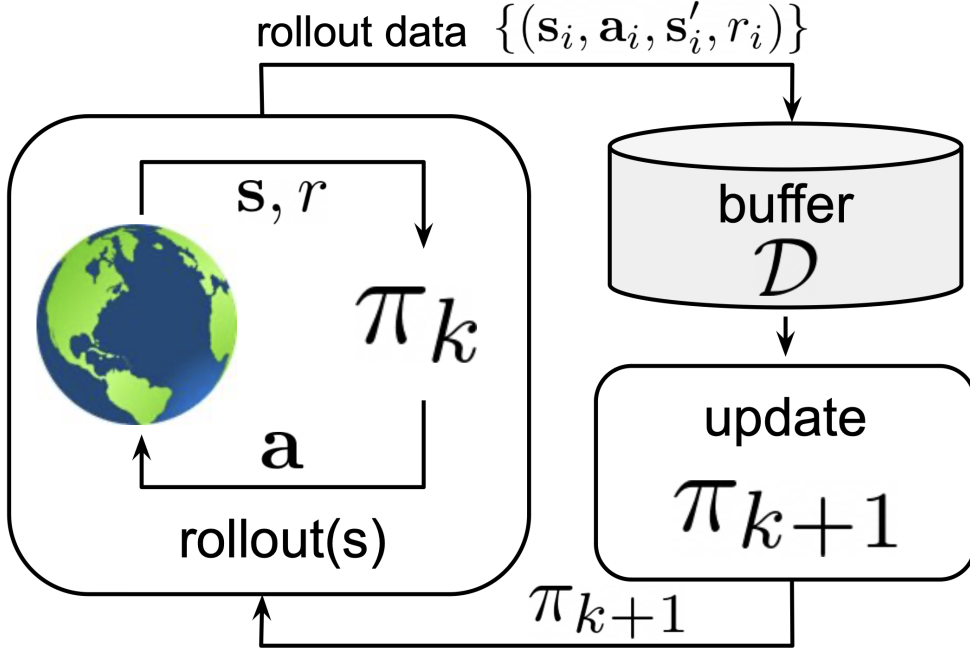


Figure 2.6: Off-policy RL algorithm configuration [16].

On-policy methods are generally less complicated than off-policy ones, and they are contemplated first. Since data is produced by a different policy in off-policy methods, they converge more slowly. Regardless, off-policy methods provide more powerful and general alternatives [15].

Reward signal: A reward signal is a target in a RL algorithm. The agent receives a single number from the environment in each time step, the reward. The only objective of the agent is to maximize the total reward over time. The reward signal provides an immediate sense of the current state and indicates what events are beneficial or detrimental to the agent [15].

Value Function: A value function determines what events are advantageous to the agent, long-term.

Environment Model: An environment model helps mimic and replicate the behavior of the environment. It allows for making inferences about how the environment will behave.

In what follows, the definitions and fundamental concepts of offline, online, supervised and unsupervised learning, machine and deep learning, are detailed.

Offline Learning: In offline learning, the learning algorithm trains an agent or artificial neural network (ANN). The agent interacts with the environment and it is updated during the training process. However, the agent is never updated after training is completed. Bartak and Vykovsky [17] and Edhah et al. [18] present representative examples of offline machine learning (ML) and deep learning (DL) algorithms, respectively. The studies of Xu et al. [19], Rodriguez et al. [20], and Yoo et al. [21] are representative examples of offline RL algorithms. Observing the configuration diagram of offline RL in Figure 2.4, a dataset ($\mathcal{D}$) is collected by the behavior policy (π_β) with the help of the states (s) and the reward function (r). A policy (π) is trained by using $\mathcal{D}$. The training process does not interact with the Markov Decision Process (MDP). The trained policy (π) is deployed to control the system. The policy (π) interacts with the environment using the states (s) and the reward function (r), and creates the action space (a).

Online Learning: In online learning, according to Hoi et al. [22], the learner keeps on learning and improves prediction to reach the best possible one by interacting with the environment as shown in Figure 2.5. A policy (π_k) creates the action space to interact with the environment using the states (s) and the reward function (r). Then, π_k is updated by using the roll-out data including states, actions, future states, and the reward function. After the updated policy (π_{k+1}) is determined, it is replaced with the current policy (π_k).

Note that for the purposes of this dissertation, the definition of online learning is extended to account for cases where the agent continues the learning process during operation, in real-time, even after completing the offline learning process, or without any basic learning.

This extension allows for 'anytime learning' while the underlying system continues to function, and accounts for the ability to modify the system model and also adapt its parameter values (time-varying systems).

Supervised Learning: In supervised learning, the learning algorithm learns from the classified and labeled dataset [10].

Unsupervised Learning: In unsupervised learning, the learning algorithm utilizes unlabeled data, which are collected from sensors, to learn the proposed task. Unsupervised learning techniques are widely used in RL [10].

Machine Learning: Machine learning, ML, is a component of Artificial Intelligence (AI), in which tasks are learned (or imitation of tasks is learned) from collected data [10].

Deep Learning: Deep learning, DL, is a category or subset of ML that involves the use of deep neural networks, DNNs, with input, hidden, and output layers to model and solve complex problems.

Moreover, RL techniques and methods are divided into model-based and model-free ones.

Model-based: In a model-based method, the agent predicts the future states and reward and also chooses the action that provides the highest expected reward [15]. Models and planning are used to solve RL problems.

Model-free: In a model-free method, the agent does not utilize the environment model but makes decisions by only using trial-and-error approaches [15].

The main difference between model-based and model-free methods is that the former relies on planning, while the latter relies on learning [15]. Next, offline learning techniques are discussed.

2.3. Offline Learning

In offline learning, the system may be trained either using collected and/or provided data (supervised learning), or, alternatively, by using feedback before its actual operation, without using any data (unsupervised learning). In this case, when operating in real-time, the agent, or the neural network (NN), is not updated nor affected by the environment.

Table 2.1 summarizes offline learning and RL approaches that have been applied to multirotor UAVs. The Table includes the publication year of the paper, the adopted or derived learning model, the application task/domain, as well as what is being learned.

Machine Learning

Most offline ML techniques applied to and used for multirotor UAVs consider an onboard monocular camera. ML-based approaches have been developed and adopted for navigation purposes, for stabilization, to track an object, to pass through waypoints with a desired velocity, and for landing purposes on stationary or dynamic targets. In addition, ML approaches are also used to tune and adjust controller parameters.

Bartak and Vykovsky [17] have combined computer vision, ML, and control theory techniques to develop a software tool for a UAV to track an object; the object is selected by a user who observes a series of video frames (images) and picks a specific object to be tracked by the multirotor. *P-N learning*, where P and N represent

Table 2.1: Offline Learning Papers

Year	Authors	Learning Model	Application - Task	What is being Learned
2015	Bartak et al. [17]	ML	Object tracking	How to detect an object
2015	Giusti et al. [23]	ML	Navigation	Image classification to determine direction
2018	Kaufmann et al. [24]	ML	Waypoint & desired velocity	How to detect an object
2021	Janousek et al. [25]	ML	Landing & flight planning	How to recognize an object
2023	Vladov et al. [26]	ML	Stabilization	How to adjust controller parameters
2015	Kim et al. [27]	DL	Navigation	Image classification to assist in flights
2017	Li et al. [28]	DL	Trajectory tracking	Control signals
2017	Smolyanskiy et al. [29]	DL	Navigation	View orientation and lateral offset
2018	Jung et al. [30]	DL	Navigation	How to detect the center of a gate
2018	Loquercio et al. [31]	DL	Navigation	How to adjust yaw angle, and probability of collision
2019	Edhah et al. [18]	DL	Hovering	How to determine propeller speed
2019	Mantegazza et al. [32]	DL	Ground target tracking	Image classification for control
2023	Cardenas et al. [33]	DL	Position control	How to determine the rotor speeds
2016	Imanberdiyev et al. [34]	RL	Navigation	How to select the moving direction
2017	Polvara et al. [35]	RL	Landing	How to detect a landmark and control vertical descent
2017	Choi et al. [36]	RL	Trajectory tracking	Control input
2017	Kahn et al. [37]	RL	Avoiding failure	Policy
2017	Hwangbo et al. [38]	RL	Stabilization	How to determine the rotor thrusts
2018	Xu et al. [19]	RL	Landing	How to determine the velocities of the UAV
2018	Lee et al. [39]	RL	Landing	How to determine the roll and pitch angles
2018	Vankadari et al. [40]	RL	Landing	How to determine the velocities of the UAV on the x and y axis
2018	Kersandt et al. [41]	RL	Navigation	How to select three actions: move forward, turn right, and turn left
2018	Pham et al. [42]	RL	Navigation	How to select the moving direction
2019	Rodriguez et al. [20]	RL	Landing	How to determine velocities of the UAV on x and y axes
2019	Liu et al. [43]	RL	Formation control	Optimal control law
2019	Lambert et al. [44]	RL	Hovering	The mean and variance of the change in states

Table 2.1: Offline Learning Papers (Continued).

Year	Authors	Learning Model	Application - Task	What is being Learned
2019	Manukyan et al. [45]	RL	Hovering	How to determine the rotor speeds
2019	Srivastava et al. [46]	RL	Target tracking	How to determine the velocities of the UAV on x, y, and z axes
2019	Wu et al. [47]	RL	Trajectory planning	How to select the moving direction
2019	Wang et al. [48]	RL	Navigation	How to determine the steering angle
2019	Zeng & Xu [49]	RL	Path Planning	How to select flight direction
2020	Yoo et al. [21]	RL	Trajectory tracking	How to adjust PD and LQR controllers gains
2020	Rubi et al. [50]	RL	Trajectory tracking	How to determine the yaw angle
2020	Pi et al. [51]	RL	Trajectory tracking	How to determine the rotor thrusts
2020	Zhao et al. [52]	RL	Formation control	How to solve Bellman equation
2020	Guerra et al. [53]	RL	Trajectory optimization	Control signal
2020	Li et al. [54]	RL	Target tracking	How to determine the angular velocity of yaw angle & Linear acceleration
2020	Kulkarni et al. [55]	RL	Navigation	How to select moving direction
2020	Hu & Wang [56]	RL	Speed optimization	How to determine the rotor speeds
2021	Kooi & Babuška [57]	RL	Landing	How to determine the total thrust, and roll and pitch angles
2021	Rubi et al. [58]	RL	Trajectory tracking	How to determine the yaw angle
2021	Bhan et al. [59]	RL	Avoiding failure	How to adjust the gains of PD position controller
2021	Li et al. [60]	RL	Trajectory planning	How to obtain the parameter vector of the approximate value function
2022	Jiang et al. [61]	RL	Landing	How to determine the velocity of the UAV on x and y axes
2022	Abo et al. [62]	RL	Landing	How to determine the roll, pitch, and yaw angles and velocity of the UAV on z axis
2022	Panetsos et al. [63]	RL	Payload transportation	How to obtain the reference Euler angles & Velocity on z axis
2022	Ye et al. [64]	RL	Navigation	How to select the moving direction and determine the velocity
2022	Wang & Ye [65]	RL	Trajectory tracking	How to determine the pitch and roll torques
2022	Farsi & Liu [66]	RL	Hovering	How to determine the rotor speeds
2023	Xia et al. [67]	RL	Landing	How to obtain the force and torque command

Table 2.1: Offline Learning Papers (Continued).

Year	Authors	Learning Model	Application - Task	What is being Learned
2023	Ma et al. [68]	RL	Trajectory tracking	How to determine the rotor speeds
2023	Castro et al. [69]	RL	Path Planning	How to find optimized routes for navigation
2023	Mitakidis et al. [70]	RL	Target Tracking	How to obtain the roll, pitch and yaw actions
2023	Shurrab et al. [71]	RL	Target localization	How to determine the linear velocity and yaw angle
2024	Huang et al. [72]	RL	Path planning	How to adjust the thrust of each rotors for all quadrotors
2024	Liu et al. [73]	RL	Fault tolerance	How to determine the PWM input for each rotor
2024	Yi et al. [74]	RL	Target tracking	How to obtain the approximate optimal control

positive and negative learning, respectively, is used by a Tracking-Learning-Detection (TLD) algorithm. The Lucas-Kanade tracker is implemented in the tracking phase, and a cascaded classification algorithm that includes an ML technique helps detect the object. A cascaded classification algorithm that consists of a patch variance classifier, an ensemble classifier, and a nearest neighbor classifier is also used. A simple RL algorithm decides the forward or backward speed of the multirotor by using a scaled size of the object. The yaw angle of the multirotor, used to follow the object, is provided to a Proportional-Integral-Derivative (PID) controller as input, to determine flight direction.

Janousek et al. [25] have developed a method to accurately guide an autonomous UAV to land in a specific area that is labeled as the 'ground object'. The landing area includes a QR code, which, after it is identified and recognized it provides specific instructions/commands to the UAV for landing. A NN is used to identify the landing area using the onboard UAV camera. The recognition process is done on a ground control station, GCS, which is a part of the overall UAV-GCS ensemble. The GCS includes a communication channel for command transmission (to and from the UAV). False detection of the landing area may occur (i.e., due to sunlight) thus, success

depends on how accurately the processed image determines the landing area, and not on the learning process itself. Regardless, when the UAV is within an 'acceptable flight altitude', a successful landing is accomplished.

Giusti et al. [23] have used a quadrotor with an onboard monocular camera to determine the path configuration and direction of forest or mountain trails. A single image is collected from the onboard camera. A DNN is trained using supervised learning to classify obtained images. Two parameters are defined, $\vec{v}$ for the direction of the camera's optical axis, and $\vec{t}$ for the direction of the trail. Based on the calculated angle α between $\vec{v}$ and $\vec{t}$ and the angle β that is 15° around $\vec{t}$, three actions are determined and classified as *Turn Left (TL)*, *Go Straight (GS)*, and *Turn Right (TR)*, represented as

- **TL** if $-90^\circ < \alpha < -\beta$
- **GS** if $-\beta \leq \alpha < +\beta$
- **TR** if $+\beta \leq \alpha < +90^\circ$

These choices consider that the onboard camera centers and focuses on the motion direction.

For performance evaluation and comparison, three alternatives are considered: learning using a Saliency-based model, learning following the method discussed in Santana et al. [75], and by using two human observers who are asked to make one of the three previously mentioned decisions. The accuracy of the DNN is 85.2%; the accuracy of the Saliency-based model is 52.3%; the accuracy of the model in [75] is 36.5%. The accuracy of Human1 is slightly better than the accuracy of DNN, 86.5%; Human2 has 82% accuracy, which is lower than the accuracy of the DNN. This methodology has been tested experimentally and has produced successful results.

Kaufmann et al. [24] have focused on the problem of autonomous, vision-based drone racing in dynamic environments with an emphasis in path planning and con-

trol. A convolutional neural network (CNN) is used to detect the location of the waypoints from raw images and to decide about the speed to pass through the gates. The planner utilizes this information to design a short minimum jerk trajectory to reach the targeted waypoints. This technique is tested via simulations and in real environments. Comparisons with other navigation approaches and professional human drone pilots are made. It is shown that this method completes the track slower than human pilots do, but with a higher success rate. The success rate is also much higher compared to using visual-inertial odometry (VIO).

Vladov et al. [26] have studied the UAV stabilization problem. They adjust PID controller parameters using a recurrent multilayer perceptron (RMLP) method to stabilize the UAV attitude angles. The determined error is the input to a NN. Instead of using a constant training rate, an adaptive training rate is implemented to overcome slow convergence in the learning part and to avoid trapping in a local minimum during the learning phase. Results show that this method has a lower attitude error when compared to the RMLP method with a constant training rate and when using only an ANN.

Deep Learning

DL algorithms (discussed in 8 papers) that have been applied to multirotor UAVs focus on navigation and control, hovering, ground target tracking, and trajectory tracking.

In Edhah et al. [18], a DNN has been used to control UAV altitude and hover. The standard feedforward, greedy layer-wise, and Long Short-Term Memory (LSTM) methods are evaluated and compared. The controller outputs, position and speed errors, are collected every 1 ms, and then used by a supervised learning technique to train the DNN. After training, the trained DNN controller (and related parameters) is replaced by a Linearâ€“Quadratic Regulator (LQR) controller. To overcome a slight offset in the output signal that results in a small error (in the system response), a

proportional corrector is added in parallel to the DNN controller to recover the error in the DNN output signal. Best results are received when using the greedy layer-wise method.

Mantegazza et al. [32] have presented three different approaches and models to track a moving user. In the first model, the ResNet [76] architecture (a CNN) is utilized [31]. Red-green-blue (RGB) images captured from 14 different people are used as input, providing x, y, and z positions as output. The second model follows the same structure, but velocities on the x, and y axes are provided as additional inputs. These additional inputs skip ResNet, but they are concatenated to the output of the NN. The outputs are control variables corresponding to four moving directions, up, down, left, and right. In the third model, a simple multilayer perceptron is utilized to map the quadrotor position on three axes and velocities on the x and y axes to control variables. The Mean Absolute Error (MAE) approach is deployed as a loss function during training of all three models. The first and second models use a simple baseline controller function. The last approach uses a combination of the first and third models.

Li et al. [28] have studied trajectory tracking without any adaptation, but they have considered quadrotor stabilization and robustness in the presence of disturbances. A DNN is trained with labeled training examples using a standard feedforward method. The DNN uses the quadrotor desired trajectory and current states of position and translational velocities (on each axis), Euler angles, angular velocities, and acceleration on the z-axis. The quadrotor reference states are provided as the DNN output. The trained DNN is placed in front of a controller, and errors between current and desired states are used as inputs to a PID controller. Results show that the DNN with current state feedback is more efficient than the DNN without current state feedback. However, the DNN using future desired state feedback provides better performance.

Kim and Chen [27], Jung et al. [30], Loquercio et al. [31], and Smolyanskiy et al. [29] have centered around the quadrotor navigation task using DL techniques.

Kim and Chen [27] have developed an autonomous indoor navigation system for a quadrotor to find a specific item using a single onboard camera. Six flight commands are used, Move Forward, Move Right, Move Left, Spin Right, Spin Left, and Stop. To establish a dataset, an expert pilot flies the quadrotor in seven different locations; images are collected from the UAV that are based on, corresponding to, (specific) flight commands. A CNN that is a modified CaffeNet model [77] is trained with the established dataset. This modification allows for faster training. During indoor flights, obtained images are classified by the trained NN, and based on image classification, specific control commands are issued to the quadrotor.

Jung et al. [30] have developed a CNN to identify accurately the center of gates during indoor drone racing. ADRNet is built and trained using the Caffe library. To build the ADRNet, the AlexNet used in [78] and applied instead of the VGG-16 employed in [79] are adopted. In [80], a convolutional layer is added among the fully connected layers of AlexNet instead of the fc6 and fc7 layers, while the fc8 layer is removed. Thus, a shorter inference time is required compared to the VGG-16-based Single-Shot-Detection (SSD) approach. ADRNet detects the center of the gate, and this information is forwarded to a Line-Of-Sight (LOS) guidance algorithm that issues specific flight control commands. Performance of three Single-Shot Detection (SSD) models, the VGG-16-based SSD, the AlexNet-based SSD, and the ADRNet are compared. ADRNet is the fastest model to detect the center of the gate.

Different from the traditional map-localize-plan methods, Loquercio et al. [31] have applied a data-driven approach to overcome UAV challenges encountered when navigating in unstructured and dynamic environments. A CNN, called DroNet, which is used to navigate quadrotors through the streets of a city, is proposed. Collecting data within a city, in urban areas, to train UAVs, is dangerous for both, pedestrians

and vehicles, even if an expert pilot flies the quadrotor. Therefore, publicly available datasets from Udacity’s project are used to learn the steering angles. The dataset includes over 70,000 driving car images collected and classified through six experiments. Five experiments are for training and one for testing. A collision probability dataset is also developed for different areas of a city by placing a GoPro camera on the handlebars of a bicycle. UAV control is achieved via commands issued based on the output of DroNet. The collision probability is used to determine the quadrotor forward velocity. The desired yaw angle in the range $[-\frac{\pi}{2}, \frac{\pi}{2}]$ is determined by using predicted scaled steering in the range $[-1, 1]$. DroNet has worked successfully to avoid unexpected situations and obstacles, predicting the collision probability and the desired steering angle. The quadrotor learned to fly in several environments, and in indoor environments such as parking lots and corridors.

Smolyanskiy et al. [29] have focused on autonomously navigating a micro aerial vehicle (MAV) in unstructured outdoor environments. A DNN called TrailNet is developed and used to estimate the view orientation and lateral offset of the MAV with respect to the center of a trail. A DNN-based controller provides for a stable flight and avoids overconfident maneuvers by utilizing a loss function that includes both label smoothing and an entropy reward. The MAV includes two vision modules, a second DNN and a visual odometry component that is called direct sparse odometry (DSO). The second DNN helps detect the objects in the environment; the DSO estimates the depth to compute a pseudo-colored depth map. Their combination with TrailNet provides an autonomous flight controller functioning in unstructured environments. ResNet-18, SqueezeNet (a miniature version of AlexNet), the DNN architecture in [23], and TrailNet are compared considering autonomous long-distance flight ability, prediction accuracy, computational efficiency, and power efficiency. Only Trailnet is 100% autonomous; SqueezeNet and mini AlexNet are the closest to TrailNet with 98% and 97%, respectively. The least autonomous architecture is the DNN in [23]

with 80%. Software modules run simultaneously in real-time, and the quadcopter successfully flies in unstructured environments.

Cardenas et al. [33] have developed a DNN-based flight position controller using a supervised DL technique. A dataset that includes position, velocity, acceleration, and motor output signals for different trajectories is created by using a PID flight controller. Five different NN architectures (from the literature) are utilized to learn the rotor speeds using a dataset, and their performance is compared. The five developed architectures are: *i*) ANN, *ii*) ANN Feedback, *iii*) LSTM, *iv*) LSTM Layers interleaved with convolutional 1D layers (LSTMCNN), and *v*) Convolutional 1D Layers cascaded with LSTM layers (CLSTM). A comparative study shows that LSTMCNN gives the best performance as a DNN-based flight position controller. LSTMCNN performance is checked against the PID position controller, and it is shown that LSTMCNN has a wider operational range than the PID position controller.

Reinforcement Learning

Offline RL has been extensively applied to multirotor UAVs. The literature review reveals 42 papers that focus on 13 different tasks, which include trajectory tracking, landing, navigation, formation control, flight control, and hovering.

In general, the RL framework uses an agent that is trained through trial-and-error to decide on an action that maximizes a long-term benefit. RL is described by a Markov Decision Process (MDP). The agent-environment interaction in an MDP is illustrated in Fig. 2.7, where, Agent, Environment, and action represent in engineering terms the controller, the controlled system, and the control signal, respectively [15].

RL algorithms are classified according to whether they are model-based or model-free, on-policy or off-policy, value function-based or policy search-based, or according to whether they are derived for planning or learning purposes. In what

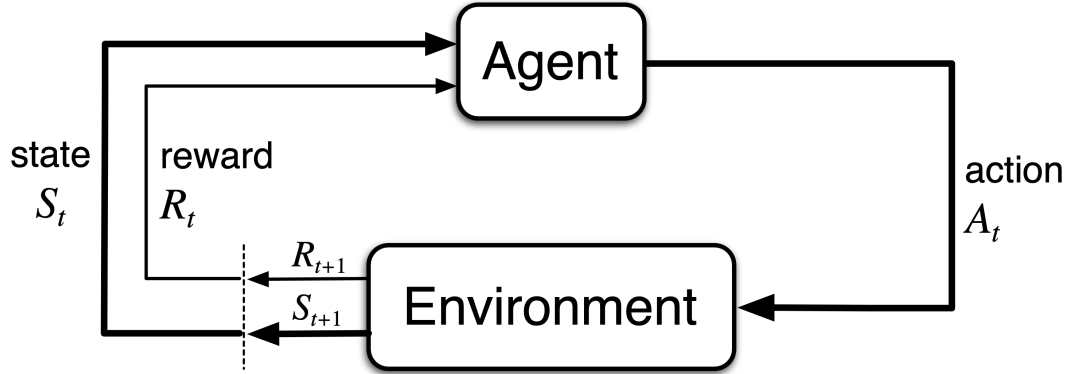


Figure 2.7: Block diagram and interaction between agent and environment [15].

Table 2.2: Classification of Reinforcement Learning (Offline)

Methods	Algorithms	Papers
Value Function-based	Q-learning	Guerra et al. [53], Pham et al. [42], Kulkarni et al. [55], Abo et al. [62], Zeng & Xu [49]
	DQN	Xu et al. [19], Polvara et al. [35], Castro et al. [69], Shurrab et al. [71], Wu et al. [47], Kersandt et al. [41]
	LSPI	Vankadari et al. [40], Lee et al. [39], Srivastava et al. [46]
	IRL	Choi et al. [36]
	Others	Imanberdiyev et al. [34], Ye et al. [64], Farsi & Liu [66], Li et al. [60], Xia et al. [67]
Policy Search-based	PPO	Kooi & BabuÅıka [57], Bhan et al. [59], Huang et al. [72]
	TRPO	Manukyan et al. [45]
	PILCO	Yoo et al. [21]
	PLATO	Kahn et al. [37]
	Others	Hu & Wang [56], Lambert et al. [44]
Actor-Critic	DDPG	Jiang & Song [61], Rodriguez et al. [20], Rubi et al. [50], Rubi et al. [58], Ma et al. [68], Mitakidis et al. [70]
	TD3	Jiang & Song [61], Kooi & BabuÅıka [57], Li et al. [54], Panetsos et al. [63]
	SAC	Jiang & Song [61], Kooi & BabuÅıka [57],
	Fast-RDPG	Wang et al. [48]
	DeFRA	Li et al. [81]
	CdRL	Wang & Ye [65]
	Others	Pi et al. [51], Hwangbo et al. [38]

follows, the classification is in terms of value function-based, policy search-based, or actor-critic. Table 2.2 offers a summary of the different variations of RL algorithms.

Value Function-Based Algorithms: Value function-based methods use state-value and action-state functions that are presented in (2.1) and (2.2), respectively, shown next

$$v_{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \text{ for all } s \in S \quad (2.1)$$

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right] \quad (2.2)$$

where $v_{\pi}(s)$ denotes the *value function* for policy π at state s while $q_{\pi}(s, a)$ represents the *action-value function* for policy π at state s and action a . $\mathbb{E}_{\pi}[\cdot]$ denotes the expected value under policy π . $\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ is the sum of discounted future rewards starting from time t in state s and represents the expected *discounted return*, and γ is the *discount rate*, $0 \leq \gamma \leq 1$, here. S_t and A_t represent the state and action at time t , respectively [15][82][83].

The discount rate γ plays a critical role in calculating the present value of future rewards. If $\gamma < 1$, the infinite sum has a finite value when the reward sequence, R_k , is bounded. If $\gamma = 0$, the agent cannot calculate future rewards; it can only calculate the immediate reward ($\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = 0^0 R_{t+1} + 0^1 R_{t+2} + \dots = R_{t+1}$). So the agent learns how to choose the action A_t to maximize R_{t+1} . If γ approaches 1, the future rewards are highlighted in the expected discount return, that is, the agent behaves more farsighted. For example, in [15] the discount factor is chosen close to 1. Li et al. [60], Hu and Wang [56] have chosen the value of 0.9, and Castro et al. [69] and Panetsos et al [63] have chosen the discount factor value to be 0.99.

Value function-based algorithms consist of variance algorithms, such as Dynamic Programming (DP), Monte Carlo (MC), and Temporal-Difference (TD). Two most popular DP methods, policy iteration and value iteration, benefit from policy evaluation and policy improvement. The MC method is also based on policy evaluation and policy improvement, but unlike the DP method, an alternative policy evaluation process is utilized. The policy evaluation in the DP method employs a bootstrapping technique, while sampling and average return techniques are applied in the MC method. The TD method is created by combining the DP and MC methods applying sampling and bootstrapping [15][82][84].

TD has been extensively applied in control algorithms for multirotors executing diverse tasks like landing, navigation, obstacle avoidance, path planning, and trajectory optimization. Q-learning and Deep Q-Networks (DQN) stand out as commonly employed RL algorithms within the TD framework, particularly in value function-based algorithms, see Guerra et al. [53], Pham et al. [42], and Polvara et al. [35].

Xu et al. [19] have applied an end-to-end control scheme that includes a DNN and a double DQN algorithm for quadrotor landing on a stable platform. The output of the underlying Deep Reinforcement Learning (DRL) model is the quadrotor speed in x and y - the velocity in the z direction is not controlled, it is considered fixed; this makes the problem easier. After testing, the improved DQN method produces good results on autonomous landing.

Imanberdiyev et al. [34] have used a model-based RL algorithm on a quadrotor to create an efficient path to reach a destination by considering its battery life. The agent uses three states, position in x and y, and the battery level. The agent follows one of eight possible actions, moving on the x-y plane, and learning the moving direction. The direction action is converted to trajectory commands that are executed using position control. In model-based RL methods, there is a limited number of actions the agent learns to create a sufficiently accurate environment model, as opposed to model-free RL methods. Model-based RL algorithms are not suitable for real-time systems since the planning and model learning aspects are computationally expensive. However, in [34] a parallel architecture is used, called TEXPLORE. Thus, it is possible to take action fast enough based on the current policy - there is no need to wait for the planning and model update. Simulation results illustrate that the approach has the ability to learn and to perform well after a few iterations and to also perform actions in real-time. Performance is compared with Q-learning algorithms. In 150 episodes, while there is no significant change in the average reward in Q-learning, the average

reward of TEXPLORE dramatically increases after the 25th episode. TEXPLORE obtains significantly more rewards than Q-learning in each episode.

In [49], the path design problem for a cellular-connected UAV has been handled to reduce mission completion time. A new RL-based UAV path planning algorithm is derived and TD is applied to learn directly the state-value function. A linear function is added to the algorithm with tile coding. Function approximation has two advantages over a table-based RL. It learns the parameter vector that has a lower dimension than the state vector, instead of storing and updating the value function for all states. It also allows for generalization. Tile coding is used to build the feature vector. The parameter vector may be updated to minimize its mean squared error based on a stochastic semi-gradient method with a linear approximation for each state-reward-nextState transition observed by the agent. It is shown that TD with a tile coding algorithm overcomes problems with cellular networks in complex urban environments of size 2 km $\times$ 2 km with high-rise buildings. Also, accumulated rewards from the TD and the TD with tile coding learning algorithms are almost identical, but tile coding provides faster convergence. When tested, the UAV reaches the desired location without running into the coverage holes of cellular networks.

The approach discussed in [35] has used two DQNs. One is utilized for landmark detection, the other is used to control the UAV vertical descent. A hierarchy representing sub-policies is applied to the DQNs to reach decisions during the different navigation phases. The DQNs can autonomously decide about the next state. However, the hierarchy decreases the sophistication of the task decision. Algorithm performance is compared with an augmented reality (AR) tracker algorithm and with human pilots. The proposed algorithm is faster than a human pilot when landing on a marked pad but also more robust than the AR tracker in finding the marker. Complete details of [35] may be found in [85].

Ye et al. [64] have developed a DRL-based control algorithm to navigate a UAV swarm around an unexplored environment under partial observations. This may be accomplished by using GAT-based FANET (GAT-FANET), which is a combination of the flying Ad-hoc network (FANET) and the graph attention network (GAT). Partial observations lead to loss of information. Thus, a network architecture named Deep Recurrent Graph Network (DRGN) is developed and combined with GAT-FANET to collect environment spatial information, and use previous information from memory via a gated recurrent unit (GRU). A maximum-entropy RL algorithm, called soft deep recurrent graph network (SDRGN), is developed, which is a multi-agent deep RL algorithm. It learns a DRGN-based stochastic policy with a soft Bellman function. The performance of the DRGN (a deterministic model) and the SDRGN are compared with DQN, multi-actor attention critic (MAAC), CommNet, and graph convolutional RL (DGN). In a partially observable environment, the stochastic policy approach is more robust than the deterministic policy one. Also, GAT-FENAT provides an advantage because of its memory unit. When the number of UAVs increases, more information is required from the GAT-FANET, and this reduces dependency on the memory unit. Results [64] show that policies based on GAT-FANET provide better performance on coverage than other policies. It is observed that graph-based communication improves performance in cooperative exploration and path planning, too. The SDRGN algorithm has lower energy consumption than DRGN, but DQN has the lowest energy consumption when compared with DRL methods. SDRGN and DRGN performance increases linearly with the number of UAVs. SDRGN shows better performance than DRL methods - this verifies that it has better transferability. Consequently, overall, SDRGN has better performance, scalability, transferability, robustness, and interpretability than other DRL methods.

Abo et al. [62] have solved the problem of UAV landing on a dynamic platform taking advantage of Q-learning. Two types of adaptive multi-level quantization (AMLQ) are used; AMLQ 4A with 4 actions and AMLQ 5A with 5 actions, and

then compared with a PID controller. The PID position magnitude errors in x and y are higher than the corresponding AMLQ errors, while the oscillation in the AMLQ models is higher than in the PID controller. The developed AMLQ reduces the error on the targeted landing platform. This solution provides faster training and allows for knowledge representation without the need of a DNN.

Path planning is effectively used in several areas that include precision agriculture. Castro et al. [69] have worked on adaptive path planning using DRL to inspect insect traps on olive trees. The proposed path planning algorithm includes two parts, the rapidly-exploring random tree (RRT) algorithm and a DQN algorithm. The former searches for path options, while the latter performs optimized route planning integrating environment changes in real-time; however, the training process of DQN is completed offline. Simulation runs are done in an area of 300 m^2 with 10 dynamic objects; the UAV is provided with a safe route determined by the proposed approach, and it arrives at the insect traps to take their picture.

Shurrah et al. [71] have studied the target localization problem and have proposed a Q-learning-based data-driven method in which a DQN algorithm helps overcome dimensionality challenges. Data measurements, from previous and the current step, the previous action, and the direction of the nearest boundary of the UAV compose the state space. The action space includes the UAV linear velocity and the yaw angle that determines flight direction. This approach is compared with the traditional uniform search method and the gradient descent-based ML technique; it returns better results in terms of localization and traveled distance.

Guera et al. [53] have emphasized detection and mapping for trajectory optimization. For detection, the aim is to minimize 'wrong detection', and for mapping, the aim is to minimize the uncertainty related to estimating the unknown environment map. The proposed MDP-based RL algorithm, inspired by Q-learning, consists of state and control estimations. The states are the UAV position depending on ac-

tions, a binary parameter that shows the presence or absence of a signal source in the environment, and the states of each cell. The action space includes the control signal to move the UAV from one cell to another in the grid (environment) map. Numerical results show that this technique provides a high probability of target detection and improves the capabilities of map exploration.

Pham et al. [42] have handled the UAV navigation problem using Q-learning. The navigation problem is formulated using a discretized state space within a bounded environment. The algorithm learns the action, which is the UAV moving direction in the described environment. The state space includes the distance between the UAV and the target position, and the distance to the nearest obstacle in North, South, West, or East directions. UAV navigation following the shortest path is demonstrated.

Kulkarni et al. [55] have also used Q-learning for navigation purposes. The objective is to determine the location of a victim by using a RF signal emitted from smart devices. The transmitted signal reaches the agent and according to the received signal strength (RSS), the agent learns to choose one of eight directions separated by 45 degrees on the x-y plane. For mapping, a grid system is utilized, and each state label is correlated to a particular RSS value (two adjacent grids in the map have different RSS values). Each location on the map has a unique state. The ϵ -greedy approach provides an action to the UAV, and each episode or iteration is completed when the RSS value of the grid is determined to be greater than -21dBm - this value means that the distance from the victim is less than 2 meters. The proposed approach is tested for different starting positions on different floor plans, demonstrating that the UAV successfully reaches the victim's position.

Choi et al. [36] have trained a multirotor UAV by mimicking the control performance of an expert pilot. A pilot collects data from several actual flights. Then, a hidden Markov model (HMM) and dynamic time warping (DTW) are utilized to create the trajectory. Inverse RL is used to learn the hidden reward function and use

it to design a controller for trajectory following. Simulations and experiments show successful results.

Wu et al. [47] have worked on the general task of 'object finding', for example, in rescue missions. A DQN algorithm is used for trajectory planning. The elimination of the loop storm effect that reflects the current sequence in MDP is repeated and it does not cause a punishment in continued actions. The Odor Storm that is caused by not reaching the highest reward value when the agent gets closer to the target increases the convergence speed of the training process. It is shown that the break loop storm technique and the odor effect reduce the training process time.

In Kersandt et al. [41] a DNN has been trained with a DRL algorithm to control a fully autonomous quadcopter that is equipped with a stereo-vision camera to avoid obstacles. Three different DRL algorithms, DQN, Double DQN (DDQN), and Dueling DDQN are applied to the system. The average performance of each algorithm with respect to rewards is 33, 120, and 116, respectively; they all are below human performance. The results of applying DDQN and Dueling techniques show that the quadrotor reaches the target with 80% success.

Liu et al. [43] and Zhao et al. [52] have used RL for formation control. In [43], a value function-based RL control algorithm is applied to leader-follower quadrotors to tackle the attitude synchronization problem. The output of each quadrotor is synchronized with the output of the leader quadrotor by the designed control system. In [52], the aim is to solve the model-free robust optimal formation control problem by utilizing off-policy value function-based algorithms. The algorithms are trained for robust optimal position and attitude controllers by using the input and output data of the quadrotors. Theoretical analysis and simulation results match, and the robust formation control method works effectively.

Vankadari et al. [40] and Lee et al. [39] have worked on the landing task using a Least Square Policy Iteration (LSPI) algorithm that is considered a form of approximate dynamic programming (ADP). Srivastava et al. [46] and Li et al. [60] have applied a LSPI algorithm in multirotor UAVs for target tracking and trajectory planning, respectively. ADP is used to solve problems with large state or action spaces. ADP approximates the value function or policy with function approximation techniques, since storing values for every state-action pair is not practical.

In [40], a LSPI algorithm has been used to study the landing problem. A RL algorithm estimates quadrotor control velocities using instantaneous position and velocity errors. The optimal value function of any policy is determined by solving the Bellman equation as applied to a linear system. In the RL algorithm, the LSPI method forecasts the value function parameterizing it into basis functions instead of calculating an optimal value function. The RL algorithm converges quickly and learns how to minimize the tracking error for a given set point. Different waypoints are used to train the algorithm for landing. The method can also be used in noisy environments, effectively. Simulations and real environment results demonstrate applicability of the approach.

Research in [66] has provided a low-level control approach for a quadrotor by implementing a structured online learning-based algorithm (SOL) [86] to fly and keep the hovering position at a desired altitude. The learning procedure consists of two stages; the quadrotor is first flown with almost equal pulse-width modulation (PWM) values for each rotor; these values are collected to create an initial model. Then, learning in a closed-loop form is applied using the initial model. Before applying closed-loop learning, three pre-run flights are completed. 634 samples are collected in 68 seconds of flying. The state samples are determined at each time step in the control loop, and then the system model is updated using an RLS algorithm. After determining the updated model, a value function (needed to find the control value

for the next step) is updated. The quadrotor is autonomously controlled. This online learning control approach successfully reaches the desired position and keeps the quadrotor hovering.

In [39] a trained NN has been adopted for guidance in a simulation environment. A quadrotor with a PID controller has been used, which has onboard a ground-looking camera. The camera provides pixel deviation of the targeted landing platform from an image frame, and a laser rangefinder that procures the altitude information. During training, the NN is trained to learn how to control the UAV attitude. In simulation studies, the UAV reaches the proposed landing location. In experiments, the AI pilot is turned off below the altitude of 1.5 m, but the AI pilot can land at the targeted location using a vision sensor. Trajectories are not smooth because the landing location in the image is not accurately determined due to oscillations, because of image processing errors in the actor NN, because signal transmission creates a total delay of 200 ms, and because of disturbances in real-world environments.

Three target tracking approaches that deserve attention are the Image-based (IBVS), Position-based (PBVS), and Direct Visual Servoing approaches. In Kanelakakis and Nikolakopoulos [87], IBVS has been found to be the more effective approach for target tracking since it tackles directly the control problem in the image space; it also has better robustness when it comes to camera calibration and to depth estimation errors. Srivastava et al. [46] track a maneuvering target using only vision-based feedback, IBVS. However, tracking is difficult when using only monocular vision without depth measurements. This deficiency is eliminated by a RL technique where optimal control policies are learned by LSPI to track the target quadrotor. Two different basis functions (with and without velocity basis), and four types of reward functions (only exponential reward, quadratic reward function without velocity control, quadratic reward function with velocity control, asymmetric reward function) are de-

scribed in [46]. The basis function with velocity basis shows better performance than the basis function without velocity basis.

In [60], the objective has been to solve the problem of cable-suspended load transportation utilizing three quadrotors. The trajectory planning method is based on a value-function approximation algorithm with the aim to reach the final position as fast as possible keeping the load stable. This method includes two processes, trajectory planning and tracking. The trajectory planning process consists of parameter learning and trajectory generation. Training and learning help determine the parameter vector of the approximate value function (parameters learning part). In the trajectory generation phase, the value function is approximated by using the learned parameters in the former stage, and the flight trajectory is determined via a greedy strategy. The results effectiveness of the load trajectory and the physical effect on the quadrotor flight are checked based on the trajectory tracking process. The quadrotors are independent; in the trajectory tracking phase, positions and attitudes are controlled with a hierarchical control scheme using PID controllers (transmitting the position of the load to the controller of the quadrotor). Results show that the actual value function is successfully estimated. Also, the value function confirms that the proposed algorithm works effectively.

Xia et al. [67] have used a RL control method for autonomous landing on a dynamic target. Unlike other studies, position and orientation constraints for safe and accurate landing are described. Adaptive learning and a cascaded dynamic estimator are utilized to create a robust RL control algorithm. In the adaptive learning part, the critic network weight is formulated and calculated in an adaptive way. Also, the stability of the closed-loop system is analyzed.

2.3.1. Policy Search-based Algorithms:

Value function-based methods calculate the value of an agent’s every possible action to choose the one based on the best value. The probability distribution over

Table 2.3: Comparison of value function-based and policy search-based methods

Value Function-Based	Policy Search-Based
Indirect policy optimization	Direct policy optimization
Generally off-policy	On-policy
Simpler algorithm	Complex algorithm
Computationally expensive	Computationally inexpensive
More iteration to converge	Less iteration to converge

all available actions plays a key role in policy-based methods, and for the agent to decide the action each time step. A comparison of value function-based and policy search-based algorithms is provided in Table 2.3.

Kooi and Babuška [57] have developed an approach using deep RL to land a quadrotor on an inclined surface, autonomously. Proximal Policy Optimization (PPO), Twin-Delay Deep Deterministic Gradient (TD3), and Soft Action-Critic (SAC) algorithms are applied to solve this problem. The TD3 and SAC algorithms trained successfully the set-point tracking policy network, but the PPO algorithm was trained in a shorter time and provided a better performance on the final policy. Trained policies may be implemented in real-time.

Hu and Wang [56] have utilized an advanced PPO RL algorithm to find the optimal stochastic control strategy for a quadrotor speed. During training, an actor and a critic NN are used. They have the same nine-dimensional state vector (Euler angles, Euler angle derivatives, errors between expected and current velocities after integration on the x, y, and z axes). An integral compensator is applied to both NNs to improve speed-tracking accuracy and robustness. The learning approach includes online and offline components. In the offline learning phase, a flight control strategy is learned using a simplified quadrotor model, which is continuously optimized in the online learning phase. In offline learning, the critic NN evaluates the current action to determine an advantage value choosing a higher learning rate to improve evaluation. In online learning, the action NN is composed of four policy trainable sub-networks.

The state vector is used as input to the four sub-networks; their outputs are the mean and variance of the corresponding four Gaussian distributions, each normalized to $[0, 1]$. Parameters of the four policy sub-networks are also used in the old policy networks that are untrainable. The old policy sub-network parameters are fixed. The four policy sub-networks in the action NN are trained to produce new actions in the next batch. When applying new actions to the quadrotor, new states are recorded in a buffer. After the integration and compensation process, a batch of the state vector is used as input to the critic NN. The batch of the advantage values is the output of the critic NN; it is used to evaluate the quality of the actions taken to determine these states. The parameters of the critic NN are updated by minimizing the advantage value per batch. The policy network is updated per batch using the action vectors taken from the old policy network, the state vector from the buffer, and the advantage value from the critic NN.

Huang et al. [72] have worked on quadrotor swarms with an end-to-end DRL algorithm that is the asynchronous version of the decentralized, independent PPO. The aim is for each drone to reach its desired target without colliding with each other, and with obstacles. The agent learns to adjust the thrust of each rotor for all quadrotors, independently. This method is real-time applicable but it still has limitations. First, it works only in a static obstacle environment. Second, the localization and obstacle detection components are not based on information received from onboard sensors. Third, according to the authors, the approach is promising for obstacle avoidance and stability, but it is not guaranteed. This method achieves comparable collision avoidance and task completion rates to SBC [88] with 4x faster inference speed and performs considerably better than GLAS [89].

In [56], the PPO and the PPO-IC algorithms have been compared with the offline PPO one and a well-tuned PID controller. The average linear velocity steady-state error of the PPO-IC approaches zero faster, and it is smaller than the PPO.

The average accumulated reward of the PPO-IC reaches a higher value. The PPO-IC converges closer to the targeted velocity in the x, y, and z axes than the PPO. PPO-IC velocity errors in the x, y, and z axes are much smaller compared to the PPO errors. The Euler angle errors are also smaller in the PPO-IC algorithm. In the offline learning phase, the nominal quadrotor weight is increased by 10% in each step until it reaches 150% of the nominal weight. The performance of the well-tuned PID controller and the proposed method are compared. When the quadrotor weight increases, the velocity error along the z-axis increases, too, but the PPO-IC algorithm demonstrates stable behavior without fluctuations in speed tracking. Moreover, 12 experiments are conducted when the nominal 0.2 m radius of the quadrotor is increased from 50% to 550%, that is, from 0.1 m to 1.1 m. PID and PPO-IC performance are similar when the radius is between 0.2 - 0.4 m. For higher values, PID performance decreases even when convergence to the desired value is observed. However, the PID controller cannot control the quadrotor - it becomes unstable when the radius increases to more than 1 m. On the contrary, changes in the radius value slightly affect the performance of the PPO-IC algorithm.

Kahn et al. [37] and Bhan et al. [59] have worked on failure avoidance and on compensating for occurred failures during flights. In [37], a Policy Learning using Adaptive Trajectory Optimization (PLATO) algorithm, a continuous, reset-free RL algorithm, is developed. In PLATO, complex control policies are trained with supervised learning using model predictive control (MPC) to observe the environment. Partially trained and unsafe policies are not utilized in the action decision. During training, taking advantage of the MPC robustness, catastrophic failures are minimized since it is not necessary to run the learned NN policy during training time. It is shown that good long-horizon performance of the resulting policy is achieved by the adaptive MPC. In [59], accommodation and recovery from fault problems occurring in an octacopter is achieved using a combination of parameter estimation, RL, and model-based control. Fault-related parameters are estimated using an Unscented

Kalman Filter (UKF) or a Particle Filter (PF). These fault-related parameters are given as inputs to a DRL, and the action NN in the DRL provides the new set of control parameters. In this way, the PID controller is updated when the control performance is affected by the parameter(s) correlated with faults.

Liu et al. [73] have applied a PPO-based RL algorithm for fault tolerance control of a quadrotor. The learning algorithm learns the PWM input for each rotor and it guarantees the UAV's stability and continuity during flight. The approach is tested via simulations and in real-world environments. It successfully handles the fault in the one of the quadrotor's rotor.

In [45], a DRL technique has been applied to a hexacopter to learn stable hovering in a state action environment. The DRL used for training is a model-free, on-policy, actor-critic-based algorithm called Trust Region Policy Optimization (TRPO). Two NNs are used as nonlinear function approximators. Experiments show that such a learning approach achieves successful results and facilitates controller design.

Yoo et al. [21] have combined RL and deterministic controllers to control a quadrotor. Five different methods, the original probabilistic inference for learning control (PILCO), PD-RL with high gain, PD-RL with low gain, LQR-RL, and LQR-RL with model uncertainty are compared via simulations when the quadrotor tracks a circular reference trajectory. The high-gain PD-RL approaches fast the reference trajectory. The low-gain PD-RL behaves less aggressively and reference trajectory tracking is delayed. The convergence rate of the PD-RL and LQR-RL methods is better. Performance is also better when compared to the original PILCO. The main advantages of combining a deterministic controller with PILCO are simplicity and rapid learning convergence.

In [44], errors on the pitch and roll angles are minimized to provide stability at hovering. A user-designed objective function uses simulated trajectories to choose the

best action. The objective function also minimizes the cost of each state. The performance of this controller is worse than a typical quadrotor controller performance. However, the proposed controller achieves hovering for up to 6 seconds after training using 3 minutes of data.

2.3.2. Actor-Critic Algorithms

Actor-critic algorithms consist of both value function-based and policy search-based methods. The actor refers to the policy search-based method and chooses the actions in the environment; the critic refers to the value function-based method and evaluates the actor using the value function.

In [61], three different RL algorithms, DDPG, TD3, and SAC, have been applied to study multirotor landing. Using the DDPG method does not result in successful landings. The TD3 and SAC methods successfully complete the landing task. However, TD3 requires a longer training period and landing is not as smooth, most likely because of noise presence in the algorithm.

Rodriguez et al. [20] have studied landing on a dynamic/moving platform using DDPG. Slow and fast scenarios have been tried in 150 test episodes. During the slow scenario, the moving platform (the moving platform trajectory is periodic) velocity is 0.4 m/s, and during the fast scenario is set at 1.2 m/s. The success rate is 90% and 78%, respectively. Using a constant velocity on the z-axis results in landing failure on the moving platform. This problem may be overcome by using the velocity on the z-axis as a state, but this makes the training process more complicated and learning the landing process becomes more challenging.

Rubi et al. [50] have solved the quadrotor path following problem using a deep deterministic policy gradient (DDPG) reinforcement learning algorithm. A lemniscate and one lap of a spiral path are used to compare agents with different state configurations in DDPG and in an adaptive Nonlinear Guidance Law (NLGL) algo-

rithm. The agent has only two states, distance error and angle error. According to the results, the adaptive NLGL has a lower distance error than the 2-state agent, but its distance error is significantly greater than the agent with the future states on the lemniscate path.

Rubi et al. [58] have also used three different approaches to solve the path following problem using DDPG. The first agent utilizes only instantaneous information, the second uses a structure (the agent expects the curve), and the third agent computes the optimal speed according to the shape of the path. The lemniscate and spiral paths are used to test the three agents. The lemniscate path is used in the training and test phases. The agents are evaluated in tests but with the assumption that the third agent is also limited by a maximum velocity of 1 m/s. For the lemniscate path, the agents are first tested with ground truth measurements. The second agent shows the best performance with respect to cross-track error. When the agents are tested with the sensor model, the third agent shows slightly better performance in terms of cross-track errors. Then, all agents are tested in the spiral path. When the performance of the agents is compared in simulations with ground truth measurements and with sensor models, the third agent (with a maximum velocity of 1 m/s) shows the best performance in terms of position error. In all tests, the third agent (without a maximum velocity limitation) completes the tracks faster.

Wang et al. [48] have handled the UAV navigation problem in a large-scale environment using a DRL. Two policy gradient theorems within the actor-critic framework are derived to solve the problem that has been formulated as a partially observable Markov decision process (POMDP). As opposed to conventional navigation methods, raw sensor measurements are utilized in the DRL; control signals are the output of the navigation algorithm. Stochastic and deterministic policy gradients for POMDP are applied to the RL algorithm. The stochastic policy requires samples from both the state and action space. The deterministic policy requires only samples from the

state space. Therefore, the RL algorithm with a deterministic policy is faster (and preferred) - it is called a fast recurrent deterministic policy gradient algorithm (Fast-RDPG). For comparisons, four different large-scale complex environments are built with random-height buildings to test the DDPG, RDPG, and Fast-RDPG. The success rate of the Fast-RDPG is significantly higher in all environments. Fast-RDPG has the lowest crash rate in one environment. DDPG provides the best performance with respect to the average crash rate in all environments. Fast-RDPG has a much lower crash rate than RDPG. However, Fast-RDPG provides a much lower stray rate than other algorithms in all environments.

Li et al. [81] have developed a new DRL-based flight resource allocation framework (DeFRA) for a typical UAV-assisted wireless sensor network used for smart farming (crop growth condition). The DeFRA reduces the overall data packet loss in a continuous action space. A DDPG is used in DeFRA, and DeFRA learns to determine the instantaneous heading and speed of the quadrotor and to choose the ground device to collect data from the field. The time-varying airborne channels and energy arrivals at ground devices cause variations in the network dynamics. The network dynamics are estimated by a newly developed state characterization layer based on LSTM in DeFRA. An MDP handles simultaneously the control of the quadrotor's maneuver and the communication schedule according to decision parameters (time-varying energy harvesting, packet arrival, and channel fading). The state space comprises the battery level, the data buffer length of all ground devices, the battery level and location of the UAV, the channel gain between the UAV and the ground devices, and the time-span parameter of the ground device. The UAV current battery level depends on the battery level of the UAV in the previous time step, harvested energy, and energy consumption. The quadrotor is required to keep its battery level at an equal or higher than the battery level threshold. Performance is compared with two DRL-based policies, DDPG-based Movement Control (DDPG-MC) and DQNs-based Flight Resource Allocation Policy (DQN-FRA) and with two non-learning heuris-

tics, Channel-Aware Waypoint Selection (CAWS) and Planned Trajectory Random Scheduling (PTRS). The DeFRA framework provides lower packet loss than other methods. The relation between the packet loss rate and the number of ground devices is investigated according to all methods. The DRL-based methods outperform CAWS and PTRS. For up to 150 ground devices, DeFRA and DDPG-MC show similar performance and better than other methods, but after increasing the number of ground devices to 300, DeFRA provides better performance than DDPG-MC.

Pi et al. [51] have created a low-level quadrotor control algorithm to hover at a fixed point and to track a circular trajectory using a model-free RL algorithm. The combination of the on-policy and off-policy methods is used to train an agent. The standard policy gradient method determines the update direction within the parameter space, while the TRPO and PPO algorithms are designed to identify an appropriate update size. However, for updating the policy, the proposed model establishes new updating criteria that extend beyond the parameter space concentrating on local improvement. The NN output provides the thrust of each rotor. The simulator is created in Python using the dynamic model of Stevens et al. [90]. The effects of the rotation matrix and quaternion are investigated in the learning process. The model with the quaternion may converge slower in the training process than the model with the rotation matrix. However, both models show similar performance when tested.

Ma et al. [68] have developed a DRL-based algorithm for trajectory tracking under wind disturbance. The agent learns to determine the rotation speed of each rotor of a hexacopter. A DDPG algorithm is used, but in addition to the existing DDPG algorithm, a policy relief (PR) method based on an epsilon-greedy exploration-based technique, and a significance weighting (SW) method are integrated into the DDPG framework. The former method improves the agent's exploration skills and its adaptation to environmental changes. The latter helps the agent update its parameters in a dynamic environment. In training, the implementation of PR and SW methods

to the DDPG algorithm provides better exploration performance and faster convergence of the learning process, respectively, even in a dynamic environment. This method reaches a higher average reward and has a lower position error compared to the DDPG, DDPG with RP, and DDPG with SW. Also, this algorithm provides higher control accuracy compared to the cascaded active disturbance rejection control algorithm in terms of position, velocity, acceleration, and attitude errors.

Hwangbo et al. [38] have proposed a method to increase UAV stabilization. A NN improves UAV stability training with RL. Monte Carlo samples are produced by on-policy trajectories and are used for the value function. The value network is used to guide policy training, and the policy network controls the quadrotor. Both are updated in every iteration. A new analytical measurement method describes the distance between action distribution and a new policy for policy optimization. This policy network gives an accurate reaction to step response. The policy stabilizes the quadrotor even under extreme situations. The algorithm shows better performance than DDPG and Trust Region Policy Optimization with a generalized advantage estimator (TRPO-gae) in terms of computation time.

Mitakidis et al. [70] have also studied the target tracking problem. A CNN-based target detection algorithm has been used on an octocopter platform to track a UGV. A DDPG-RL is applied in a hierarchical controller (instead of a position controller). The CNN learns to detect the UGV, and the DDPG-RL algorithm learns to determine the roll, pitch, and yaw actions in the outer loop of the controller. These actions are taken from the NN output as normalized between $[-1, 1]$, but these normalized values are multiplied by a spectrum of acceptable values. While the roll and pitch actions expand between -3 and 3 degrees, the yaw action spreads between -5 and 5 degrees. An experiment is conducted with a low-altitude octocopter and with manual control of a UGV. Fluctuations are observed in the distance error due to the aggressive maneuver of the UGV, but overall, the results are good.

Li et al. [54] have studied the target tracking problem in uncertain environments. The proposed approach consists of a TD3 algorithm and meta-learning. The used algorithm is named meta twin delay deep deterministic policy gradient (meta-TD3). TD3 learns to control the linear acceleration of the UAV and the angular velocity of the heading angle. The state space includes the position of the quadrotor in the x-y plane, the heading angle, the linear velocity, the angle between the motion direction and the straight line between the UAV and the target, and the Euclidean distance between the UAV and the target. Meta-learning overcomes the multi-task learning challenge. Tasks are trajectories of the ground vehicle that is followed. A reply buffer is built for the task experience. When the agent interacts with the environment, the state space, action space, reward value, and the next step state space that corresponds to the task are saved into the reply buffer. The method provides a significant improvement in convergence value and rate. Meta-TD3 adapts to the different movements of the ground vehicle faster than TD3 and DDPG algorithms. Meta-TD3 tracks the target more effectively.

Panetsos et al. [63] have offered a solution for the payload transportation challenge using a DRL approach. An attitude PID controller is used in the inner loop of the cascaded controller structure, while a position controller in the outer loop is replaced with a TD3-based DRL algorithm. The DRL algorithm in the outer loop learns to create the reference Euler angles, roll and pitch, and the reference translational velocity of the octocopter on the z-axis. The method controls the system successfully to reach desired waypoints.

Wang and Ye [65] have developed a consciousness-driven reinforcement learning (CdRL) for trajectory tracking control. The CdRL learning mechanism consists of online attention learning and consciousness-driven actor-critic learning. The former selects the best action. The latter increases the learning efficiency based on the cooperation of all subliminal actors. Two different attention-learning methods are

utilized for online attention learning: short-term attention learning and long-term attention learning. The aim of the former is to select the best action. The latter selects the best action to sustain the system’s stability. The long- and short-term attention arrays are combined to make a decision about which actor should be given more attention. This learning algorithm is compared with Q-learning; the position error in the proposed algorithm is lower than in Q-learning. The same is also seen in the velocity error. However, this method is slightly better than Q-learning when it comes to attitude error. The UAV is successfully controlled to track the desired trajectory by the CdRL algorithm.

Yi et al. [74] have proposed a visual servoing tracking control method for tracking an unknown ground vehicle by a quadrotor without using the global navigation satellite system. A safe reinforcement learning-based control method is used to obtain the approximate optimal control. The approach may work in real-time, and the stability of the proposed visual servoing control laws is proven. The advantage of this method is that the actor-critic neural networks work synchronously without complete system knowledge. The method is compared with LQR and state feedback controllers, and result show that it achieves better tracking performance based on the position tracking error.

Xu et al. [91] have created a benchmark using PPO, SAC, DDPG, DQN algorithms for single-agent tasks and multi-agent PPO (MAPPO), heterogeneous-agent PPO (HAPPO), multi-agent DDPG (MADDPG), and QMIX algorithms for multi-agent tasks with different drone systems. Single-agent tasks include hovering, trajectory tracking, and flythrough. Multi-agent tasks cover hover, trajectory tracking, flythrough, and formation. To increase task variation, payload, inverse pendulum, and transportation challenges are integrated into the single- and multi-agent tasks. Learning performance differs based on specific tasks.

Table 2.4: Online Learning Papers

Year	Paper	Task	Algorithm	Model-free or -based	Advantages	Compared with	Offline part	Sim/Exp
2018	Yang et al. [92]	Navigation Synchronization	[92]	Model-free	Solves inhomogeneous algebraic Riccati equations online	Adaptive control approach in [93]	No	Sim
2018	Wang et al. [94]	Environment Exploration	Data-driven approach based on Gaussian Process	Model-free	Reduces the possible crashes in the online learning phase	-	No	Sim
2019	Sarabakha and Kuyucu [95]	Trajectory tracking	Back-propagation	Model-free	-	Offline trained network PID controller	Yes	Sim
2019	He et al. [96]	Agile mobility in dynamic environment	StateRate	Model-free	Adjusts finely the prediction framework, and onboard sensor data are effectively used	Previous OPT Signal-to-noise rate (SNR) SampleRate CHARM	Yes	Sim
2019	Wang et al. [97]	Robust Control	DPG-IC	Model-free	Elimination of the steady error	PID controller DDPG	Yes	Sim
2020	Shin et al. [98]	Speed Optimization	SSD MobileNet	Model-free	Quicker object detection time	-	No	Sim
2020	Shiri et al. [99]	Path Planning	oHJB	Model-free	The algorithm keeps working even if UAV loses the connection with BS	aHJB mHJB	No	Sim
2022	Jaiton et al. [100]	Speed optimization	Neural Proactive Control	Model-free	Computationally inexpensive	MPC	No	Exp
2023	O'Connell et al. [101]	Stabilization	DAIML	Model-free	Can control a wide range of quadrotors Not require pretraining for each UAV	Mellinger & Kumar [102] $\mathcal{L}_1$ Adaptive Controller Incremental Nonlinear Dynamics Inversion Controller	Yes	Exp
2023	Jia et al. [103]	Trajectory tracking	RFPID	Model-based	Strong learning ability	PID Fuzzy-PID	No	Sim
2023	Zhang et al. [104]	Stabilization	RBLC	Model-free	Significant improvement for the stabilization in roll and pitch Not show the same performance on yaw	PID	No	Exp

2.4. Online Learning

In online learning, an agent learns and it is updated diuring the system operation by incorporating collected data from sensors used to make and improve decisions, also interacting with the environment. The agent learns in real-time, enhancing its decision and prediction capabilities (with respect to the assigned mission). Before proceeding, it is important to mention three more perspectives that clarify further what online learning is (compared to the definition provided in Section 2.2) and also they offer a more 'rounded' and more 'open-minded' point of view to consider.

Srinivasan and Jain [105], state that the intelligent behavior of an agent may be limited given the extreme difficulty in developing knowledge structures and rule bases, which completely describe the task of the agent if the task by nature is complex. This problem can be partially overcome by causing the agent to learn on its own during this task.

Hoi et al. [22], define online learning as a method of ML based on which data is arriving in sequential order, and where a learner aims to learn and to update the best prediction for future data at every step. Online learning is able to overcome the drawbacks of batch learning because the predictive model can be updated instantly for any new data instances.

Otterlo and Wiering [106], state that an important aspect during the learning task (during the learning process) is the distinction between online and offline learning. The difference between these two types is influenced by factors such as whether one wants to control a real-world entity or whether all necessary information is available. Online learning performs learning directly on the problem instance. Offline learning uses a simulator of the environment as a cheap way to get many training examples for safe and fast learning.

The literature review (from 2015) reveals that 11 papers have used online learning-based algorithms to control multirotor UAVs, in particular quadrotors. Their review and comparison provide a more accurate understanding of what the research is all about, what is being learned, why, and how, and what results have been produced.

Yang et al. [92] have developed optimal control protocols to solve the distributed output synchronization problem for leader-follower multiagent systems. The adopted RL algorithm solves the underlying non-homogeneous Algebraic Riccati Equations (AREs) in real-time - this is basically a distributed optimal tracking problem. Solving the AREs guarantees synchronization of the followers' and the leader's output. A distributed observer is derived to forecast the leader's state and to produce the reference signal for each follower. The advantage of the proposed RL algorithm is that it does not require knowledge of the quadrotor dynamics. This method gives better results than the adaptive control approach developed by Das and Lewis [93]. Moreover, when the transient response of the output synchronization error for each follower is compared, the error in [92] converges to zero faster than in [93].

A neural proactive closed-loop controller has been developed in [100] that learns control parameters. The proposed solution provides two advantages. The first is computational inexpensiveness due to using only three neurons instead of a neural network. The second is that the learning process can be completed within a few trials. No knowledge of the UAV dynamic model is required. This technique has been used

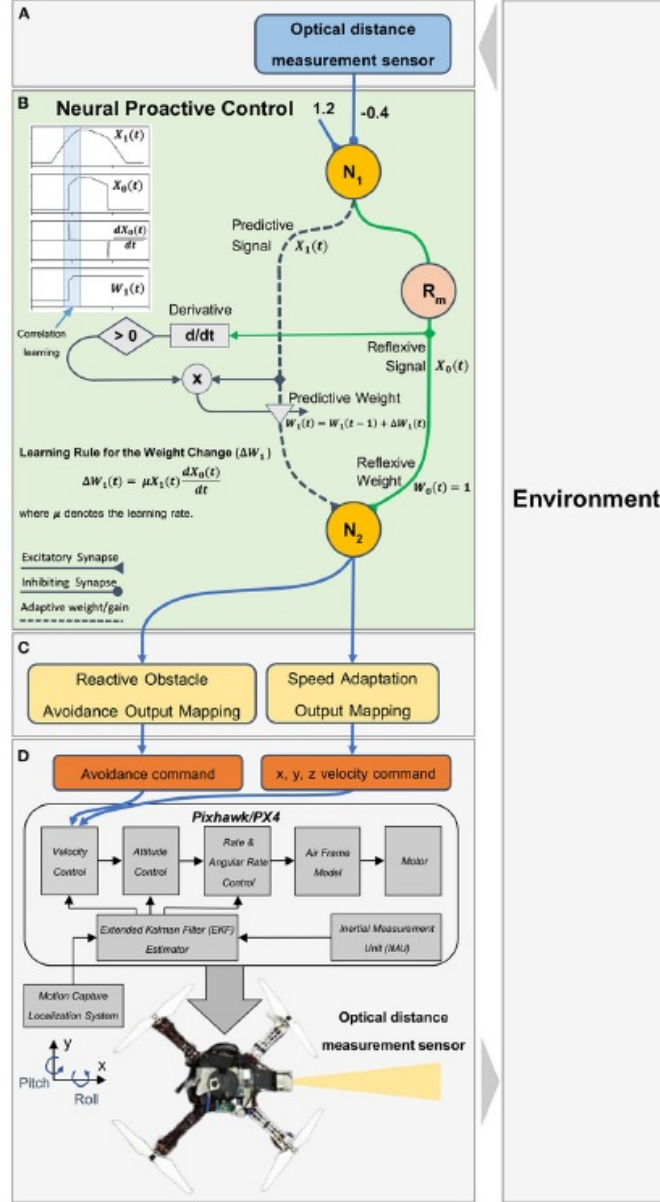


Figure 2.8: Online learning controller scheme of [100].

for quadrotor speed adaptation and obstacle avoidance following the detailed block diagram configuration shown in Fig. 2.8. For performance evaluation purposes, an MPC and the proposed method are implemented and compared. The MPC requires knowledge of the quadrotor dynamic model. Thus, the neural proactive controller method is more appealing. It also provides robust results for speed adaptation even

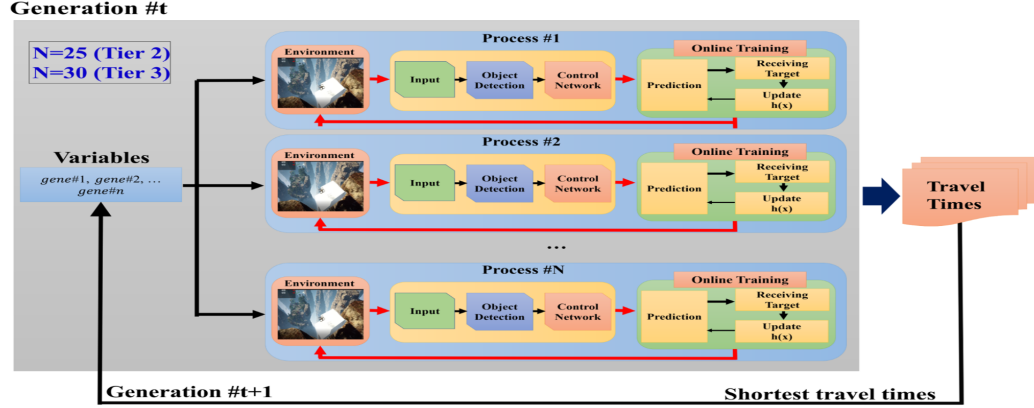


Figure 2.9: Flow diagram of the racing parameter optimization during online training phase [98].

in the presence of wind disturbances. This approach has been implemented and tested on UAVs with different dynamics, in a Gazebo environment, under different maximum flying speeds. The neural proactive controller generates a control command 56.32% faster (on average) than the MPC; it is also 99.47% faster than the MPC in total learning and optimization time. The UAV is trained within 3-4 trials adapting its speed and learns to stay away from obstacles at a safe distance.

Shin et al. [98] have applied online learning to optimize speed parameters with the aim to complete missions in less time (i.e., drone races through gates). A pre-trained actor network, modified to be suitable for online training, detects a specific object. The control network, which comprises the online learning component, provides UAV optimized linear velocity and acceleration in the x, y, and z directions, as well as the distance to a gate to pass through with maximum velocity, and without collisions. Each gate and the gate center are detected by a monocular camera system. The UAV coordinates are determined based on the provided information, and a loss function is used to make sure that the quadrotor passes through the center of the gate. The maximum velocity, acceleration, and threshold distance (that determines whether or not the quadrotor reaches the gate location) are parameters that are optimized for each gate. They are updated each time the quadrotor passes through each gate. As

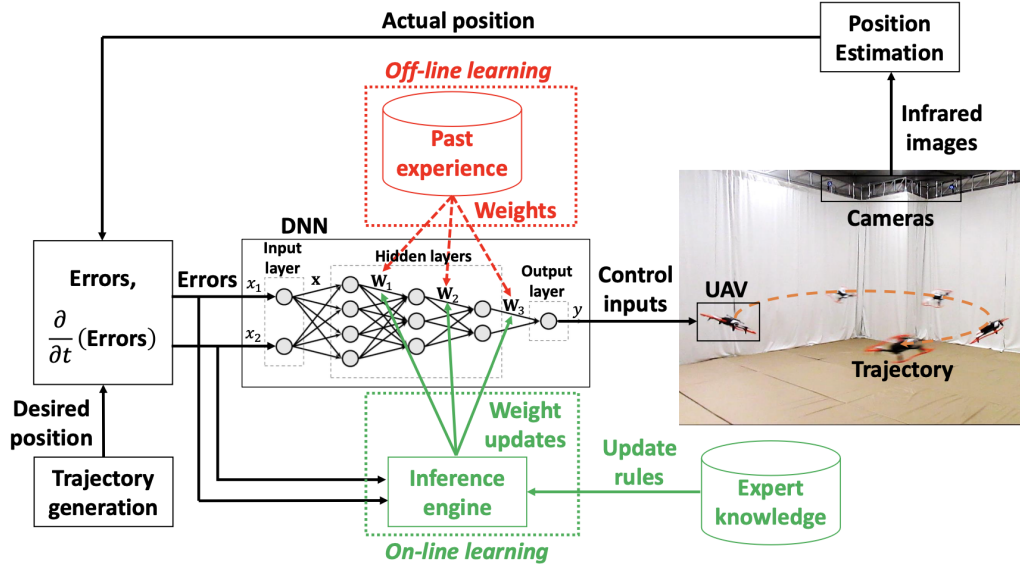


Figure 2.10: Block diagram of the online learning-based control developed by Sarabakha and Kayacan [95].

opposed to several conventional navigation approaches, this method requires a single neural network to control the quadrotor. The evolution-based approach is combined with the gradient-based method, and this cooperation provides an advantage to increase the system performance in drone racing. The derived network has successfully handled all race processes in environments with demanding obstacle avoidance and navigation requirements. When tested in real-time, the quadrotor completed the race track in around 180 seconds on the first try, but in about 60 seconds after about 9000 times.

Sarabakha and Kayacan [95] have proposed an online learning-based control method to improve UAV trajectory tracking. Total thrust and the three torques around the x, y, and z axes are used as control inputs. The online learning component learns to update the weight of a DNN to improve control performance. This method consists of two phases, pre-training called offline learning, and, post-training called online learning. In pre-training, supervised learning is used to control the quadrotor by mimicking a PID controller (using an input-output dataset for a set of trajectories).

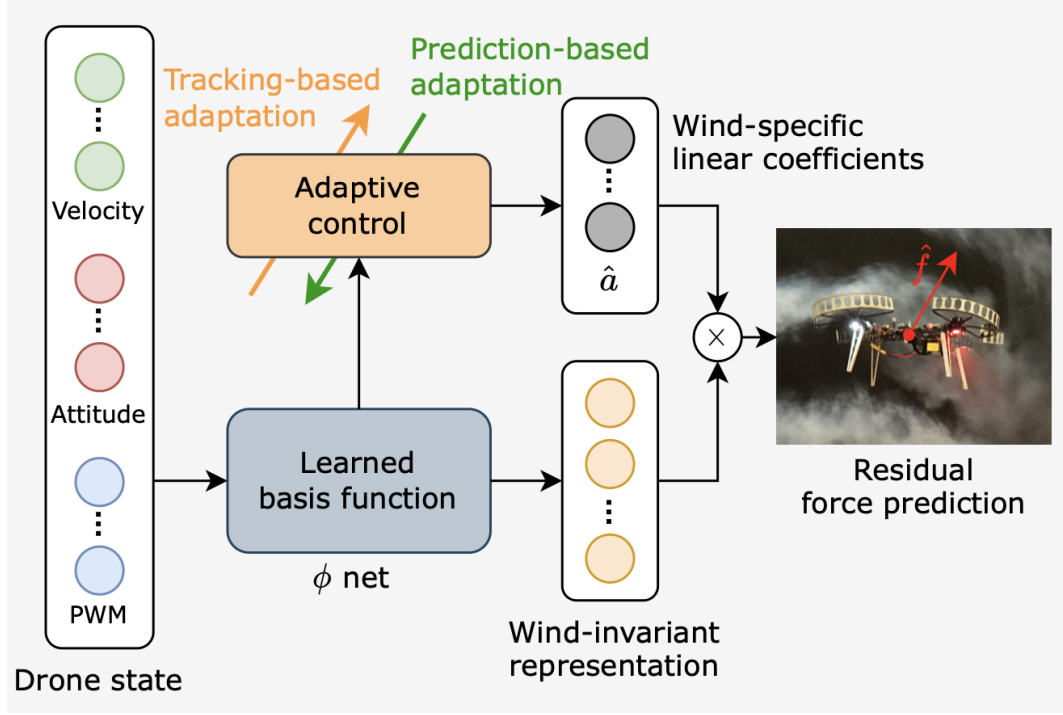


Figure 2.11: Block diagram of Neural-Fly algorithm offered by O’Connell et al. [101].

When the quadrotor is controlled by the trained DNN controller, a fuzzy logic system (FLS) keeps training the DNN online by providing feedback about its performance. The advantage of using the fuzzy mapping in the online learning phase is that the computation time is considerably reduced. Also, this provides an opportunity for real-time applicability. The offline learning-based performance is not different from the classic PID controller performance. However, online learning allows for the system to accurately predict evolution and desired signal estimation. This approach has been tested in slow circular, fast circular, and square-shaped trajectories. Performance of the well-tuned PID controller used in offline training, offline-trained DNN, and the online training approach are compared, with the last one clearly showing better performance for the slow and fast circular trajectories. The square-shaped trajectory was not used in the offline training phase. When the three are implemented and tested in the square-shaped trajectory, again, the online learning method slightly outperforms other approaches.

O’Connell et al. [101] have proposed Neural-Fly that includes an online learning phase to overcome instabilities caused by wind effects. The approach includes offline and online learning. In offline learning, the DNN output is a set of basis functions that represent aerodynamic effects. The latter phase is an online adaptive control phase that learns to adapt the control system to new wind conditions rapidly and robustly. For offline learning, the domain adversarially invariant meta-learning (DAIML) algorithm is developed to learn aerodynamic effects under wind-invariant conditions. A stochastic gradient descent method is used in the DAIML algorithm for training. For online learning, a Kalman Filter-based adaptation algorithm estimates the wind-dependent linear coefficients. The position tracking error and the aerodynamic force prediction error terms are utilized in this estimation under wind-variant conditions. The online learning component provides an advantage of fast adaptation to wind variations. Also, the other advantage of the proposed approach is that the approach may be used to control several quadrotors without the need to pre-train each UAV. The proposed control algorithm is tested with the quadrotor in the Caltech Real Weather Wind Tunnel. Neural-Fly shows better performance than the nonlinear tracking controller found in [102], $\mathcal{L}_1$ adaptive controller, and an incremental nonlinear dynamics inversion controller when the quadrotor is subjected to time-variant wind profiles.

Jia et al [103] have provided a solution to the trajectory tracking problem by combining a fuzzy logic method, a radial basis function (RBF) NN, and a classical PID controller. The PID output and the current UAV position information are provided to the RBF NN as input values, and the network learns to adjust controller parameters. The fuzzy logic component selects the initial controller parameters, while the RBF NN adjusts them. Both are combined to create a new set of parameters for the PID controller. In the fuzzy logic component, a database created from expert knowledge is used to decide about the initial controller parameters. However, this dataset cannot be used in the RBF NN component since the adopted algorithm is an unsupervised learning method. However, the combination of both methods overcomes

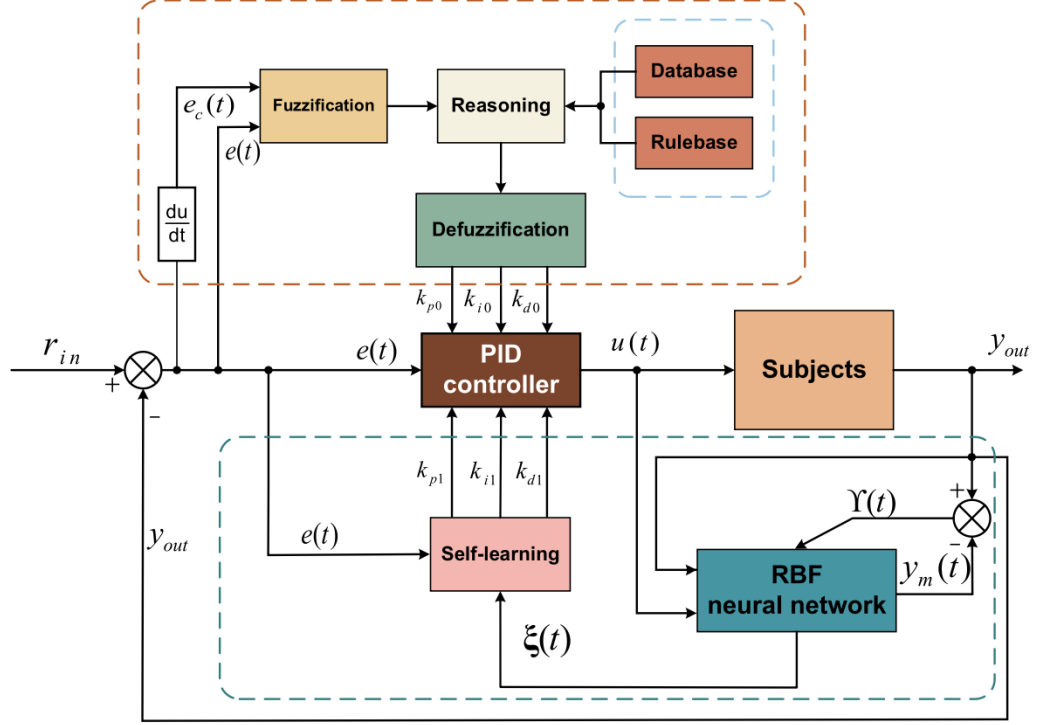


Figure 2.12: Block diagram of Fuzzy-RBF NN adaptive PID controller [103].

this limitation and provides online learning abilities. When the fuzzy logic system adjusts the PID gains, the RBF NN tweaks the PID parameters in real-time to overcome PID weaknesses caused by environmental disturbances. This approach when compared with PID and fuzzy-PID (FPID) controllers shows better performance for trajectory tracking.

Zhang et al.[104] have investigated the problem of adaptive control and have offered a real-time brain-inspired learning control (RBiLC) method as a solution. Three attitude angle errors are set as input and a NN provides the control parameter increment as output. In the RBiLC method, a PID controller is used, and the controller parameter is updated in each interaction. A DRL method learns the controller parameter rate. The algorithm uses the Nesterov momentum technique for gradient descent. Controller stability and convergence of the tracking error is demonstrated. The quadrotor takes off and reaches a 10-meter altitude, then it hovers for 3 sec-

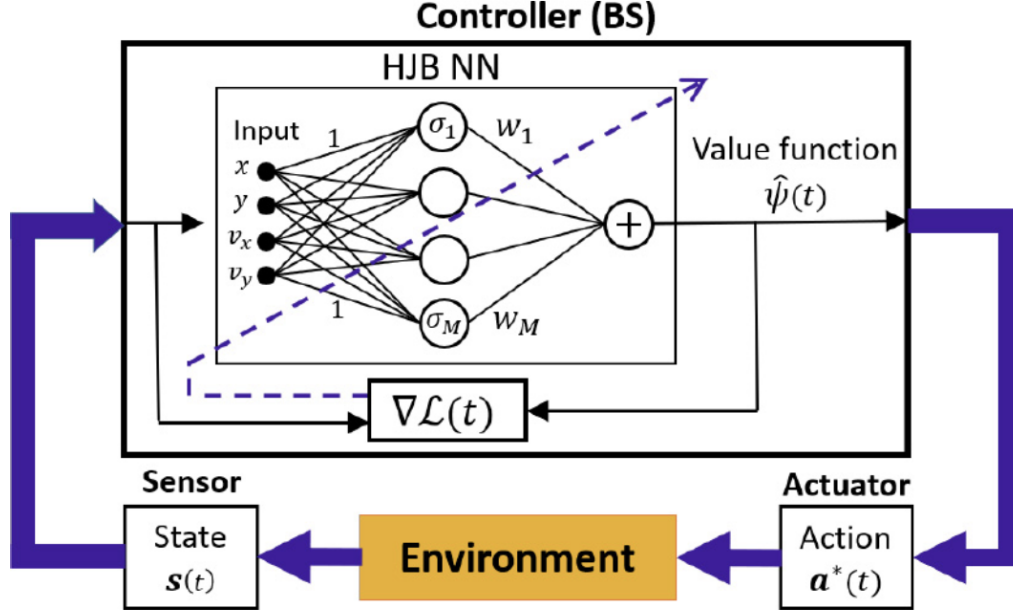


Figure 2.13: Block diagram of NN-based control architecture in [99].

onds in this position with the initial controller parameters. The learning algorithm is activated using a switch system, and the RBiLC algorithm updates PID gains in real-time in an environment with wind disturbances. This approach learns controller parameters in 3 to 5 minutes, which is a much shorter time than in offline learning methods. This method shows significant improvement for stabilization in roll and pitch angles, but performance is not the same in the yaw angle. Under wind disturbances, the method provides a shorter rise time and steady-state time for roll and pitch when compared to the classic PID controller.

Shiri et al. [99] have studied the online path planning problem using NN-based opportunistic control. An online-trained NN learns to solve the Hamilton-Jacobian-Belmann (HJB) equation in real-time. The opportunistic HJB, oHJB, control algorithm learns whether it will upload the control action (aHJB) that is the output of the NN or the current online trained NN (mHJB). A base station (BS) is utilized to handle the online learning algorithm. The UAV state is downloaded to the BS. First, the NN is trained online to solve the HJB in real-time, in the BS. Then, the output

of the trained NN (the control action aHJB), is uploaded to the UAV. Secondly, the current online trained NN (represented as mHJB), is uploaded to the UAV (instead of the aHJB), and the mHJB is fed with the current states. Then, the UAV takes the action that is locally assessed by the uploaded mHJB in real-time. The oHJB control provides the decision mechanism to switch from aHJB to mHJB according to the connection between the multirotor UAV and the BS. Based on the oHJB, the UAV can keep taking actions using the last uploaded mHJB even if it loses connection with the BS. Since the size of the trained NN model in the BS is larger than the size of the action space, a trade-off occurs between uploading delays and control robustness against poor channel conditions. The oHJB arrives at the targeted location in a shorter time than aHJB and mHJB. The aHJB and mHJB may fail to arrive at the desired location.

Wang et al. [94] have used a data-driven approach (Gaussian processes) to learn quadrotor models applied in partially unknown environments. Barrier certificates are learned and utilized to adjust the controller to not encroach in an unsafe region. A safe region is certified and it is progressively spread with new data. Collecting more data points about system dynamics reduces uncertainty and maximizes the volume of the safe region. Discretizing the state space helps sample a finite number of points to affirm the barrier certificates; so adaptive sampling decreases the number of the required sampling points. Also, the state space is adaptively sampled by enhancing the Lipschitz continuity of the barrier certificates without taking any risk on safety guarantees. Sampling the most uncertain state in the safe set of the system boosts learning efficiency during the exploration phase. A kernel function is utilized to decide data relevance, and 300 data points are chosen in the recursive Gaussian process by eliminating irrelevant data points online. The approach reduces the possibility of an accident occurring during the online learning phase. The adaptive sampling strategy provides significantly better results in decreasing the required sample points. The quadrotor never violates safe and unsafe regions; it uses barrier certificates to

regulate the learning algorithms. A high probability of safety guarantee is produced by a Gaussian process for the dynamic system, and this process learns unmodeled dynamics to help the quadrotor successfully stay within the safe region. No information is provided related to real-time applicability, but the authors report that they keep the Gaussian process interface time under 20 *ms*, and this allows for a suitable opportunity for online learning. When the tracking error of the learning-based controller is compared to the tracking error without Gaussian process inference, the tracking error is much smaller in the learning-based controller.

He et al. [96] have solved the time-varying channel(s) problem on air-to-ground links caused by UAV mobility in dynamic environments. In the offline learning phase, the agent learns to minimize the prediction loss function in the prediction network and the evaluation loss function in the evaluation network. In online learning, the aim is to learn to minimize the difference between two networks. A state-optimized rate adaptation algorithm called StateRate is developed to solve this problem using the onboard UAV sensors. The evaluation and prediction networks are exploited in online training. The received signal strength indicator and the channel state information are used as inputs for both networks; the prediction network, additionally, uses the UAV's states as input. The output of the evaluation NN is used for supervision and compared with the output of the prediction network by using a fully connected layer. The StateRate algorithm accurately predicts the optimal rate. The rate prediction is handled as a multi-class classification problem using online learning. Retraining the pre-trained NN is not suitable, and StateRate with online learning provides an advantage to overcome this unsuitable situation. This method is applied and tested in a commercial quadrotor (DJI M100) and has shown better performance than the best-known rate adaptation algorithms applied in UAVs with 2-6 *m/s* velocity.

Wang et al. [97] have worked on robust control of a quadrotor using a DRL-based controller. The method includes both offline and online learning. During offline

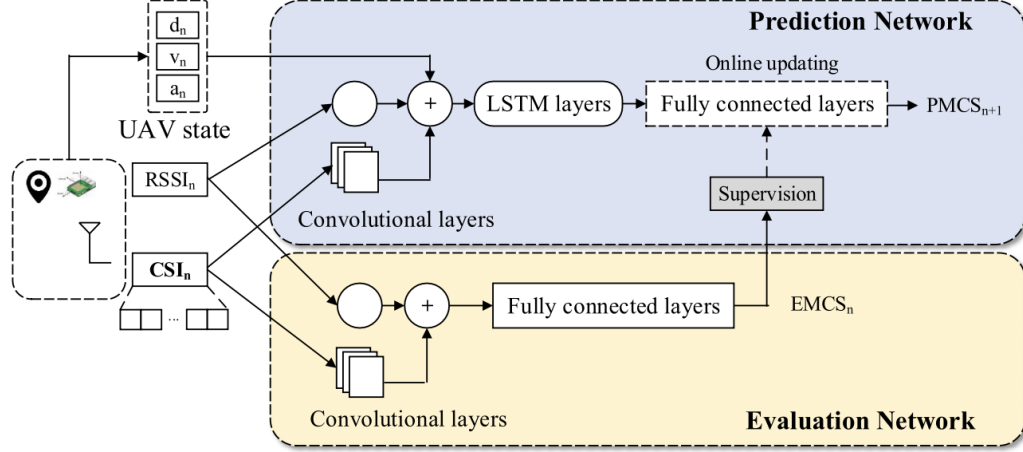


Figure 2.14: Block diagram of the online training in StateRate proposed by [96].

learning, an offline learning control policy is learned. The actor-network learns the normalized thrusts produced by each rotor between 0 and 1. During online learning, the offline control policy continues to be optimized. In the offline phase, the Deterministic Policy Gradient-Integral Compensator (DPG-IC) algorithm is trained with random actor and critic NN weights in episodic structure. In the online learning phase, the trained DPG-IC is used as the initial NN structure that is trained continuously. The offline DPG-IC also runs alongside the online algorithm. The system switches to the offline algorithm when the states are close to the safety limits, and the quadrotor goes back to the safe range. The aim of the online learning phase is to close the gap between the simplified model and the model with real flight dynamics. The main advantage of DPG-IG is to remove the steady-state error. A well-tuned PID controller shows similar performance compared to the offline DPG-IC policy when the size of the quadrotor is increased from 0.12 m to 0.4 m, but for larger sizes, the PID controller is not stable while the proposed method shows successful performance for sizes up to 1 m. The PID controller and the offline DPG-IC algorithm are also compared for different payloads. The payload of the quadrotor increases by 10% each step; the PID cannot control the quadrotor with a 30% increased payload. On the other hand, the proposed offline learning method successfully controls the quadrotor

with up to 50% payload increase. Then, the performance of the online learning policy is compared with the offline learning policy. The model with a 0.4 m radius and 20% increased payload is used. After only 200 online training steps, performance is significantly increased.

In summary, since RL is built on the interaction between the agent and environment and it is based on a reward system, RL algorithms are useful for online learning and they are widely preferred for applications related to control of multirotor UAVs.

2.5. Advantages and Limitations

Four reviewed articles include both offline and online learning phases, while seven articles have only online learning phases. The online learning phase (used after offline learning) provides an opportunity to learn how to compensate for differences between basic and complex UAV models, and environmental disturbances, or to reduce errors. Applying directly online learning without an offline learning phase helps reduce considerably training time. For example, in [100] only 3-4 trials are required for learning, while in [98] learning occurs in every attempt reducing mission completion time. However, an important observation in online learning algorithms is that NNs cannot be used alone; they are always combined with auxiliary components, such as conventional controllers, fuzzy logic systems, and optimization or adaptation algorithms, to decrease computational cost and learning time or to increase adaptation abilities. In addition, BS may be used to increase the computational capacity, and 5G technology helps transfer the trained agent or output of the NNs to the multirotor UAVs as shown in [99]. Reducing the number of neurons also offers advantages because of the reduction of the computational cost [100].

A common characteristic of online learning techniques is that all methods are model-free except the proposed approach in [103].

Online learning allows for an agent to continuously learn and update its decision-making process using real-time data, enhancing its capabilities to complete different missions. In online learning, as opposed to offline learning, the agent has the ability to learn directly from problem instances. Online learning reviewed articles for multirotor UAV control demonstrate efficacy in tasks such as trajectory tracking, speed adaptation, obstacle avoidance, and path planning. Used techniques include neural network-based controllers, RL algorithms, and adaptive control methods. These methods offer advantages such as computational efficiency, real-time adaptability, and robust performance in dynamic environments or under disturbances, like wind gusts.

Overall, online learning emerges as a powerful tool for enhancing UAV control and navigation in complex scenarios, but it still includes significant limitations as mentioned before. Given that eventually, there will be 'almost infinite computational power' that will require 'almost zero computational time' to return results, this review offers a starting point for subsequent studies on what is hard real-time implementable.

2.6. Remarks

This literature review provides a comprehensive summary and overview of the evolving landscape of multirotor UAV navigation and control, particularly focusing on the integration of learning-based algorithms over the past decade. Given the limited published articles in online learning, this Chapter highlights challenges of online learning techniques.

The next Chapter derives the mathematical model of a multirotor UAV (quadrotor), followed by parameter estimation approaches that are used.

CHAPTER 3: QUADROTOR MATHEMATICAL MODEL DERIVATION

3.1. Comprehensive N-E Based Quadrotor Mathematical Model

A comprehensive, complete, quadrotor UAV mathematical model is derived that includes gyroscopic and drag effects, as well as rotor dynamics following the N-E formulation.

3.1.1. Basic Model:

The aerial robot platform considered in this dissertation is a plus-configuration quadrotor. While two rotors of the quadcopter rotate clockwise, the other two rotate counterclockwise. The pitch angle (θ) is occurred by the speed difference of the first and third rotors. It induces the vehicle to move forward if the speed of the third rotor is greater than the first rotor's. In the opposite condition, the aerial robot moves backward. The distinction between the speed of the second and fourth rotors causes the quadrotor to move leftward or rightward, and the roll angle (ϕ) appears by this difference. The configuration of the rotation direction of the propeller is significant to cancel or control the rotation around the z -axis. The yaw angle (ψ) is zero if the total torque produced by the first and third rotors is equal to the total torque produced by the second and fourth rotors. These three angles are called the Euler angles ($\eta = (\phi, \theta, \psi)$).

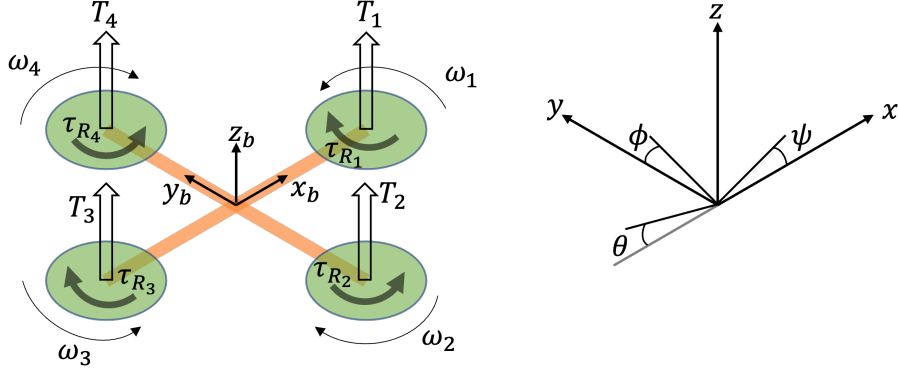


Figure 3.1: Initial and body-fixed frames and forces and torques applied on the quadrotor [107].

As shown in Fig. 3.1, the inertial frame is denoted by $E = \{x, y, z\}$, and the frame rigidly attached to the airframe is called body-fixed frame and denoted by $B = \{x_b, y_b, z_b\}$.

Each rotor in the quadrotor produces force and reactive torque, and they are given by (3.1) and (3.2), respectively.

$$f_i = k\omega_i^2, \quad i \in \{1, 2, 3, 4\} \quad (3.1)$$

$$\tau_{M_i} = b\omega_i^2 + I_R\dot{\omega}_i \quad (3.2)$$

where ω_i is the i^{th} propeller number, k and b are the thrust and rotor reaction drag coefficients, respectively.

The total thrust is obtained by the summation of the forces of all rotors and expressed by

$$T = f_1 + f_2 + f_3 + f_4 \quad (3.3)$$

The forces generated by the rotors create torques at the center of gravity (CG) on the fuselage and these torques are given by $\tau = [\tau_1, \tau_2, \tau_3]^T$, with

$$\begin{aligned}\tau_1 &= l(f_2 - f_4) \\ \tau_2 &= l(f_1 - f_3) \\ \tau_3 &= -\tau_{M_1} + \tau_{M_2} - \tau_{M_3} + \tau_{M_4}\end{aligned}\tag{3.4}$$

where l represents the projection of the distance between airframe CG and the forces created by rotors on x_b or y_b axis in the body-fixed frame. Therefore, τ is represented in the body-fixed frame.

The linear transformation that maps the propeller rates to the generated thrust T and torques $\tau = [\tau_\phi, \tau_\theta, \tau_\psi]^T$ expressed in B , is modeled as

$$\begin{bmatrix} T \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} k & k & k & k \\ 0 & -kl & 0 & kl \\ -kl & 0 & kl & 0 \\ b & -b & b & -b \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}\tag{3.5}$$

Equations of motion of the quadrotor are written using the Newton-Euler approach in the following form:

$$\dot{\xi} = v\tag{3.6}$$

$$\dot{v} = -g\hat{z} + \frac{1}{m}TR\hat{z}\tag{3.7}$$

$$\dot{R} = RS(\omega^B)\tag{3.8}$$

$$I_f\dot{\omega}^B = -\omega^B \times (I_f\omega^B) + \tau\tag{3.9}$$

where $\xi = (x, y, z)^T$ denotes the position of the body frame B with respect to the inertial frame E , and $v = (v_x, v_y, v_z)^T$ expresses the linear velocity of the body-fixed frame B with respect to the inertial frame E , the acceleration of gravity and the

mass of the fuselage are designated by g and m , respectively, $\hat{z} = (0, 0, 1)^T$ is the unit vector in the inertial frame E , $R(\eta)$ is the rotation matrix parametrized by using the Euler angles and is produced and represented by (3.10) and (3.11), respectively.

$$R(\eta) = R_z(\psi)R_x(\phi)R_y(\theta) \quad (3.10)$$

$$R = \begin{bmatrix} c_\theta c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ c_\theta s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi \\ -s_\theta & s_\phi c_\theta & c_\phi c_\theta \end{bmatrix} \quad (3.11)$$

where $c_\alpha \triangleq \cos \alpha$, $s_\alpha \triangleq \sin \alpha$, $\alpha \in \{\phi, \theta, \psi\}$.

The rotation matrix maps vectors in the body-fixed frame to the inertial frame, and $R(\eta) \in SO(3)$. Its properties are $RR^T = R^T R = I$, $\det(R) = 1$, and each column and row of the rotation matrix is a unit vector, and unit vectors are reciprocally orthogonal.

When $\theta = \pm\pi/2$, singularities occur, and the meaning is that there is no solution for the problem related to the inverse of the rotation matrix. ω^B in (3.8) represents the angular velocities of the fuselage in the body-fixed frame and is expressed by $\omega^B = (p, q, r)^T$. Also, $S(\cdot)$ in (3.8) denotes the skew-symmetric matrix, and $S(\omega^B)$ is defined as follows:

$$S(\omega^B) = \begin{bmatrix} 0 & -r & q \\ r & 0 & -p \\ -q & p & 0 \end{bmatrix} \quad (3.12)$$

The multiplication of the matrix $S(\omega^B)$ and a vector η means the cross product of ω^B and η , $(\omega^B \times \eta)$. I_f in (3.9) denotes the symmetric and positive definite inertia matrix of the fuselage with respect to the body-fixed frame located at the airframe CG.

3.1.2. Modeling of Gyroscopic Effects

The multirotor UAV airframe experiences an additional torque caused by the interplay between the angular movements of the rotors and the airframe. Due to the high angular velocity of the propeller, it can be assumed that the propellers behave like rigid disks that rotate around the e_z axis in the body-fixed frame at a rate of ω_i . The axis of the rotation of the rotor moves with the rotary motion of the fuselage, which induces gyroscopic torque on the airframe. The gyroscopic torques are given by

$$G_a = \sum_{i=1}^4 (-1)^{i+1} \omega^B \times I_r \vec{\omega}_i \quad (3.13)$$

where $\vec{\omega}_i = (0, 0, \omega_i)^T$. $I_r \in \mathbb{R}^{3 \times 3}$ denotes the moment of inertia of the rotor [108].

After applying the gyroscopic effect on the system, (3.9) is rewritten as follows:

$$I_f \dot{\omega}^B = -\omega^B \times (I_f \omega^B) - G_a + \tau \quad (3.14)$$

3.1.3. Modeling of Drag Effects

To increase the multirotor UAV model accuracy, drag effects must be considered and included in the equations of motion [108]. The different types of drag are defined and presented next.

Blade Flapping: Since all rotor blades flap due to the flexibility of the blade, blade flapping occurs when they are exposed to the translational motion of the quadrotor. This phenomenon is explained in detail in [109].

The equation of blade flapping is given by

$$\Delta_i = T_i \left(A_{flap} \frac{V_{p_i}}{\omega_i} + B_{flap} \frac{\omega^B}{\omega_i} \right) \quad (3.15)$$

where $T_i = C_T \omega_i^2$, and A_{flap} and B_{flap} are given by

$$A_{flap} = \frac{1}{r} \begin{bmatrix} -A_{1c} & A_{1s} & 0 \\ -A_{1s} & -A_{1c} & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (3.16)$$

$$B_{flap} = \begin{bmatrix} -B_2 & B_1 & 0 \\ B_1 & -B_2 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (3.17)$$

Also, V_{p_i} and ω_i respectively denote the translational velocity and the speed of the i^{th} rotor. The translational velocities of each rotor are given by

$$\begin{aligned} V_1 &= V + \omega^B \times [d \ 0 \ h]^T \\ V_2 &= V + \omega^B \times [0 \ -d \ h]^T \\ V_3 &= V + \omega^B \times [-d \ 0 \ h]^T \\ V_4 &= V + \omega^B \times [0 \ d \ h]^T \end{aligned} \quad (3.18)$$

where V is the velocity of the body-fixed frame with respect to the inertial frame expressed in the body-fixed frame, and the distance and height of the rotor from fuselage CG are denoted by d and h , respectively.

Induced Drag: If β angle blade flapping occurs in the rotors, this causes the backward inclination of the force in the rotor. This backward inclination reveals the induced drag and the equation of the induced drag is represented by

$$D_I = K_I V_p \quad (3.19)$$

where K_I denotes the induced drag coefficient.

Translational Drag: The airflow passing through the rotor induces a drag, and it is called translational drag, also termed as momentum drag in [110]. The equation of the translational drag is expressed by (3.20).

$$D_T = K_T V_p \quad (3.20)$$

where K_T represents the translational drag coefficient.

Profile Drag: When the quadrotor moves horizontally through the air, the transverse airflow through the rotor blades reveals the profile drag. The angle attack of the blades' airfoil does not change the profile drag, but the airspeed changes it in a limited manner. Its equation is represented as seen in (3.21).

$$D_p = K_p V_p \quad (3.21)$$

where K_p is the profile drag coefficient.

Parasitic Drag: Parasitic drag is created from the air friction to the airframe and rotors, and it is given by

$$D_{par} = K_{par} |V| V \quad (3.22)$$

The total drag force on i^{th} rotor is given by

$$D_i = D_{I_i} + D_{T_i} + D_{p_i} + \Delta_i \quad (3.23)$$

The total drag force applied on the quadrotor is given by

$$D_b = \sum_{i=1}^4 D_i + K_{par} |V| V \quad (3.24)$$

The total drag force on each rotor causes torque on the quadrotor with respect to the fuselage CG and the torques are given by

$$\begin{aligned} \tau_{D_1} &= D_1 \times [d \ 0 \ h]^T \\ \tau_{D_2} &= D_2 \times [0 \ -d \ h]^T \\ \tau_{D_3} &= D_3 \times [-d \ 0 \ h]^T \\ \tau_{D_4} &= D_4 \times [0 \ d \ h]^T \end{aligned} \quad (3.25)$$

The total torque generated by the drag forces on the body is given by

$$\tau_{Drag} = \sum_{i=1}^4 \tau_{D_i} \quad (3.26)$$

After applying the drag effects to the equations of motion of the quadrotor, they are rewritten as

$$\dot{\xi} = v \quad (3.27)$$

$$\dot{v} = -g\hat{z} + \frac{1}{m}TR\hat{z} - \frac{1}{m}RD_b \quad (3.28)$$

$$\dot{R} = RS(\omega^B) \quad (3.29)$$

$$I_f \dot{\omega}^B = -\omega^B \times (I_f \omega^B) - G_a + \tau + \tau_{Drag} \quad (3.30)$$

Table 3.1: Equations of Motion of a quadrotor UAV based on N-E formulation

Models	Equations
Basic Model (BM)	$\dot{\xi} = v$ $\dot{v} = -g\hat{z} + \frac{1}{m}TR\hat{z}$ $\dot{R} = RS(\omega^B)$ $I_f\dot{\omega}^B = -\omega^B \times (I_f\omega^B) + \tau$
BM with Gyroscopic Effect	$\dot{\xi} = v$ $\dot{v} = -g\hat{z} + \frac{1}{m}TR\hat{z}$ $\dot{R} = RS(\omega^B)$ $I_f\dot{\omega}^B = -\omega^B \times (I_f\omega^B) - G_a + \tau$
BM with Gyros & Drag Effects	$\dot{\xi} = v$ $\dot{v} = -g\hat{z} + \frac{1}{m}TR\hat{z} - \frac{1}{m}RD_b$ $\dot{R} = RS(\omega^B)$ $I_f\dot{\omega}^B = -\omega^B \times (I_f\omega^B) - G_a + \tau + \tau_{Drag}$

3.1.4. Motor Dynamics

Delays in electromechanical energy conversion do not allow for the immediate/instant application of the commanded angular velocity to the rotor. As such, the modeled relation between the motor speed and the controlled input speed is given by the following relation:

$$\dot{\omega}_i = \tau_m^{-1}(\omega_{c,i} - \omega_i) \quad (3.31)$$

where τ_m denotes the motor time constant, while $\omega_{c,i}$ represents the control input [111], [112].

Table 3.1 lists the quadrotor equations of motion that include the basic model, the model including gyroscopic effects, and the complete model including gyroscopic and drag (aerodynamic) effects. The complete model is used throughout the dissertation.

3.1.5. Wind Gust Model

Wind disturbances are also added to the system (quadrotor model) to improve model accuracy and to make the simulation environment as realistic as possible. The wind gust model used for testing follows the one reported in [113] and described by (3.32) as follows:

$$V_{\omega} = \begin{cases} 0 & t < t_i \\ V_{cos} & t_i < t < t_i + T_{\omega} \\ 0 & t > t_i + T_{\omega} \end{cases} \quad (3.32)$$

where,

- $V_{cos} = (V_{\omega_{max}}/2) (1 - \cos(2\pi((t/T_{\omega}) - (t_i/T_{\omega}))))$
- T_{ω} is the period of the wind gust
- t_i is the initial time of the wind gust
- $V_{\omega_{max}}$ is the wind gust peak
- t is the time expressed in seconds

This wind gust model allows for, and enables, simulation of intermittent wind disturbances, enhancing the fidelity of the environment by mimicking realistic gust patterns encountered in actual flight conditions. The model, based on Anderson's work in [113], provides a structured approach to simulate gusts with adjustable on-set, duration, and intensity. By incorporating this dynamic disturbance, the system's response to sudden environmental changes is assessed, allowing for a more comprehensive evaluation of the controller's robustness and adaptability under variable wind conditions.

3.2. Remarks

In this chapter, the quadrotor comprehensive mathematical model has been derived, which includes gyroscopic and drag effects and motor dynamics. Also, the wind gust model is implemented in the system to make a comprehensive test for the robustness and adaptability of the controller under variable environmental disturbance. In the next Chapter, the proposed architectural framework is detailed.

CHAPTER 4: PROPOSED NAVIGATION CONTROL ARCHITECTURE

4.1. Introduction

As a starting point, in this Chapter, the classical PID controller, Linear-Quadratic Regulator (LQR) controller, and Model-Reference Adaptive Controller (MRAC) are derived for the mathematical quadrotor model provided in Section 3.1. The chosen quadrotor parameters are adopted from published literature [114], [115].

Table 4.1: UAV and Controller Parameters

<i>Param.</i>	<i>Value</i>	<i>Name</i>
m	1.04 kg	Total mass
l	0.2 m	Quadrotor arm lenght
I_x	$7.691 \times 10^{-3} \text{ kgm}^2$	Total moment of inertia along x_B
I_y	$7.691 \times 10^{-3} \text{ kgm}^2$	Total moment of inertia along y_B
I_z	$15.254 \times 10^{-2} \text{ kgm}^2$	Total moment of inertia along z_B
I_{r_x}	$1.406 \times 10^{-5} \text{ kgm}^2$	Rotor moment of inertia along x_{rot}
I_{r_y}	$1.406 \times 10^{-5} \text{ kgm}^2$	Rotor moment of inertia along y_{rot}
I_{r_z}	$2.813 \times 10^{-5} \text{ kgm}^2$	Rotor moment of inertia along z_{rot}
k	1.149×10^{-5}	Thrust coefficient
b	2.690×10^{-8}	Rotor reaction drag coefficient
K_I	$5 \times 10^{-3} \text{ kgs}^{-1}$	Induced drag coefficient
K_T	$1 \times 10^{-3} \text{ kgs}^{-1}$	Traslational drag coefficient
K_p	$2 \times 10^{-3} \text{ kgs}^{-1}$	Profile drag coefficient
K_{par}	$7 \times 10^{-4} \text{ kgs}^{-1}$	Parasitic drag coefficient
K_{P_ξ}	$diag(6, 6, 10)$	Position proportional controller gain
K_{I_ξ}	$diag(4, 4, 8)$	Position integral controller gain
K_{D_ξ}	$diag(4, 4, 6)$	Position derivative controller gain
K_{P_η}	$diag(5, 5, 5)$	Attitude proportional controller gain
K_{I_η}	$diag(2, 2, 0.5)$	Attitude integral controller gain
K_{D_η}	$diag(2, 2, 2)$	Attitude derivative controller gain

The MATLAB/Simulink environment has been used for the presented simulation studies. While Simulink offers a useful tool for PID controller tuning, challenges arise due to difficulties in linearizing the plant in MATLAB/Simulink. Therefore, at first, manual tuning of PID parameters is performed. The quadrotor and the manually tuned controller parameters are reported in Table 4.1. The derived PID controller is then combined with the DDPG algorithm to fine-tune the manually tuned controller gains and also to estimate the controller gains. A Neural Network (NN) is designed for the aforementioned tasks. In addition, reward functions are specified as explained in the rest of this chapter.

Tuning the controller is quite a challenging issue for the underlying testbed, that is, the quadrotor with motor dynamics included. The proposed estimation approach is applied for PID, LQR, and MRAC controller parameters, and this allows for comparative studies and performance evaluation. Changes in the learning approach for each controller are also reported in this chapter.

4.2. Conventional Controllers

In order to achieve stable and accurate control of a quadrotor, several conventional control strategies have been widely employed for both research and practical application purposes. This section provides an overview and comparison of three commonly used controllers Proportional-Integral-Derivative (PID), Linear Quadratic Regulator (LQR), and Model Reference Adaptive Control (MRAC). Each of the three controllers offers distinct ways to address challenges associated with quadrotor control, such as managing nonlinear dynamics, external disturbances, and trajectory tracking. Subsequent sub-sections present a detailed study and exploitation of these controllers, examining their design principles, implementation methodologies, and effectiveness in ensuring robust quadrotor performance.

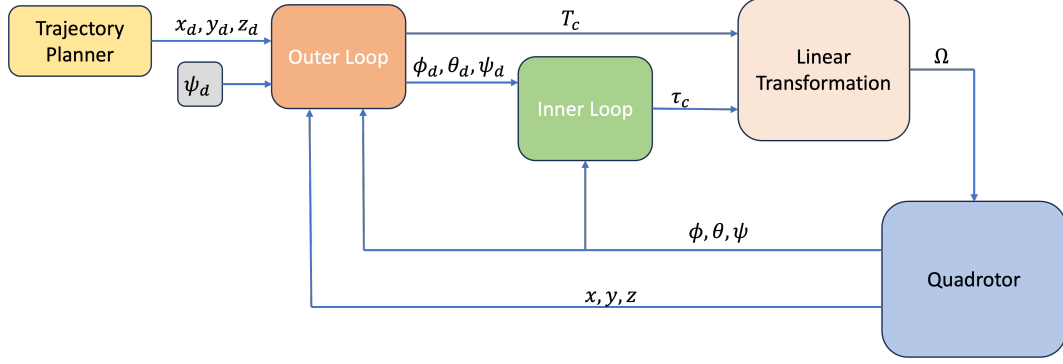


Figure 4.1: Hierarchical controller structure.

4.2.1. PID Controller

PID controllers have been widely adopted and used for multirotor UAV navigation and control because: they reduce errors and accumulated past errors (preventing steady-state errors); they improve system stability; they anticipate future errors by considering the rate of change, and they enhance the ability to respond quickly to disturbances [116].

For underactuated systems, like quadrotor UAVs, a hierarchical control configuration is employed, which is composed of an outer loop (for position control), and an inner loop (for attitude control). The outer loop produces thrust and the desired Euler angles, while the inner loop provides the desired torques as shown in Fig 4.1. Control actions are derived following the N-E formulation.

Outer Loop: The outer loop PID control law is formulated as

$$\dot{v} = \dot{v}_d + K_{P_\xi}(\xi_d - \xi) + K_{I_\xi} \int (\xi_d - \xi) dt + K_{D_\xi}(v_d - v). \quad (4.1)$$

with K_{P_ξ} , K_{I_ξ} and K_{D_ξ} being 3×3 control gain diagonal matrices that will be tuned. Using the same approach as in [117] and [118], the virtual control signal $U = \dot{v} = (U_1, U_2, U_3)^T$ is considered, and a second virtual control signal is derived

compensating for the drag force component

$$U^* = U + \frac{1}{m}RD_b \quad (4.2)$$

Following the derivation in [119], the desired pitch, desired roll, and control thrust are given by, respectively

$$\theta_d = \arctan \left(\frac{U_1^* \cos(\psi_d) + U_2^* \sin(\psi_d)}{U_3^* + g} \right) \quad (4.3)$$

$$\phi_d = \arcsin \left(\frac{U_1^* \sin(\psi_d) - U_2^* \cos(\psi_d)}{\sqrt{U_1^{*2} + U_2^{*2} + (U_3^* + g)^2}} \right) \quad (4.4)$$

$$\begin{aligned} T_d = & m((s_\theta c_\phi c_\psi + s_\phi s_\psi)U_1^* + \\ & + (s_\theta s_\psi c_\theta - s_\phi c_\psi)U_2^* + c_\phi c_\theta(U_3^* + g)) \end{aligned} \quad (4.5)$$

Inner Loop: The inner loop control law is designed as a PID controller with feedback linearization. The dynamic compensation includes the aerodynamic drag torques and is given as

$$\tau_d = I_f y + \omega^B \times I_f \omega^B + G_a - \tau_D \quad (4.6)$$

where y is the virtual control vector defined as

$$\begin{aligned} y = & \dot{\omega}_d^B + K_{P_\eta}(\eta_d - \eta) + \\ & + K_{I_\eta} \int (\eta_d - \eta) dt + K_{D_\eta}(\dot{\eta}_d - \dot{\eta}) \end{aligned} \quad (4.7)$$

and K_{P_η} , K_{I_η} and K_{D_η} being 3×3 control gain diagonal matrices to be tuned. Consequently, the proposed control scheme includes a cascaded PID controller with feedback linearization.

4.2.2. LQR Controller

The LQR controller is an optimal controller that is designed for dynamic systems by minimizing a cost function that balances state error and control effort. It is particularly suitable for multivariable systems, though determining the control input vector may be challenging due to the iterative nature of the tuning process [120]. For quadrotors, a hierarchical configuration similar to the PID controller setup is used, where the outer loop handles position control and the inner loop manages attitude stabilization. For the purposes of this dissertation, the outer loop structure from the PID controller is retained, while implementing an LQR-based control law in the inner loop.

Outer Loop: The outer loop control law follows the one as defined in (4.3)-(4.5), which provides the desired Euler angles and thrust by processing the position error. This layer handles translation control of the quadrotor, producing virtual control signals that act as inputs to the inner loop.

Inner Loop: The inner loop is redesigned using an LQR controller to regulate the attitude of the quadrotor. The control objective is to minimize a quadratic cost function, defined as

$$J = \int_0^\infty (x^T Q x + u^T R u) dt \quad (4.8)$$

where x represents the state vector of rotational dynamics (including the roll, pitch, and yaw rates), u is the control input vector (desired torques), and Q and R are weight matrices that reflect the trade-off between the state error and control effort.

In this setup, the state vector $x = [\eta \ \dot{\eta}]^T$ is composed of the attitude angles and angular rates, while $u = \tau_d$ represents the control torques required to stabilize

the quadrotor's orientation. The system dynamics are linearized around the hover condition, and the resulting state-space representation is:

$$\dot{x} = Ax + Bu \quad (4.9)$$

where A and B are system matrices obtained through linearization of the rotational dynamics.

The optimal control law is derived by considering that:

$$u = -Fx \quad (4.10)$$

where $F = R^{-1}B^TK$ is the gain matrix, and K is the solution of the algebraic Riccati differential equation:

$$\dot{K} = A^TK + KA - KBR^{-1}B^TK + Q \quad (4.11)$$

Using this feedback control law, the LQR controller computes the required torques τ_d to drive the attitude dynamics to the desired state. The weights Q and R are tuned to balance the response time and control effort, enhancing stability and disturbance rejection in the quadrotor's attitude control.

Under this framework, the control strategy combines a PID-based outer loop for position control with an LQR-based inner loop for attitude stabilization, offering a structured approach to managing the quadrotor's translational and rotational dynamics, effectively.

4.2.3. Model-Reference Adaptive Control)

Model-Reference Adaptive Control (MRAC) is an adaptive control strategy designed to handle systems with uncertainties by dynamically adjusting control param-

eters to match a reference model's response. MRAC is particularly advantageous for systems like quadrotors, where parameter variations and external disturbances are prevalent [121]. Under this setting, the MRAC controller is used in the inner loop for attitude control, while the outer loop remains the same as in the PID controller setup to manage position.

Outer Loop: The outer loop PID control law is kept, as defined in (4.3)-(4.5), which generates the desired Euler angles and thrust based on position errors. This outer layer controls the quadrotor's translation and produces virtual control signals that act as reference inputs to the inner loop.

Inner Loop: The inner loop employs an MRAC controller to regulate the quadrotor's attitude by matching it to a desired reference model. The MRAC design begins with selecting a reference model that describes the desired closed-loop behavior for the attitude dynamics. The reference model can be represented as:

$$\dot{x}_m = A_m x_m + B_m r \quad (4.12)$$

where x_m is the state vector of the reference model, r is the command input (derived from the outer loop), and A_m and B_m are model parameters chosen to reflect the desired dynamics.

The quadrotor's inner-loop dynamics are represented as:

$$\dot{x} = Ax + Bu \quad (4.13)$$

where $x = [\eta \ \dot{\eta}]^T$ represents the actual attitude and angular rates, and $u = \tau_d$ is the control torque input.

The adaptive control law is designed as:

$$u = \theta^T \phi(x, r) \quad (4.14)$$

where θ is an adaptive parameter vector that adjusts over time to minimize the error $e = x - x_m$, and $\phi(x, r)$ is a vector of basis functions that depend on the states and on the command input.

The parameter adaptation law is derived to minimize the error between the reference model and the actual system response, typically using the following form:

$$\dot{\theta} = -\Gamma \phi(x, r) e^T P B \quad (4.15)$$

where Γ is a positive definite adaptation gain matrix, and P is the solution to the Lyapunov equation:

$$A_m^T P + P A_m = -Q \quad (4.16)$$

with Q as a positive definite matrix.

Through this adaptive control scheme, the MRAC controller continuously adjusts to minimize the tracking error e , enhancing the quadrotor's ability to reject disturbances and to adapt to dynamic changes in system parameters. The combination of a PID-based outer loop for position control and an MRAC-based inner loop for attitude stabilization provides a robust and adaptive solution for controlling the quadrotor's translational and rotational dynamics under varying conditions.

4.3. Learning-Based Parameter Estimation Approaches

This section details learning-based approaches that relate to UAV parameter estimation. To be specific, in this dissertation, the DDPG, an RL technique, is chosen for parameter estimation - note that RL algorithms follow the trial-and-error principle.

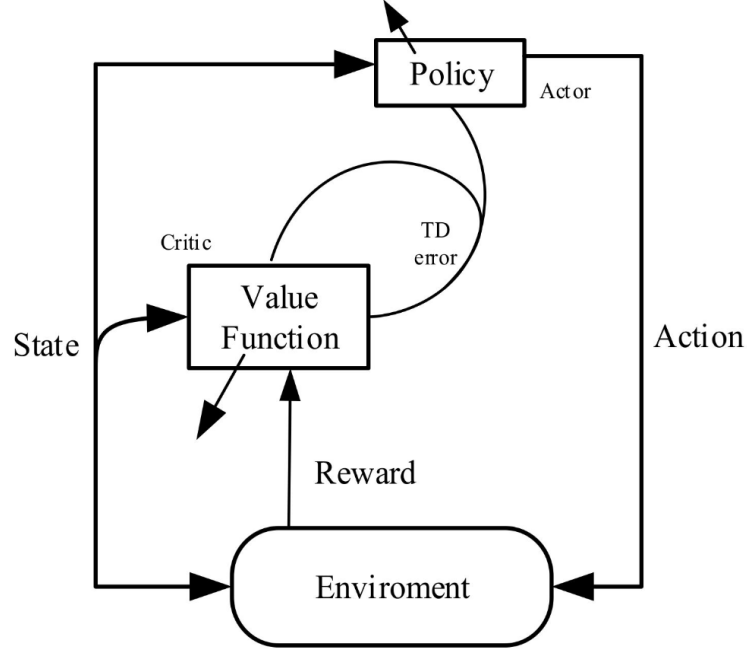


Figure 4.2: The block diagram of DDPG algorithm [123].

DDPG is an off-policy actor-critic algorithm. According to Silver et al. [122], the deterministic policy gradient may be estimated more efficiently compared to the stochastic policy one, and it may need less samples than the stochastic policy gradient if the action space is high dimensional. The DDPG algorithm may need less computational cost and less learning process time. The general scheme of the DDPG algorithm is provided in Fig. 4.2.

4.3.1. PID Controller Gains Tuning and Estimation

It is essential to properly adjust controller gains as this directly impacts performance. For example, for a PID controller, an excessively high proportional gain may lead to overshooting, an insufficient integral gain may result in a steady-state error, and a very high derivative gain may lead to instabilities. Thus, achieving an 'optimal balance' among the three gains is crucial to ensure a PID controller's accurate and stable response (when applied to different dynamic systems). This situation is also valid for other controllers. However, the constant controller parameter may not

work for the multirotor UAV under different environmental disturbances, Therefore, it is advantageous to estimate the controller gains online for the UAV's control and navigation task.

When an agent is trained to estimate the controller parameters with complex mathematical model and environmental disturbances, the agent can tune and estimate the control gains online during the mission. However, in this initial phase, motor dynamics and wind disturbances are excluded from the simulation environment during training. These factors, along with the full mathematical model of the quadrotor and environmental disturbances, are introduced comprehensively in the second phase of the study.

Controller Gains Tuning

The starting step is a manual tuning of controller parameters. This is followed by employing RL to further refine manual tuning. The block diagram configuration of controller parameter tuning using RL is illustrated in Fig. 4.3. The overall system consists of three components: controller, plant (quadrotor), and agent (RL component). The overall configuration of Fig. 4.3 includes a trajectory planner, the linear transformation (3.5) and parameter tuning parts. The notation has already been previously explained.

The state space S (for tuning) includes positions (ξ) and velocities ($\dot{\xi}$) along the x, y, and z axes, as well as Euler angles (η) and their corresponding rates ($\dot{\eta}$). It also includes position and velocity errors ($e_\xi, e_{\dot{\xi}}$) along the x, y, and z axes, errors in Euler angles and their rates ($e_\eta, e_{\dot{\eta}}$). The agent learns to dynamically adjust the normalized weights ($n_{P_\eta}, n_{I_\eta}, n_{D_\eta}$), which create the action space ($A \in \mathbb{R}^9$) within the range of $[-1, 1]$ for all controller parameters. The state and action spaces are denoted as follows

$$\begin{aligned} S &= [\xi, \dot{\xi}, \eta, \dot{\eta}, e_\xi, e_{\dot{\xi}}, e_\eta, e_{\dot{\eta}}] \in \mathbb{R}^{24} \\ A &= [n_{P_\eta}, n_{I_\eta}, n_{D_\eta}] \in \mathbb{R}^9 \end{aligned} \tag{4.17}$$

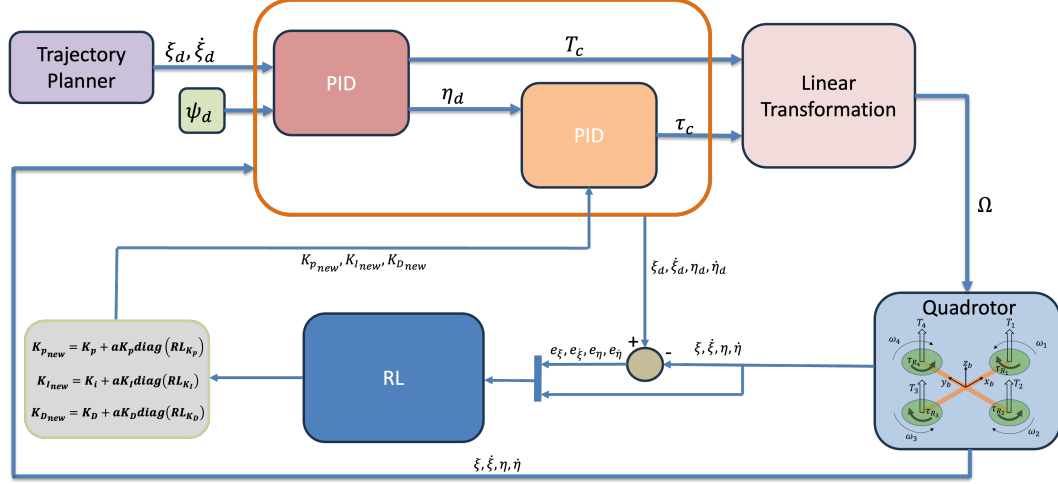


Figure 4.3: Block diagram of the PID parameter tuning based on RL.

Considering the normalized weights, the corresponding parameter-tuning equations are given by

$$\begin{aligned}
 K_{P_{\eta},new} &= K_{P_{\eta}} + a n_{P_{\eta}} K_{P_{\eta}} \\
 K_{I_{\eta},new} &= K_{I_{\eta}} + a n_{I_{\eta}} K_{I_{\eta}} \\
 K_{D_{\eta},new} &= K_{D_{\eta}} + a n_{D_{\eta}} K_{D_{\eta}}
 \end{aligned} \tag{4.18}$$

where $K_{P_{\eta},new}, K_{I_{\eta},new}, K_{D_{\eta},new}$ are the RL fine-tuned proportional, integral, and derivative gains (parameters), respectively. a is the search rate with an assigned value of 0.5. The manually tuned inner loop controller parameters serve as the "base" gains. The combination of the search rate and the normalized weights helps tune controller parameter values within the interval of $[-50\%, 50\%]$ relative to the base controller gains.

Since RL is then applied to further tune the inner loop controller gains, the reward function is defined as a function of the attitude error norm. Considering simulation studies after manually tuned gains, it is observed that the norm of the attitude error ranges from 10^{-4} to 10^{-2} rads. Given this relatively wide range, the reward function is set as a piece-wise function of the attitude error norm as shown in

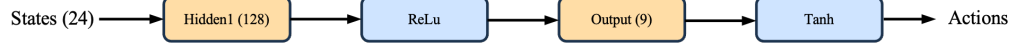


Figure 4.4: Structure of the actor neural network used for PID parameter tuning.

$$(4.19) \quad R(e_\eta) = \begin{cases} -25, & 0.04 < \|e_\eta\| \\ -15, & 0.02 < \|e_\eta\| \leq 0.04 \\ -10, & 0.01 < \|e_\eta\| \leq 0.02 \\ -5, & 0.005 < \|e_\eta\| \leq 0.01 \\ -1, & 0.001 < \|e_\eta\| \leq 0.005 \\ 10, & \|e_\eta\| \leq 0.001 \end{cases} \quad (4.19)$$

The choice of the highest critical value as 0.04 rad reflects the observation that the norm of the attitude error surpasses this threshold within (the first) 5s.

For training, the complete N-E model-based multirotor UAV shown in (3.27)-(3.30) is used. Given the considerable duration of the training process, the simulation period is set to be 20s. This does not affect training as it has been observed that the dynamics stabilize much sooner. The criterion to complete training is based on achieving an average reward. The target average reward value is determined by analyzing the distribution of step counts across the error intervals shown in (4.19) relative to the norm of the attitude error observed with the manually tuned parameters. The step count distribution returns the values of [16, 31, 13, 5, 67, 269]. Using this distribution, the target average reward value is computed as 1603. To enhance performance, the average reward value is then adjusted to 1843, which represents a 15% increase over the baseline average reward of 1603.

The underlying actor and critic neural network (NN) structures are depicted in Fig. 4.4 and 4.5, respectively. The *tanh* activation function is used in the output of the actor, and the agent output is set between $[-1, 1]$. The training phase parameters to tune controller gains based on RL are shown in Table 4.2.

Table 4.2: Training Parameters

Parameters	Tuning	Estimation
Sampling time	0.05	0.05
Reward discount factor γ	0.99	0.99
Learning rate for actor	10^{-3}	10^{-3}
Learning rate for critic	10^{-3}	10^{-3}
L_2 Regularization factor	10^{-5}	10^{-4}
Optimizer parameter ϵ	10^{-8}	10^{-8}
Minimum batch size	1024	64
Experience buffer length	10^6	10^6

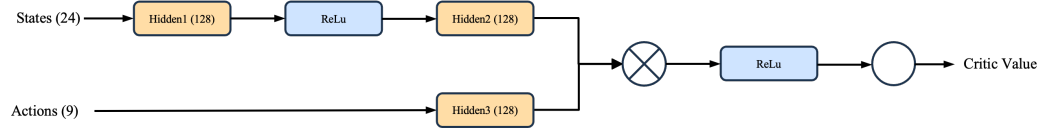


Figure 4.5: Structure of the critic neural network used for PID parameter tuning.

The next section details PID parameter estimation based on RL.

PID Controller Gains Estimation

The aim is to estimate the inner loop attitude controller parameters in this section. A DDPG algorithm is, again, used for the estimation process. The block diagram configuration of the RL-based parameter estimation is shown in Fig. 4.6.

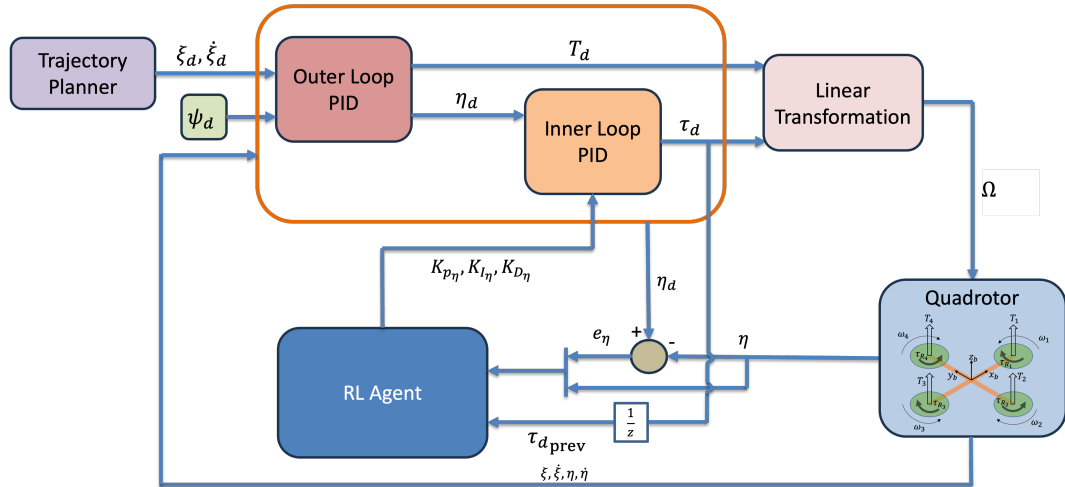


Figure 4.6: Block diagram of PID parameter estimation based on RL.

The agent is trained to estimate the PID controller parameters. In this case, for this task, the state space (S) includes attitude angles and corresponding errors, while the action space (A) consists of the diagonal elements of the inner loop proportional, integral, and derivative gain matrices ($K_{P_\eta}, K_{I_\eta}, K_{D_\eta}$). The state space is represented as six-dimensional, and the action space as nine-dimensional, as follows

$$\begin{aligned} S &= [\eta, e_\eta] \in \mathbb{R}^6 \\ A &= [K_{P_\eta}, K_{I_\eta}, K_{D_\eta}] \in \mathbb{R}^9 \end{aligned} \tag{4.20}$$

where k denotes ϕ , θ , and ψ .

The reward function for parameter estimation has the following form

$$R = -\|e_\eta\|^2 - 0.1\|\tau_{d_{prev}}\|^2 - 250D \tag{4.21}$$

where $\tau_{d_{prev}}$ denotes the previous control action of the inner loop controller, and D denotes the Done function, defined as

$$D = \begin{cases} 1, & |e_{\xi_i}| \geq 1 \vee |e_{\eta_k}| \geq 1 \wedge t < T_f \\ 0, & otherwise \end{cases} \tag{4.22}$$

where, t is the simulation time, T_f is the final time, i denotes x, y or z, and k represents ϕ , θ or ψ . This implies that the D function is activated once, if and only if $|e_{\xi_i}|$ or $|e_{\eta_k}|$ surpass the given thresholds before the episode is completed.

In RL, the exploration and exploitation dilemma is a fundamental problem. An agent improves action selection over time by balancing the exploration of new options with the exploitation of known strategies based on past experience [15],[83]. During the training process, the Done function acts as a termination condition for each episode. Without using D in the reward function, the agent receives a higher reward when it fails at the beginning of an episode compared to failing after 2s.

This causes the aforementioned dilemma. To resolve the dilemma, the used Done function prevents the agent from repeating the same failing output. In this manner, the reward function is designed for punishment and behaves as a cost function. Thus, the long-term aim of the agent is for the reward to converge to zero.



Figure 4.7: Structure of the actor neural network used in PID parameter Estimation.

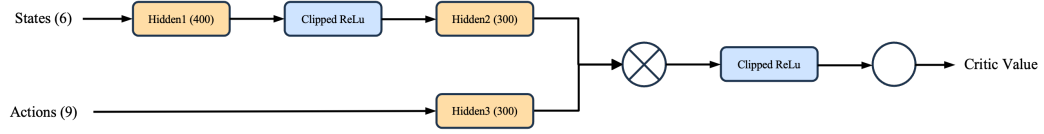


Figure 4.8: Structure of the critic neural network in PID parameter estimation.

Training phase parameters for controller gain estimation are shown in Table 4.2. The actor and critic NN structures are displayed in Fig. 4.7 and 4.8, respectively. A clipped ReLu activation function is used in the output of the actor NN, thus, the agent estimates the controller gains in the set range of $[0, 21000]$.

To create a more realistic simulation environment, motor dynamics are incorporated into the system, with (3.31) implemented immediately after the linear transformation equations shown in Fig. 4.6. Due to the propeller speed behavior during initial movements (illustrated in Figs. 4.9 and 4.10), the learning process exhibits instability with the current neural network (NN) and reward function. To address this challenge, the state space, reward function, and a few hyper-parameters are revised for the PID controller parameter estimation task.

Also, to realistically simulate environmental disturbances, the wind gust model in (3.32) has been considered. This revised model introduces wind gusts with defined onset, duration, and peak velocity, impacting each axis of motion (x, y, and z) independently. In this setup, the gust is initiated at the 20th second ($t_i = 20$ s) and remains active for $T_\omega = 7$ seconds, with peak velocities for each axis set to $V_{\omega_{max}} = [8, -9, -5]$ m/s. This intermittent wind gust applies cosine-shaped velocity

profiles on each axis during this interval, providing a time-varying disturbance that aligns with the natural dynamics of the quadrotor flight.

The wind velocities along the x, y, and z axes over time are illustrated in Fig. 4.11, highlighting the sinusoidal variations imposed by the wind gust model during the active disturbance period. The resulted wind speed magnitude is displayed in Fig. 4.11, combining the velocities on all axes to reflect the overall gust intensity the quadrotor encounters. By incorporating this structured disturbance profile into the simulation studies, the model allows for accurate assessment of the controller's response under realistic and sudden wind changes. This also allows for testing the robustness of the control algorithms, as they must maintain stability and performance

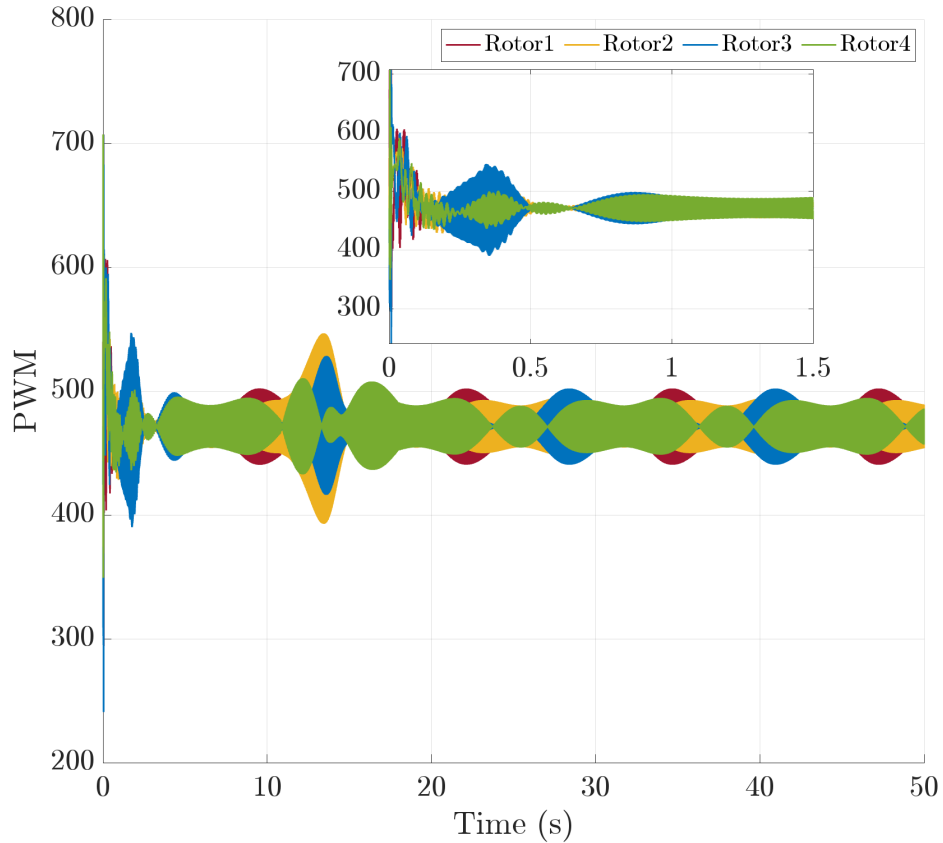


Figure 4.9: Change of the rotor speeds without motor dynamics.

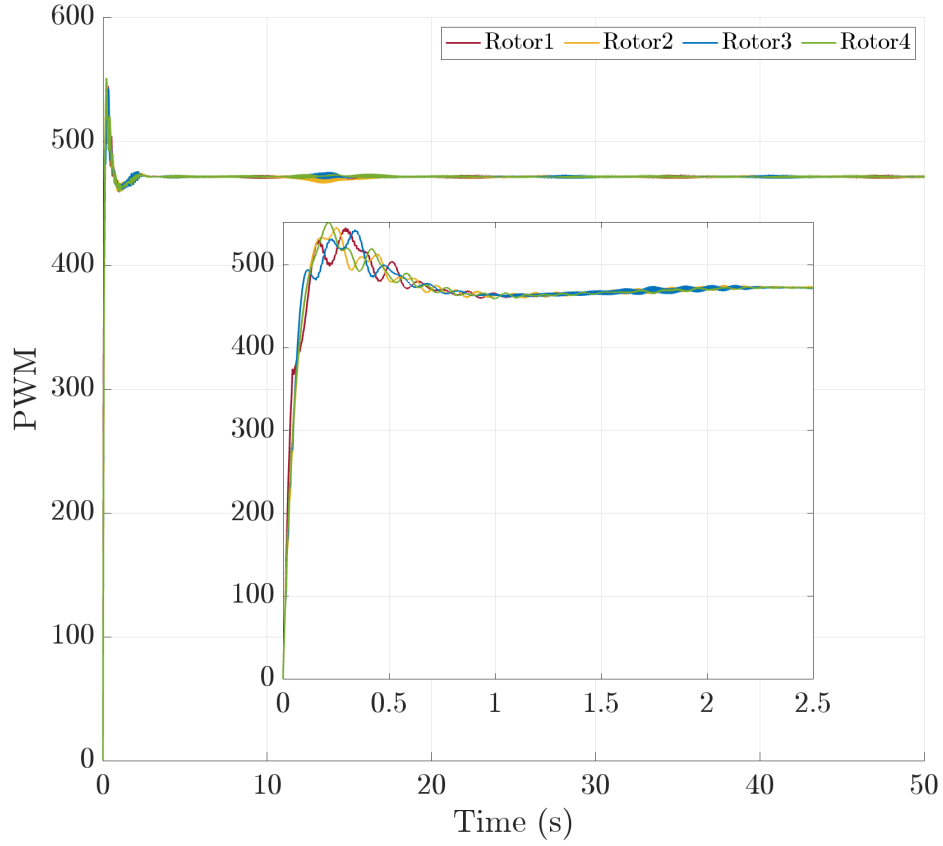


Figure 4.10: Change of the rotor speeds with the motor dynamics.

despite fluctuations in aerodynamic conditions. This wind disturbance is used during training and testing of the PID, LQR, and MRAC controller parameter estimation.

The updated block diagram configuration for RL-based parameter estimation, incorporating motor dynamics and environmental disturbances, is shown in Fig. 4.12.

While the action space does not change, the updated state space is provided as follows:

$$\begin{aligned}
 S &= [\xi, \dot{\xi}, \eta, e_{\xi}, e_{\dot{\xi}}, e_{\eta}, \Omega] \in \mathbb{R}^{22} \\
 A &= [n_{P_{\eta}}, n_{I_{\eta}}, n_{D_{\eta}}] \in \mathbb{R}^9
 \end{aligned} \tag{4.23}$$

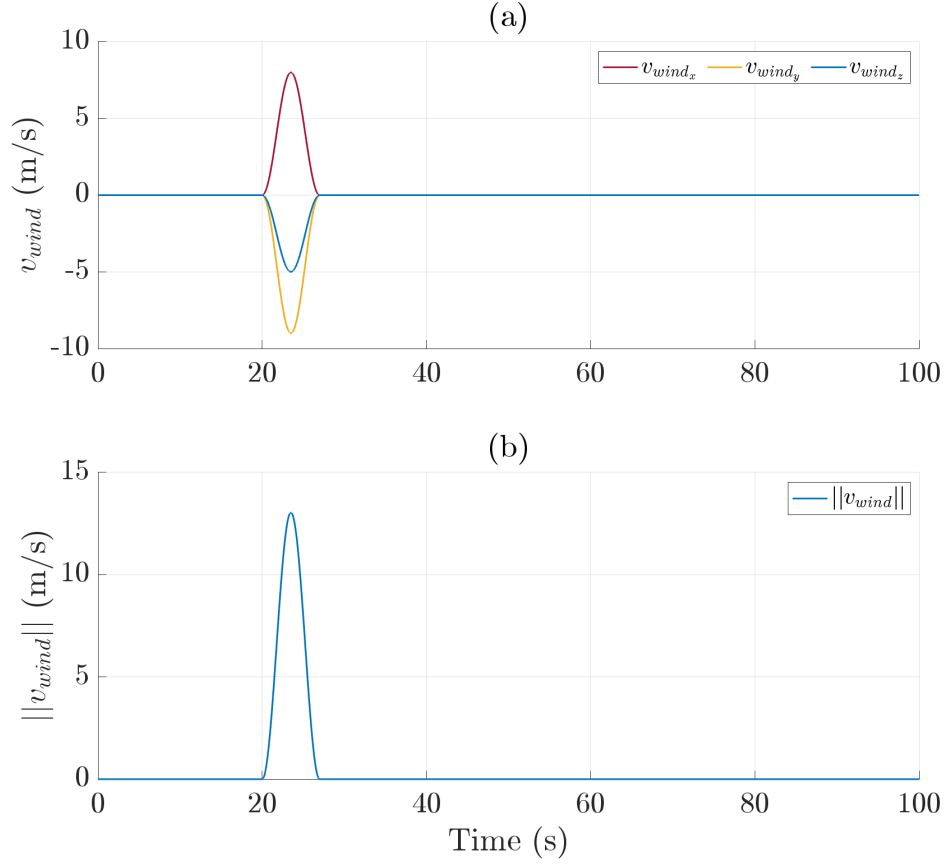


Figure 4.11: (a) Wind velocities on x, y, and z axes in time, and (b) The magnitude of wind in time.

To accommodate the added complexity of motor dynamics and environmental disturbances, the reward function has been revised to enhance the learning process under these conditions. The updated reward function is given by

$$R = -\|e_\xi\|^2 - 0.1\|e_\eta\|^2 - 0.1\|e_\xi\|^2 - 20(20 - t)D \quad (4.24)$$

where $\|e_\xi\|^2$ penalizes the position error, $\|e_\eta\|^2$ penalizes the attitude error, and the additional term $\|e_\xi\|^2$ penalizes the velocity error, which becomes significant with the inclusion of motor dynamics. This modification introduces a direct feedback mech-

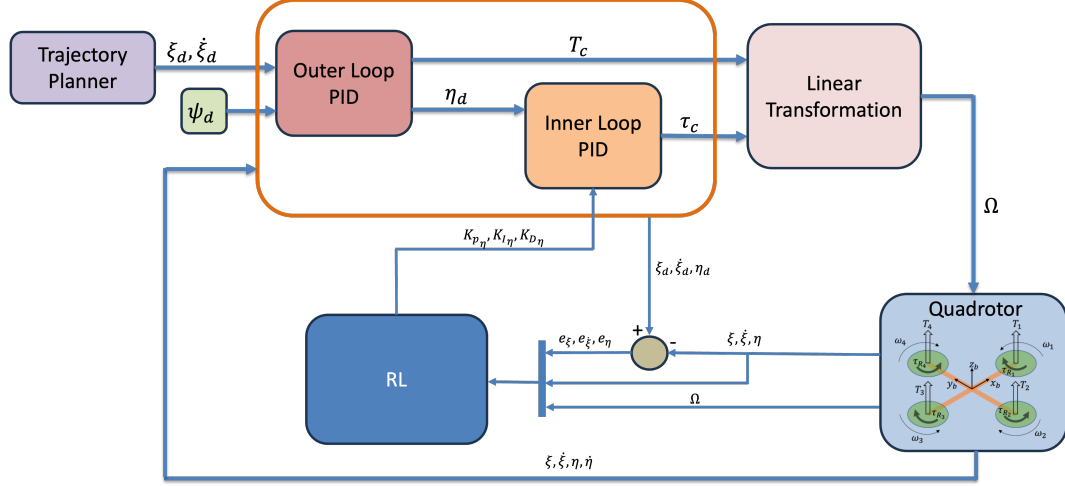


Figure 4.12: Block diagram of PID parameter estimation based on RL.

anism to control and optimize velocity deviations, helping the agent better manage disturbances from wind gusts and other dynamic effects.

The Done function in (4.22), D , is not changed, but its usage in the reward function for penalizing early episode termination is adjusted. The term $20(T_f - t)D$ imposes a scaled penalty based on the remaining time to the final simulation time, T_f . This change strongly discourages conditions that trigger the Done function, which is activated if the position or attitude errors exceed specified thresholds before the episode concludes. This scaled penalty emphasizes stability and encourages the agent to maintain control even under disturbances, promoting a smoother learning trajectory in the presence of motor dynamics and environmental factors.

Table 4.3 presents the training of the hyper-parameters used for configurations both with and without motor dynamics (MD) and environmental disturbances (EM).

In the following sections, the same comprehensive mathematical model, incorporating motor dynamics and wind disturbances, is used.

Table 4.3: Comparison of hyperparameters for PID controller parameter estimation task.

Parameters	Without MD & ED	With MD & ED
Sampling time	0.05	0.05
Reward discount factor γ	0.99	0.99
Learning rate for actor	10^{-3}	10^{-3}
Learning rate for critic	10^{-3}	10^{-3}
L_2 Regularization factor for actor	10^{-4}	10^{-4}
L_2 Regularization factor for critic	10^{-4}	10^{-5}
Optimizer parameter ϵ	10^{-8}	10^{-8}
Minimum batch size	64	256
Experience buffer length	10^6	10^6

4.3.2. LQR Controller Parameter Estimation

In this section, the aim is to estimate the LQR controller parameters using the same approach that is used in the PID parameter estimation. Although the NN structure, hyper-parameters, and reward function remain unchanged, the state space and the action space are adjusted according to the LQR controller; they are defined as follows:

$$\begin{aligned}
 S &= [\xi, \dot{\xi}, \eta, e_\xi, e_{\dot{\xi}}, e_\eta] \in \mathbb{R}^{18} \\
 A &= [\{Q_{ii}|i = 1, \dots, 6\}, \{R_{ii}|i = 1, \dots, 3\}] \in \mathbb{R}^9
 \end{aligned} \tag{4.25}$$

The action space includes the diagonal elements of the state weight matrix (Q) and the control weight matrix (R). Also, the block diagram configuration for RL-based LQR parameter estimation is shown in Fig. 4.13.

4.3.3. MRAC Parameter Estimation

The same approach is followed for the MRAC controller with the one used in the PID and LQR parameter estimation for consistency purposes. The reward function defined in (4.24) is again utilized to penalize deviations from desired performance, facilitating a uniform performance criterion across all controller types. In addition to

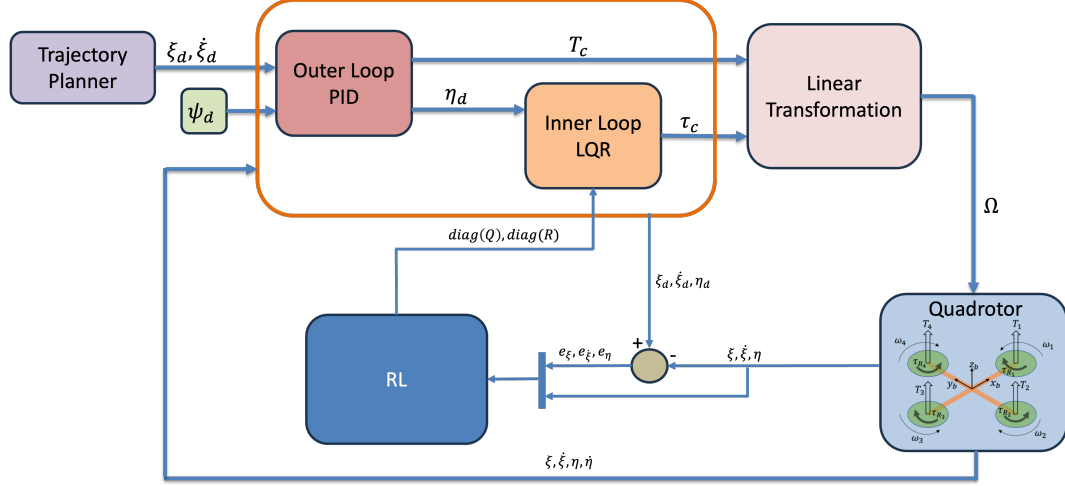


Figure 4.13: Block diagram of LQR parameter estimation based on RL.

the reward function, the neural network (NN) structure and hyper-parameters also remain unchanged, preserving the comparability of the adaptive strategies.

The parameter adaptation law's diagonal elements within the Γ matrix are the crucial parameters that are estimated. They are critical in defining the adaptive gains that adjust the control parameters to meet the system's desired response, making accurate estimation essential for optimal MRAC performance.

The block diagram configuration for RL-based MRAC parameter estimation is shown in Fig. 4.14.

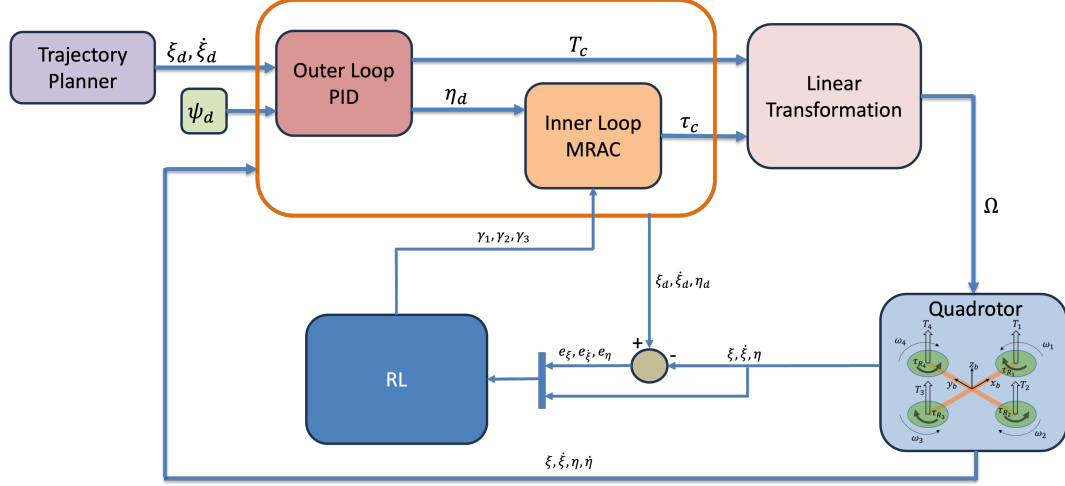


Figure 4.14: Block diagram of MRAC parameter estimation based on RL.

4.4. Estimation of Aerodynamic Uncertainties

From Section 3.1.3, the aerodynamic uncertainties (drag effects) cause additional forces and torques on the multirotor UAV model, and these forces and torques affect motion as shown in Table 3.1. The drag coefficients, provided in Table 4.1, can change according to the configuration, structure, or propeller of the multirotor. Considering performing physical aerodynamic tests, estimation of the aerodynamic uncertainties is more convenient. The drag forces and torques can be calculated using the estimated aerodynamic uncertainties (drag coefficients provided in Section 3.1.3). The forces and torques, which are calculated using the estimated drag coefficients, may be compensated using the feedback technique in the controller structure.

CHAPTER 5: RESULTS

5.1. Introduction

The proposed controller framework has been validated for stability and navigation control of the derived mathematical model in MATLAB/Simulink. The manually hand-tuned controller gains, the RL-based tuned controller gains, and the PID, LQR, and MRAC controllers with estimated gains based on RL have been tested for a helix trajectory tracking task in a simulation environment. The desired helix trajectory is illustrated in Figs. 5.1 and 5.2.

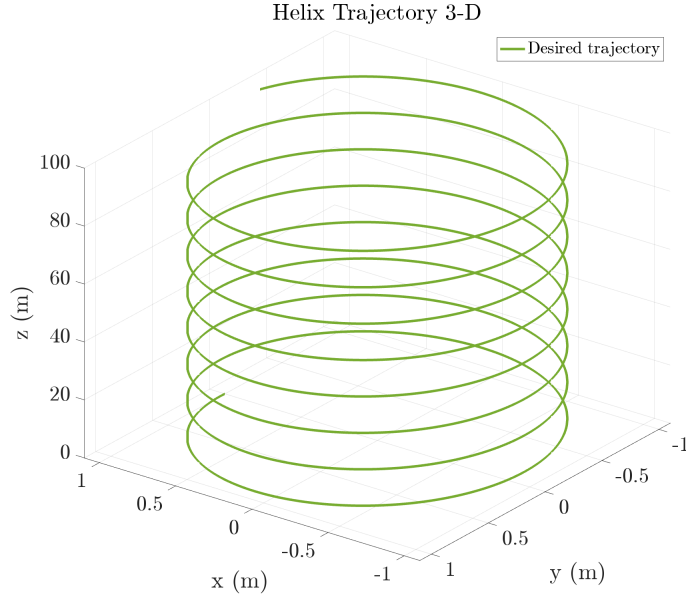


Figure 5.1: The desired helix trajectory in 3-D.

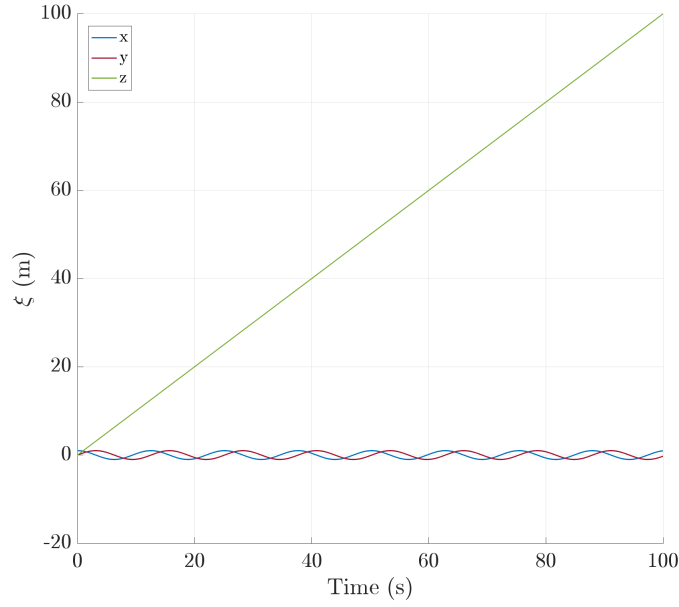


Figure 5.2: The desired helix trajectory.

5.2. Proposed Controller Frameworks Results

5.2.1. PID Controller Framework

As a first step, a hierarchical controller for the trajectory tracking is derived using PID controller in the inner and outer loop as given in Fig. 4.1. It has been attempted to tune the inner and outer controller using the automated tuning tool in Simulink, but the tuning tool linearizes the plant before tuning. The used mathematical model is non-linear and challenging to handle by the tuning tool in Simulink. Hence, the inner and outer controllers have been manually tuned. The manual-tuned PID controller position and attitude errors are shown in Figs. 5.3 and 5.4, respectively.

By using the proposed RL technique, the controller parameters are tuned to improve the trajectory tracking performance. Training runs over 20 s in each episode, and it is done in 268 episodes (around 1 hour 20 minutes). Then, the trained agent is simulated over 100 s. The trained agent tunes the inner loop controller parameters

online as shown in Fig. 5.5. The agent changes the parameters in the first 1.6s, and then all parameters become constant.

The RL-based controller parameter estimation technique is applied to the control framework. The training also runs over 20 s in each episode. This training section for estimation is completed in 11055 episodes (around 27 hours) which is much longer than the tuning. The trained agent in this part estimates the inner loop controller parameters online, but only in the first step the parameters are changed and rest of the simulation the parameters become constant as shown in Fig. 5.6.

Next, the performance of the manually tuned, RL-based-tuned, and RL-based-estimated controllers are compared. The attitude and position errors and error norms are shown in Figs. 5.7-5.10, respectively. Note that, since the outer loop produces the desired Euler angle references, the initial error of the attitude dynamics is not zero. This is more evident in the RL-based estimation agent assigns values of 256.5 and

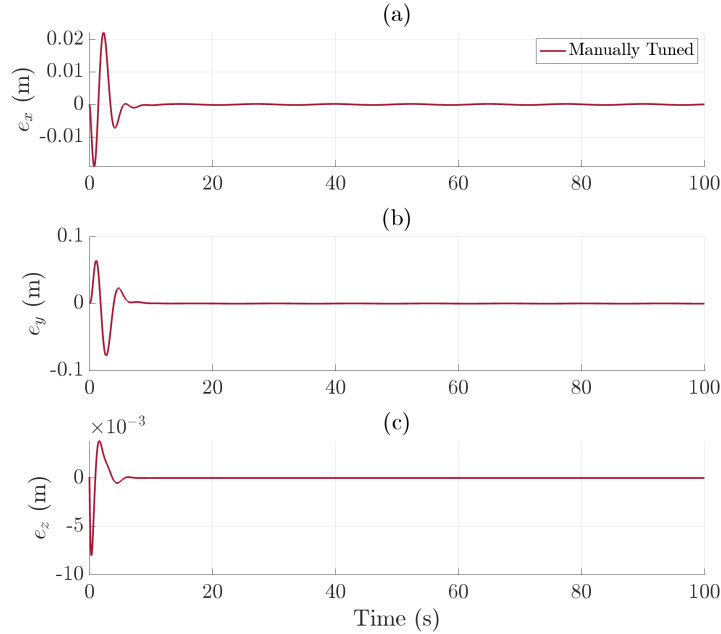


Figure 5.3: Manual-tuned PID position error in x, y, and z axes.

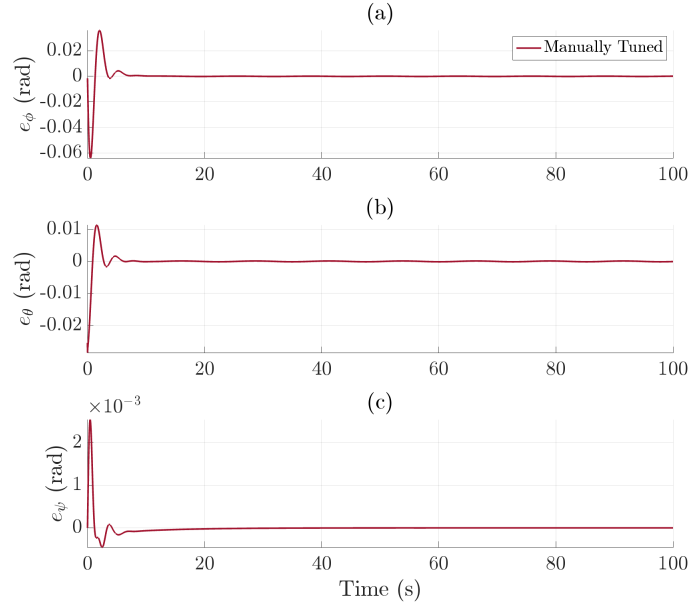


Figure 5.4: Manual-tuned PID attitude error in roll, pitch, and yaw angles.

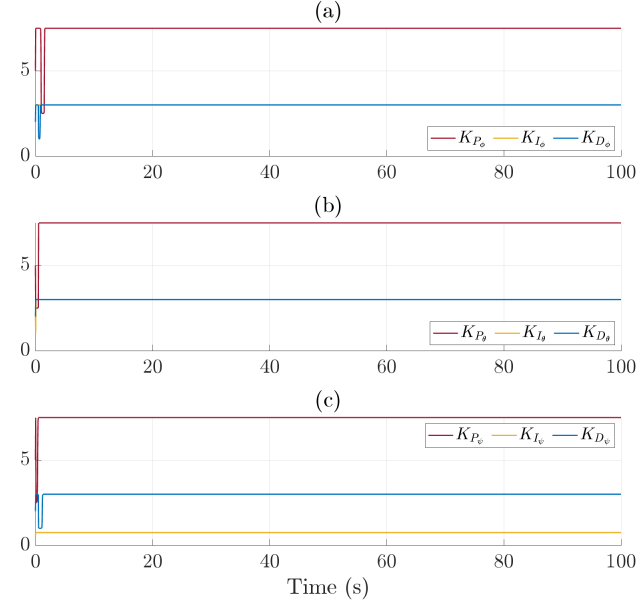


Figure 5.5: The tuned parameters based on RL in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ .

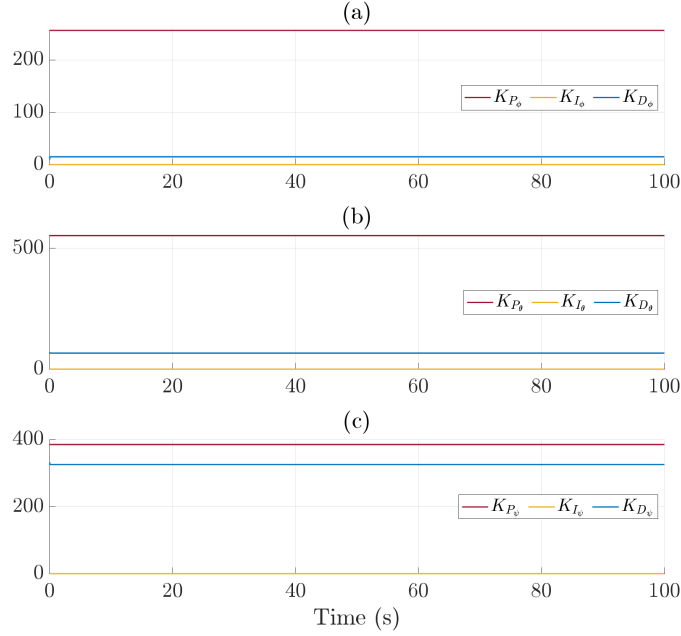


Figure 5.6: The estimated parameters based on RL in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ .

552 to K_{ϕ_p} and K_{θ_p} , respectively. These elevated values result in higher values for U_1^* and U_2^* in Eqs. (4.3) and (4.4). Consequently, the desired ϕ and θ increase, leading to higher errors in these parameters initially. However, the elevated controller gains facilitate rapid convergence.

As depicted in Fig. 5.9, the norm of the initial attitude error of the estimator is significantly higher due to the aforementioned scenario, but the error converges to zero much quicker than the error of the manually-tuned and RL-based tuned PID controllers.

The root mean square errors (RMSE) of the normalized attitude and position trajectory are calculated and the values are provided in Table 5.1.

Findings indicate that, compared to the manually tuned controller, the RL-based tuned controller achieves 20.45% smaller RMSE, while the RL-based estima-

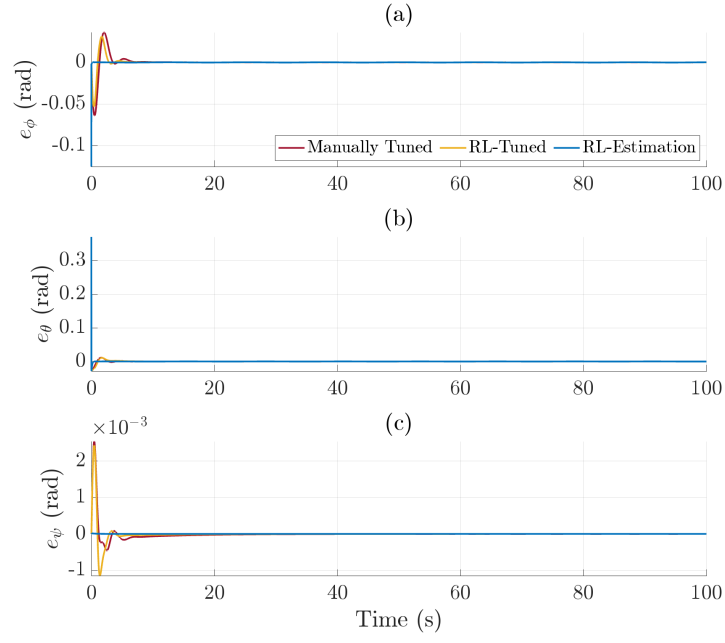


Figure 5.7: Attitude errors: (a) roll angle ϕ , (b) pitch angle θ , (c) yaw angle ψ .

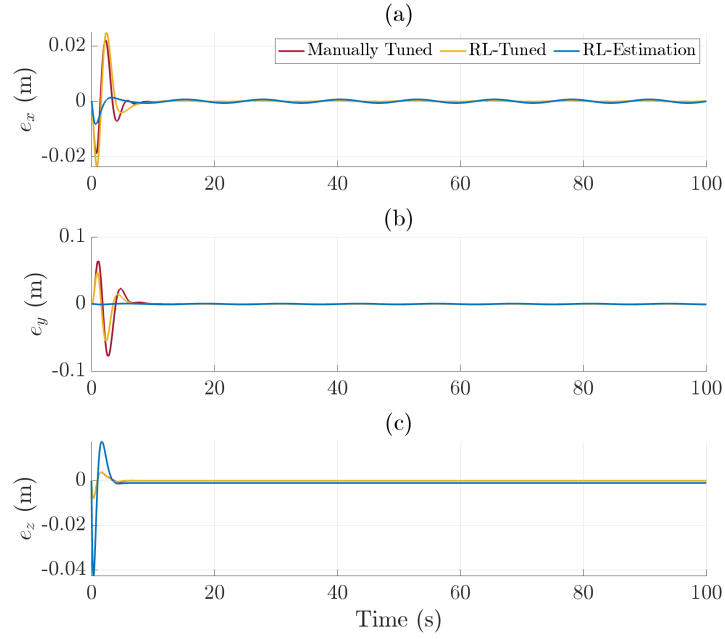


Figure 5.8: Position errors: (a) x -axis, (b) y -axis, (c) z -axis.

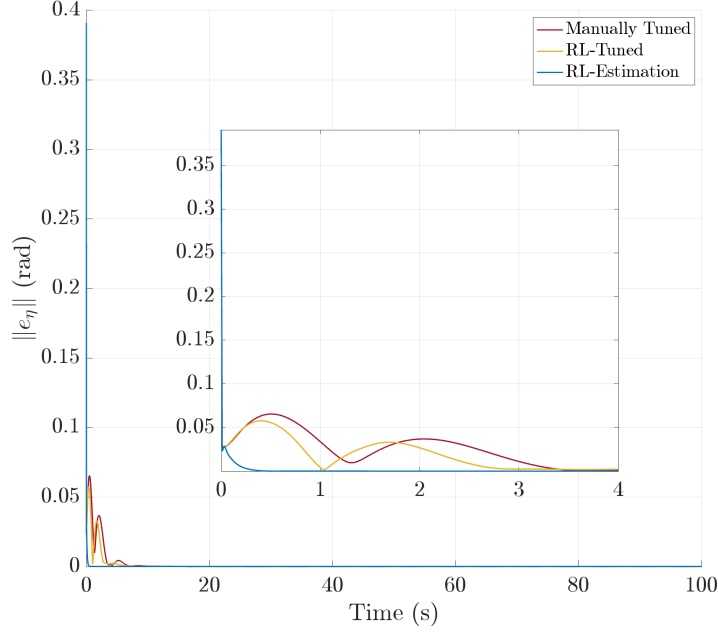


Figure 5.9: The attitude error norms of the manually tuned, RL-based tuned, and RL-based estimated controllers.

Table 5.1: RMSEs of Attitude and Position

	$RMSE_{\eta}$	$RMSE_{\xi}$
Hand-Tuned	$6.6 \times 10^{-3} \text{ rad}$	$10.3 \times 10^{-3} \text{ m}$
Tuning	$5.2 \times 10^{-3} \text{ rad}$	$7.4 \times 10^{-3} \text{ m}$
Estimation	$4 \times 10^{-3} \text{ rad}$	$3.7 \times 10^{-3} \text{ m}$

tion method results in a 39.13% performance gain in terms of the RMSE value of the attitude. Additionally, the RL-based tuned controller exhibits a 27.66% improvement over the manually tuned controller, while the RL-based estimation method performance shows a substantial 63.99% increase in the RMSE of the normalized position.

Finally, the estimated controller gains converge to a constant value, and the integral gain is set to zero by the agent. Thus, the controller effectively behaves as a PD controller. This suggests that by compensating for external disturbances (i.e., gyroscopic and aerodynamic effects) in the feedback linearization, a PD solution is able to achieve zero steady-state error. Moreover, by eliminating the integral control

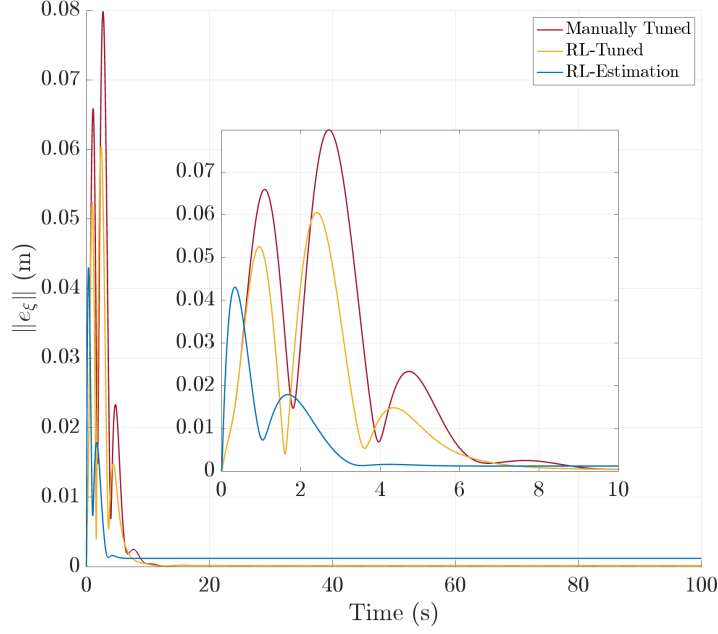


Figure 5.10: The position error norms of the manually tuned, RL-based tuned, and RL-based estimated controllers.

action, the resulting attitude dynamics show less oscillatory behavior, as verified in Figs. 5.7 and 5.9.

After this point, the simulation setting is enhanced by using motor dynamics and environmental disturbances. The RL-based PID parameter estimation framework, illustrated in Fig.4.12, is set up, and training is completed for a comprehensive scenario.

Training, in this phase of the research, also runs over 20 s in each episode. The proposed algorithm provides dynamic controller parameters; while the controller on the roll angle (ϕ) is a PID one at the beginning, it becomes a PI controller, and the controller on the yaw angle (ψ) becomes a PID controller although it was a PI controller, initially, as shown in Fig. 5.11

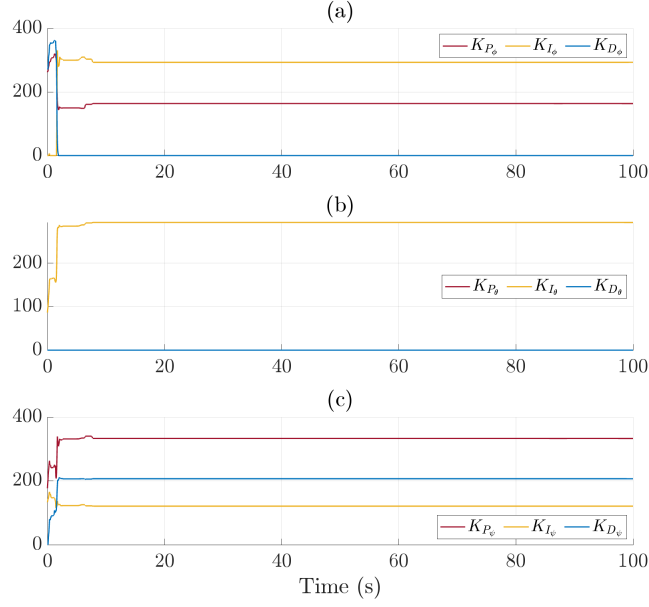


Figure 5.11: The RL-based estimated PID parameters in time: (a) The parameters for roll angle ϕ , (b) The parameters for pitch angle θ , (c) The parameters for the yaw angle ψ .

For performance evaluation, attitude and position errors and error norms are illustrated in Figs 5.12-5.15. During simulations, a wind disturbance is introduced at the 20th second, lasting for 7 seconds. This disturbance induces oscillations in the quadrotor's position along the x, y, and z axes, as observed in Fig. 5.12. While the position error along the y-axis remains relatively low, the z-axis error increases to more than 0.1 m, with the x-axis error reaching approximately 0.02 m. Overall, the position errors induced by the wind disturbance are mitigated within 8 seconds. Following the cessation of the wind disturbance at the 27th second, the errors converge back to their regular levels by around the 33rd second, as shown in Fig. 5.13.

The attitude errors show a similar profile and response, converging within 6 seconds under nominal conditions. The highest initial attitude error occurs in the pitch angle (θ). However, under wind disturbance, the roll angle (ϕ) experiences the largest error, as depicted in Fig. 5.14. The norm of the attitude error, $\|e_\eta\|$, reaches

a maximum of 0.0139 rad during the disturbance as shown in Fig. 5.15. Despite the added challenge of the wind disturbance, the proposed RL-based PID controller framework effectively attenuates the errors, successfully stabilizing the quadrotor and completing the simulation with all errors converging within acceptable limits.

5.2.2. LQR Controller Framework

The same wind disturbance scenario that was applied to the RL-based PID controller framework was also tested on the RL-based LQR controller. As with the previous controller, the position errors along the x , y , and z axes converge within 8 seconds, demonstrating a similar error recovery pattern. The wind disturbance induces oscillations in the quadrotor's position along these axes, as illustrated in Fig. 5.17. Although the wind effect ends at the 27th second, the position errors gradually return to their regular levels approximately close to the 33rd second, as shown in Fig. 5.18.

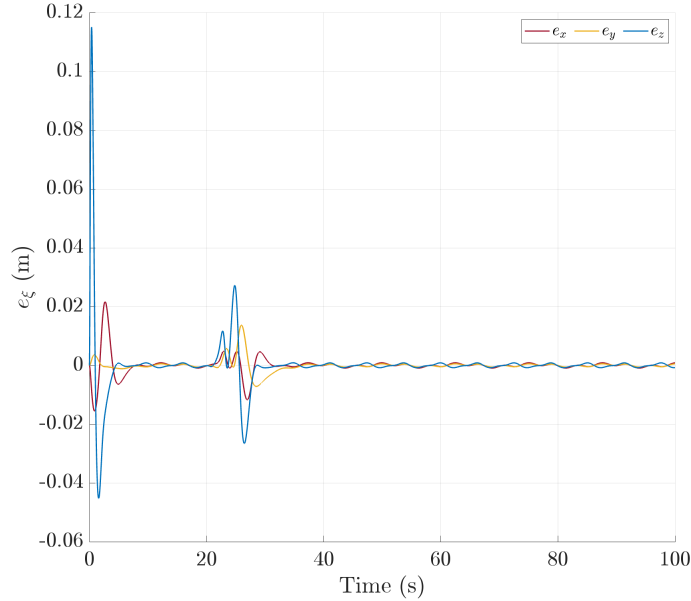


Figure 5.12: Position errors in x , y , and z axes for RL-based PID parameter estimation.

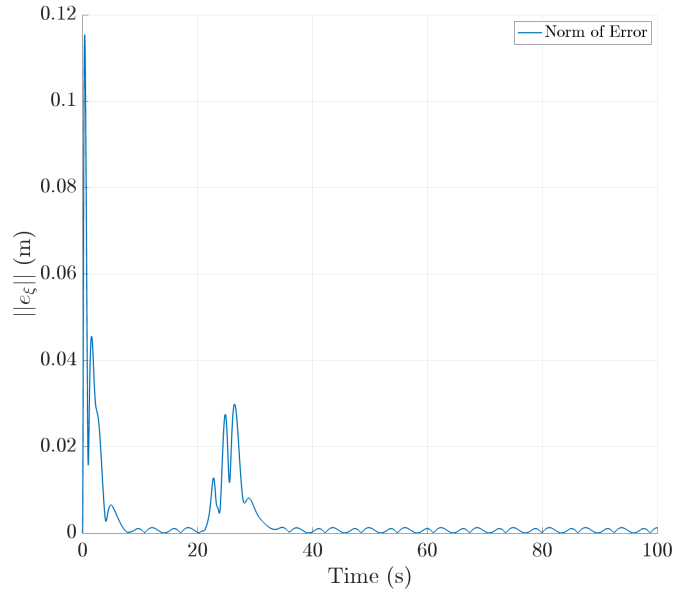


Figure 5.13: The position error norms of RL-based PID controller under the comprehensive scenario

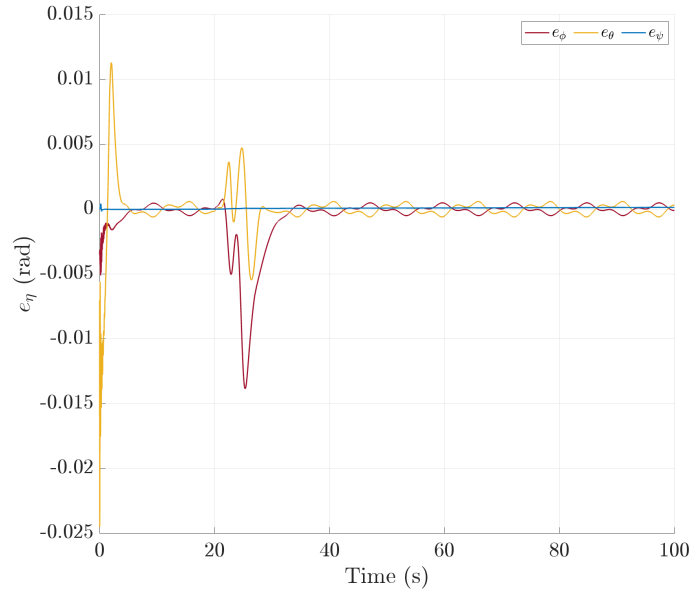


Figure 5.14: Attitude errors in the roll, pitch, and yaw angles for RL-based PID parameter estimation.

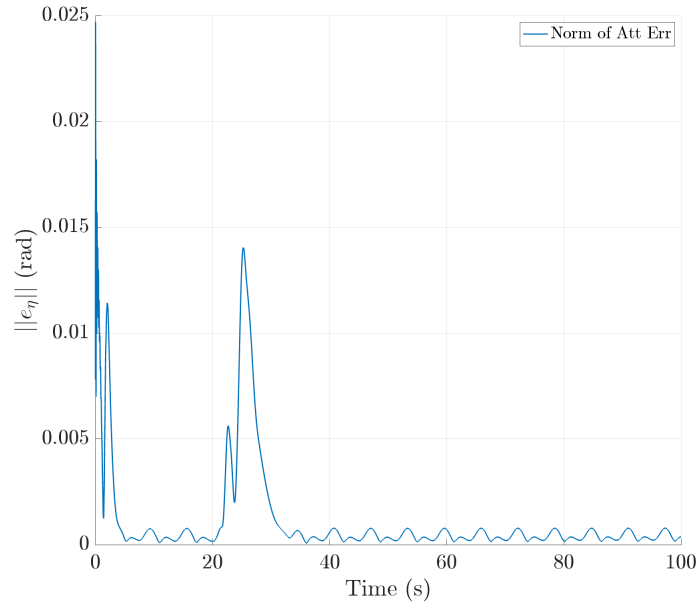


Figure 5.15: The attitude error norms of RL-based estimated controllers for the comprehensive scenario.

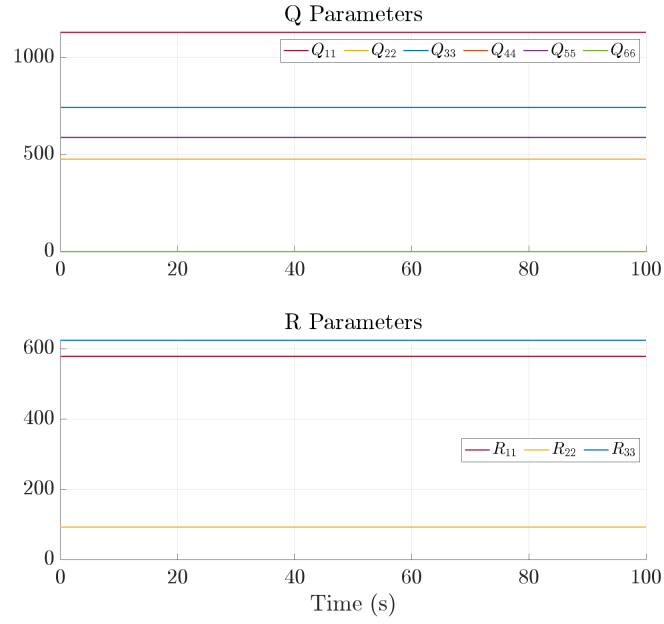


Figure 5.16: The estimated parameters based on RL in time: (a) Q matrix, (b) R matrix

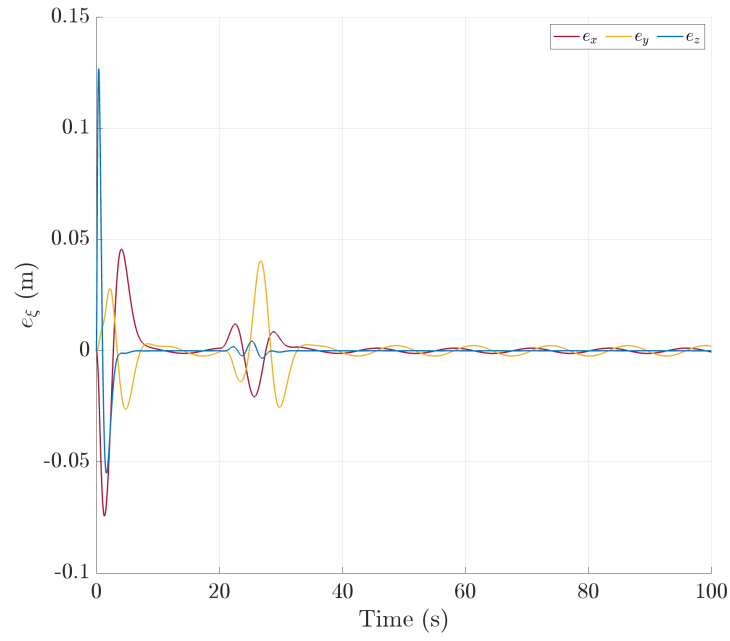


Figure 5.17: Position errors in x , y , and z axes for RL-based LQR parameter estimation.

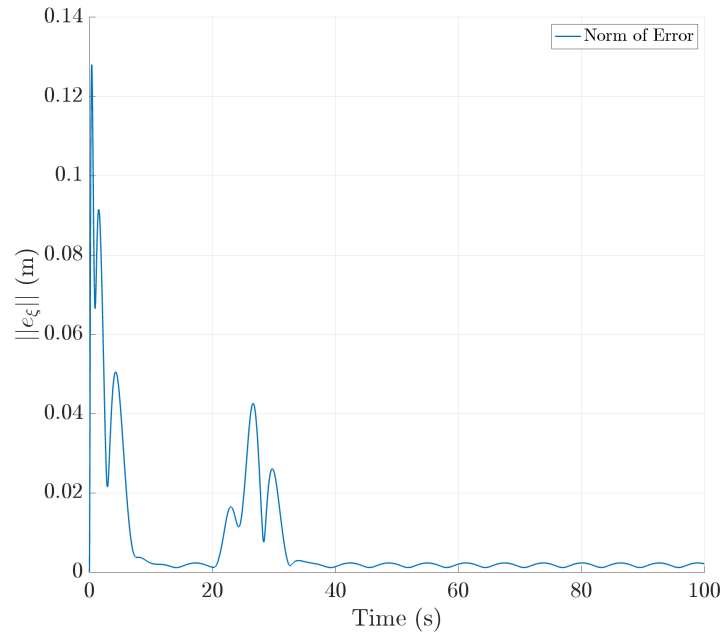


Figure 5.18: The position error norms of RL-based LQR controller.

The attitude errors exhibit a comparable response, converging within 8 seconds in the initial stages of motion. The highest initial attitude error is observed in the pitch angle (θ). However, wind disturbance, the roll angle (ϕ) shows the largest error magnitude, consistent with the findings of the RL-based PID controller framework. The effect of the wind disturbance on the attitude error persists until the 29th second, as displayed in Fig. 5.20.

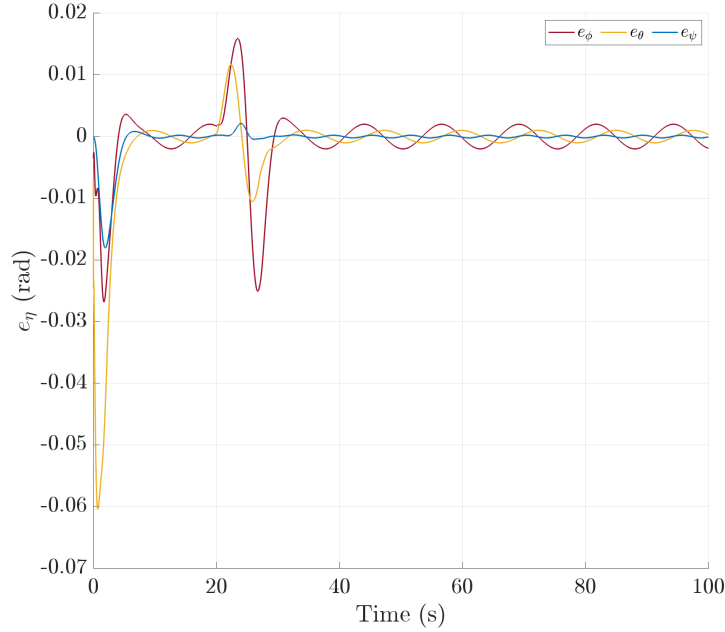


Figure 5.19: Attitude errors in the roll, pitch, and yaw angles for RL-based LQR controller.

The norm of the attitude error $\|e_\eta\|$ reaches a maximum value of 0.0265 rad under wind disturbance. Despite this temporary increase, the RL-based LQR controller framework effectively compensates for the disturbance, ensuring the quadrotor's stability, successfully completing the given task to be executed.

5.2.3. MRAC Controller Framework

The MRAC framework has also been evaluated under the same wind disturbance scenario applied to the RL-based PID and LQR controllers, providing a direct comparison of controller robustness under adverse conditions. In this setup, the MRAC

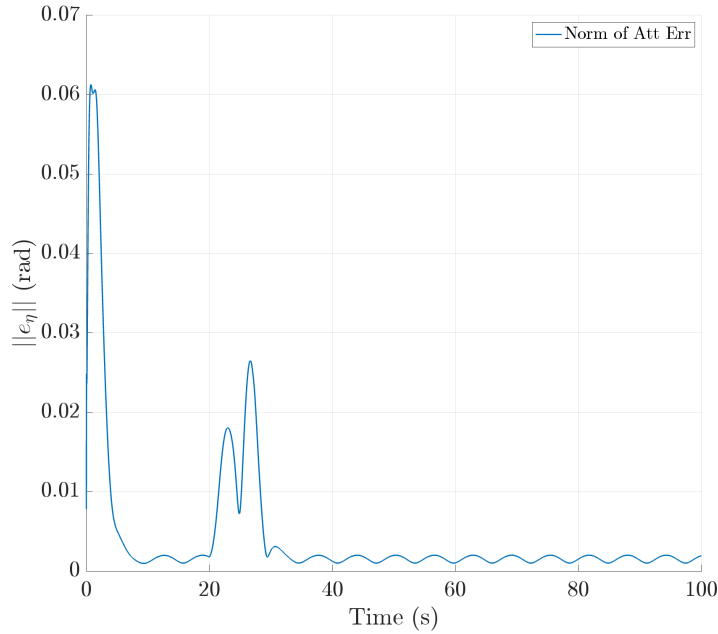


Figure 5.20: The attitude error norms of RL-based LQR controllers.

controller demonstrates effective disturbance rejection, as evidenced by the position error behavior. Position errors on the y-axis remain negligible, while errors on the z-axis exceed 0.12 m, and on the x-axis reach approximately 0.02 m. Despite these initial disturbances, all position errors settle rapidly, converging within 6 seconds, showcasing the MRAC's ability for quick adaptation and error correction in response to external forces.

The influence of the wind disturbance on the position is visible as oscillations in the x, y, and z axes, as shown in Fig. 5.22. This external effect diminishes around the 27th second, with the system returning to steady-state behavior by the 30th second, as illustrated in Fig. 5.23. In terms of attitude control, the MRAC framework also displays rapid error stabilization, achieving convergence within 6 seconds. Initially, the highest attitude error appears in the pitch angle (θ), but when the wind gust is present, the errors in both roll (ϕ) and pitch (θ) align closely in magnitude, indicating balanced control performance across these axes.

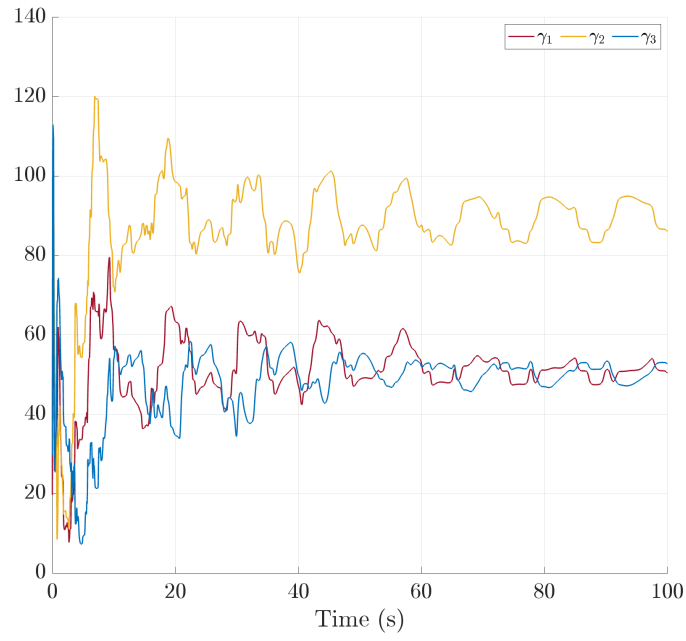


Figure 5.21: The estimated MRAC parameters based on RL in time.

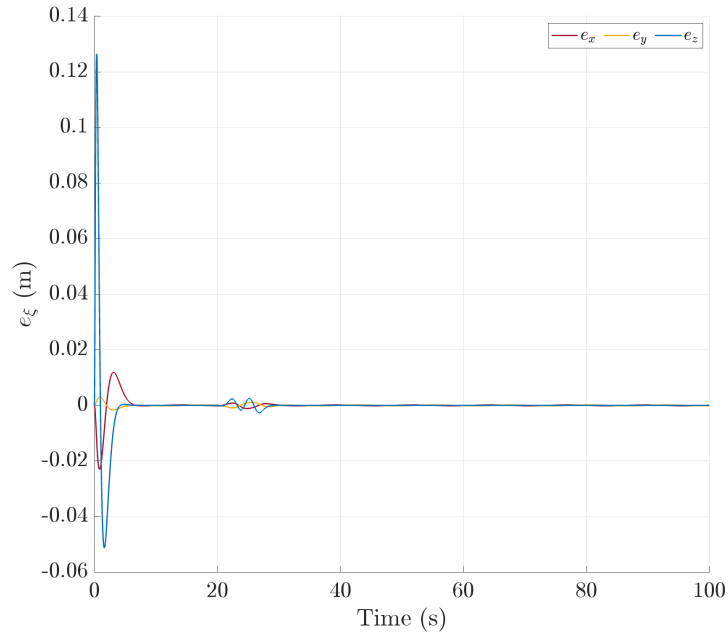


Figure 5.22: Position errors in x , y , and z axes for RL-based MRAC parameter estimation.

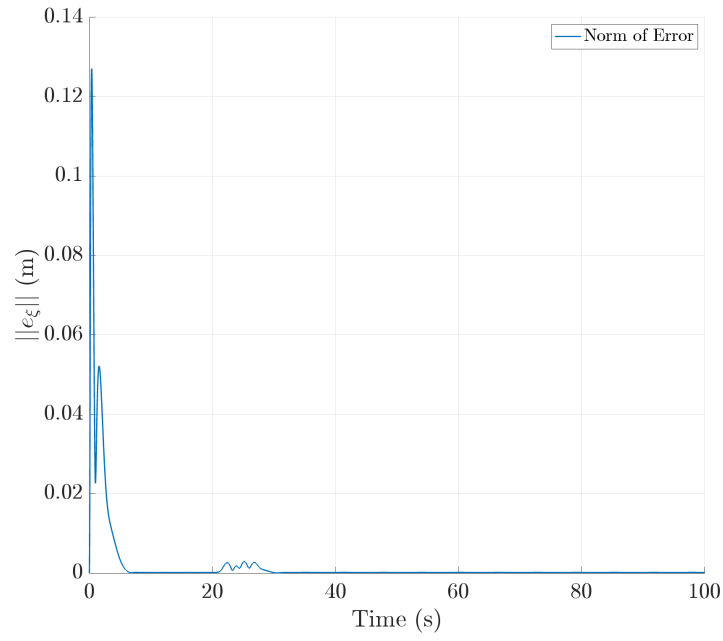


Figure 5.23: The position error norms of RL-based MRAC.

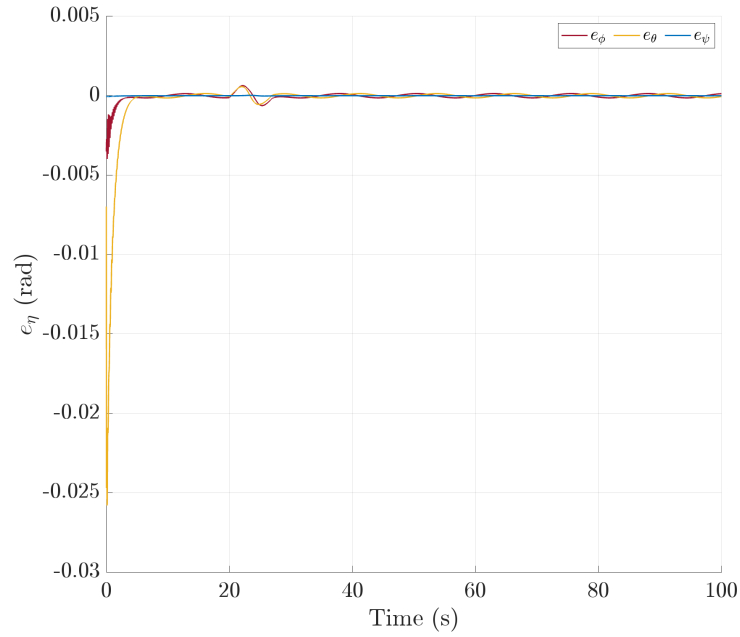


Figure 5.24: The attitude errors in the roll, pitch, and yaw angles for RL-based MRAC.

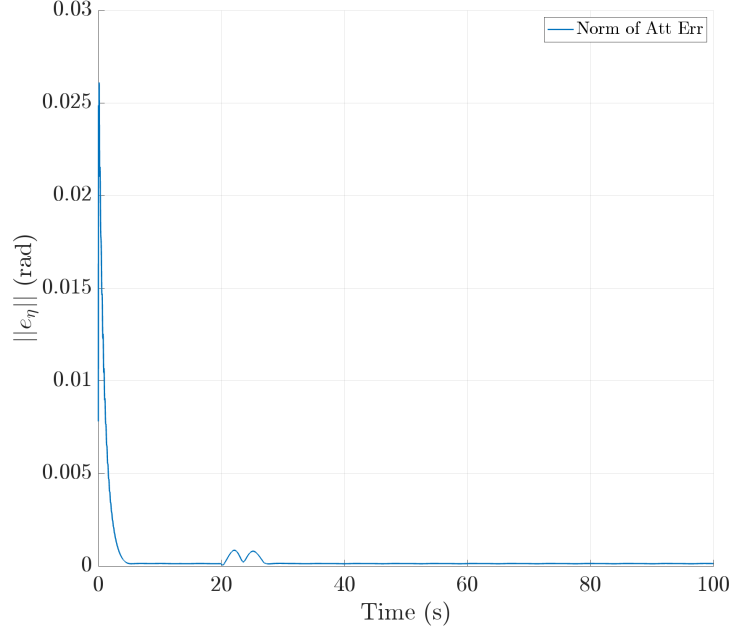


Figure 5.25: The attitude error norms of RL-based MRAC.

The MRAC's handling of attitude errors under wind disturbance is further confirmed in Figs. 5.24 and 5.25, where the norm of the attitude error, $\|e_\eta\|$, peaks at only 0.00085 rad during the wind gust event. By comparison, the MRAC framework outperforms the PID and LQR controllers in maintaining attitude stability, minimizing both the peak error and the time to recovery. This superior stability in the face of wind disturbance emphasizes the MRAC's ability to adapt dynamically, achieving more precise attitude control than the PID and LQR frameworks under identical conditions.

Table 5.2: Attitude and Position RMSEs of Controller Frameworks

	$RMSE_\eta$	$RMSE_\xi$
PID	$2.7 \times 10^{-3} \text{ rad}$	$10.2 \times 10^{-3} \text{ m}$
LQR	$10.2 \times 10^{-3} \text{ rad}$	$16.8 \times 10^{-3} \text{ m}$
MRAC	$1.9 \times 10^{-3} \text{ rad}$	$10.1 \times 10^{-3} \text{ m}$

Table 5.2 presents the root mean square errors (RMSE) for position, $RMSE_\xi$, and attitude, $RMSE_\eta$, for the PID, LQR, and MRAC controller frameworks over

a full 100-second simulation. Overall, the MRAC controller achieves the lowest RMSE in both attitude and position, with 1.9×10^{-3} rad and 10.1×10^{-3} m, respectively, demonstrating a superior ability to minimize tracking errors across the entire simulation duration. The PID controller shows slightly higher RMSE values, with 2.7×10^{-3} rad for attitude and 10.2×10^{-3} m for position. The LQR controller, however, exhibits the highest RMSE values of all three, with 10.2×10^{-3} rad in attitude and 16.8×10^{-3} m in position, indicating a less effective control performance in both attitude and position tracking under disturbance.

Table 5.3: Attitude and Position RMSEs of Controller Frameworks after 10^{th} second

	$RMSE_{\eta}$	$RMSE_{\xi}$
PID	$2.3 \times 10^{-3} \text{ rad}$	$4.9 \times 10^{-3} \text{ m}$
LQR	$5.2 \times 10^{-3} \text{ rad}$	$7.9 \times 10^{-3} \text{ m}$
MRAC	$0.21 \times 10^{-3} \text{ rad}$	$0.56 \times 10^{-3} \text{ m}$

To further examine the performance of the controllers after the initial transient response, Table 5.3 provides RMSE values calculated after the 10th second, once the system has settled and initial disturbances have dissipated. Here, the MRAC controller again demonstrates the best performance, with RMSE values significantly reduced to 0.21×10^{-3} rad for attitude and 0.56×10^{-3} m for position. This highlights MRAC’s robust adaptability and effectiveness in steady-state operation. The PID controller also improves after the initial disturbances, achieving RMSE values of 2.3×10^{-3} rad for attitude and 4.9×10^{-3} m for position, which are closer to the MRAC values but still notably higher. The LQR controller, while improved, still shows comparatively higher RMSE values after the 10th second, with 5.2×10^{-3} rad in attitude and 7.9×10^{-3} m in position.

5.3. Discussion of Results

A DDPG-based RL algorithm has been utilized for PID controller fine-tuning and parameter estimation; the method is implemented and tested to track a helix trajectory, which reflects a realistic flight scenario. The key points and contributions

of this study is the derivation of a comprehensive quadrotor model that includes gyroscopic, drag and aerodynamic effects, which are considered in the training phase of the agent. By leveraging the DDPG algorithm, an off-policy and actor-critic method, a robust and effective means of optimizing controller performance has been achieved. Performance evaluation and comparison of the manually tuned, RL-based tuned, and RL-based estimated PID controllers reveals that the RL-based estimated controller achieves the best performance. Although the targeted average reward value is increased by 15% for the RL-based PID parameter tuning as mentioned in Section 4.3.1, the performance of the RL-based tuned controller reaches 20.45% and 27.66% in terms of the RMSE values of the attitude and position, respectively. On the other hand, the RL-based estimator has converted the controller, designed as PID, to PD setting the integral component of the parameters as zero. In conclusion, the proposed DDPG-based RL approach presents a viable method to tune and estimate PID controller parameters with substantial performance improvements.

Then, motor dynamics and wind disturbances have been integrated into the training and test environment. Obtained results emphasize that the RL-based MRAC framework not only maintains superior performance over the entire simulation duration but also exhibits exceptional precision after initial disturbances have subsided, making it the most effective among the three controllers in terms of both transient and steady-state accuracy. The PID controller performs moderately well, with lower RMSE than the LQR, particularly after the initial disturbance phase, but still, the PID does not match the MRAC's level of accuracy. The LQR controller's higher RMSE values in both tables suggest that, while it provides a stable response, it is less effective in managing disturbances and minimizing tracking errors in comparison to MRAC and PID.

5.4. Remarks

In this chapter, first of all, the hierarchical PID controller has been derived for the complex mathematical model of the quadrotor. The inner loop PID controller parameters have been manually tuned and simulated for the helix trajectory tracking. The controller shows a good performance for the aggressive maneuver. To improve the performance of the controller, the agents for the control parameter tuning and estimation have been trained using the RL-based learning algorithm. The trained agents are applied to the controller framework, and the RL-based tuned and RL-based estimated controllers are compared in terms of performance. In the second step, a similar approach was applied to the RL-based PID, LQR, and MRAC controller frameworks considering motor dynamics and wind disturbance. While the RL-based MRAC controller framework shows the best performance on trajectory tracking tasks in terms of position and attitude errors, the RL-based PID and LQR controller frameworks perform slightly close to each other. Overall, the proposed RL-based controller parameter estimation method works successfully on the PID controller, optimal control method (LQR controller), and adaptive control method (MRAC). In addition, these RL-based controllers can compensate for the wind disturbances successfully.

CHAPTER 6: DISCUSSION AND CONCLUSION

6.1. Introduction

This dissertation first presented a comprehensive literature review of offline and online learning algorithms that are suitable for navigation and control of multirotor UAVs, putting emphasis on real-time implementable and applicable methods. Then, a detailed and comprehensive multirotor / quadrotor mathematical model based on the Newton-Euler formulation was introduced that includes gyroscopic effects, drag effects, motor dynamics, and wind disturbances. Three conventional controllers, PID, LQR, and MRAC, enhanced by a derived learning algorithm have been derived, implemented and tested, tuning and estimating control gains online using an offline learning-based algorithm. Obtained results pave the way to real-time implementation and testing of learning-based approaches to multirotor navigation and control.

6.2. Discussion and Conclusions

This dissertation highlights the advantages and challenges associated with learning-based control strategies for multirotor UAVs, focusing on both offline and online learning approaches. Through a comprehensive literature review, it was observed that online learning provides a critical advantage in real-time adaptability, allowing for UAV controllers to respond dynamically to environmental disturbances, changes in mission requirements, and model uncertainties. While offline learning is valuable for reducing initial training demands, online learning enhances the system's ability to fine-tune performance during actual missions. However, due to computational constraints, artificial neural networks (ANNs) in online learning

are often used in conjunction with conventional control methods, fuzzy logic, or optimization algorithms to improve efficiency and adaptability.

A DDPG-based RL algorithm has been integrated into the PID controller to tune and estimate PID controller parameters. The new PID controller framework has been tested using the complete quadrotor mathematical model including gyroscopic and drag effects. The proposed RL-based estimation PID controller framework performs better than the manually-tuned PID controller, and the RL-based fine-tuned PID controller framework. The proposed DDPG-based RL algorithm can also be integrated into optimal (LQR) and adaptive (MRAC) controller methods. This method has also been tested using the more complex quadrotor mathematical model including motor dynamics and wind disturbances, in addition to the provided aerodynamic effects.

One observation is that although it was expected that the RL-based estimated LQR controller would outperform the RL-based estimated PID controller, obtained results indicate the opposite. This may be attributed, most likely, to the learning phase (training phase) process, where the diagonal elements of the Q and R matrices are estimated and updated by the RL algorithm and they are fed into the LQR inner loop; the algebraic Riccati equation shown in (4.11) is solved iteratively. The solution of the Riccati equation requires integrating $\dot{K}$ to obtain K, which relies on using discrete integral blocks in Simulink.

However, discrete integral blocks have inherent limitations. Numerical approximation errors, sensitivity to time-step size, and stability challenges may lead to inaccuracies in the computation of K. These issues are compounded in the LQR framework, where precise computation of K is critical. For example, errors or instability during integration may cause K to exceed reasonable values, destabilizing the learning process. To mitigate this, saturation blocks were introduced, limiting the maximum values of the K and F matrices to 10,000. While this ensured stabil-

ity, it also restricted the controller’s ability to explore and optimize Q and R values, preventing the system from achieving higher reward values. Consequently, the LQR controller’s performance was compromised, leading to inferior results compared to the PID controller framework.

The proposed method has two main advantages. First, the PID controller can also be turned to a P, PI, or PD controller depending on given constraints. Second, the overall framework may be applicable to control families of nonlinear systems, from ground vehicles to spacecraft.

On the other hand, the proposed and adopted estimation method is somehow limited on identifying aerodynamic uncertainties of the underlying system. To be specific, estimation of aerodynamic uncertainties, specifically the drag coefficients, does not adequately capture the necessary precision for robust control adjustments. It lacks the sensitivity to accurately estimate parameters within this order of magnitude, leading to challenges in real-time compensation for drag-related forces and torques.

At this stage, based on gained experience from the conducted research, obtained results and observations, it is essential to (try to) provide some systematic guideline with the aim to hopefully assist researchers and engineers in implementing and testing the proposed methodology on their testbeds with minor modifications. Key attributes, challenges, that need to be considered relate to the applied nonlinear controller and/or chosen platform model. The NN structure, state and action spaces, reward function and hyper-parameters may be changed for implementation purposes based on different platform models or controller framework, and for improving training phase performance.

- **NN Structure:** The number of layers and nodes in each layer must be adjusted to consider underlying system complexity, computational cost limitation, or implementation limitations i.e., board memory capacity.

- **State and Action Spaces:** The controller framework, platform model, and tasks play key roles when choosing the state space. For instance, the roll and pitch angles are crucial for even a basic quadrotor, while they can be negligible for a ground vehicle, UGV.
- **Reward Function:** The reward function reflects what the system should aim for, and directly influences the system’s learning process, as it determines which behaviors are reinforced. It also balances competing objectives and overcomes the exploration and exploitation dilemma. Briefly, the reward function is the bridge between the system’s learning process and the task’s requirements. Its design requires a deep understanding of the task, constraints, and possible trade-offs. A carefully crafted reward function ensures efficient training, robust performance, and alignment with the intended objectives.
- **Hyper-parameters:** Hyper-parameter selection must align with the task’s complexity, dynamics, and platform constraints. Systematic tuning and task-specific adaptation ensure efficient training and robust performance.

6.3. Future Work

This dissertation has developed a control framework that improves quadrotor control through reinforcement learning-based parameter estimation and adaptation techniques. However, it also paves the way to future research challenges, some of which are presented below.

First, actual real-time implementation of the proposed control architecture on a physical quadrotor is the logical next step. Real-world testing would allow for evaluation of the controller’s performance under actual flight conditions, including more complex disturbance profiles and sensor noise, providing insight for robustness. Such implementation may also highlight any hardware-related limitations or adjustments needed to ensure reliable control.

Further research related to developing online learning algorithms will significantly enhance the controller’s adaptability. An online learning approach may enable the quadrotor to continuously learn and adapt its control parameters in response to real-time changes in the environment, such as varying wind conditions, payload shifts, or unexpected obstacles. This may also allow for the controller to maintain optimal performance across a wider range of operational scenarios, making it more suitable to function in uncertain environments. Developing an efficient, low-computation online learning strategy could thus greatly expand the practical utility of the proposed framework.

Incorporating these advancements could make the proposed controller framework even more resilient and versatile, bridging the gap between simulation and real-world quadrotor applications.

REFERENCES

- [1] A. M. Turing, *Computing machinery and intelligence*. Springer, 2009.
- [2] B. J. Copeland, *The essential turing*. Clarendon Press, 2004.
- [3] C. Martinez, C. Sampedro, A. Chauhan, and P. Campoy, “Towards autonomous detection and tracking of electric towers for aerial power line inspection,” in *2014 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 284–295, IEEE, 2014.
- [4] H. Ren, Y. Zhao, W. Xiao, and Z. Hu, “A review of uav monitoring in mining areas: Current status and future perspectives,” *International Journal of Coal Science & Technology*, vol. 6, pp. 320–333, 2019.
- [5] M. A. Olivares-Mendez, C. Fu, P. Ludvig, T. F. Bissyandé, S. Kannan, M. Zuread, A. Annaiyan, H. Voos, and P. Campoy, “Towards an autonomous vision-based unmanned aerial system against wildlife poachers,” *Sensors*, vol. 15, no. 12, pp. 31362–31391, 2015.
- [6] R. Bassoli, C. Sacchi, F. Granelli, and I. Ashkenazi, “A virtualized border control system based on uavs: Design and energy efficiency considerations,” in *2019 IEEE Aerospace Conference*, pp. 1–11, IEEE, 2019.
- [7] A. Carrio, J. Pestana, J.-L. Sanchez-Lopez, R. Suarez-Fernandez, P. Campoy, R. Tendero, M. García-De-Viedma, B. González-Rodrigo, J. Bonatti, J. G. Rejas-Ayuga, *et al.*, “Ubristes: Uav-based building rehabilitation with visible and thermal infrared remote sensing,” in *Robot 2015: Second Iberian Robotics Conference: Advances in Robotics, Volume 1*, pp. 245–256, Springer, 2016.
- [8] L. Li, Y. Fan, X. Huang, and L. Tian, “Real-time uav weed scout for selective weed control by adaptive robust control and machine learning algorithm,” in *2016 ASABE Annual International Meeting*, p. 1, American Society of Agricultural and Biological Engineers, 2016.
- [9] S. Sönmez, M. J. Rutherford, and K. P. Valavanis, “A survey of offline-and online-learning-based algorithms for multirotor uavs,” *Drones*, vol. 8, no. 4, p. 116, 2024.
- [10] A. Carrio, C. Sampedro, A. Rodriguez-Ramos, and P. Campoy, “A review of deep learning methods and applications for unmanned aerial vehicles,” *Journal of Sensors*, vol. 2017, 2017.

- [11] A. S. Polydoros and L. Nalpantidis, “Survey of model-based reinforcement learning: Applications on robotics,” *Journal of Intelligent & Robotic Systems*, vol. 86, no. 2, pp. 153–173, 2017.
- [12] S. Y. Choi and D. Cha, “Unmanned aerial vehicles using machine learning for autonomous flight; state-of-the-art,” *Advanced Robotics*, vol. 33, no. 6, pp. 265–277, 2019.
- [13] A. T. Azar, A. Koubaa, N. Ali Mohamed, H. A. Ibrahim, Z. F. Ibrahim, M. Kazim, A. Ammar, B. Benjdira, A. M. Khamis, I. A. Hameed, *et al.*, “Drone deep reinforcement learning: A review,” *Electronics*, vol. 10, no. 9, p. 999, 2021.
- [14] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, “Safe learning in robotics: From learning-based control to safe reinforcement learning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.
- [15] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [16] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” *arXiv preprint arXiv:2005.01643*, 2020.
- [17] R. Bartak and A. Vykovský, “Any object tracking and following by a flying drone,” in *2015 Fourteenth Mexican International Conference on Artificial Intelligence (MICAI)*, pp. 35–41, IEEE, 2015.
- [18] S. Edhah, S. Mohamed, A. Rehan, M. AlDhaheri, A. AlKhaja, and Y. Zweiri, “Deep learning based neural network controller for quad copter: Application to hovering mode,” in *2019 International Conference on Electrical and Computing Technologies and Applications (ICECTA)*, pp. 1–5, IEEE, 2019.
- [19] Y. Xu, Z. Liu, and X. Wang, “Monocular vision based autonomous landing of quadrotor through deep reinforcement learning,” in *2018 37th Chinese Control Conference (CCC)*, pp. 10014–10019, IEEE, 2018.
- [20] A. Rodriguez-Ramos, C. Sampedro, H. Bavle, P. De La Puente, and P. Campoy, “A deep reinforcement learning strategy for uav autonomous landing on a moving platform,” *Journal of Intelligent & Robotic Systems*, vol. 93, no. 1, pp. 351–366, 2019.

- [21] J. Yoo, D. Jang, H. J. Kim, and K. H. Johansson, “Hybrid reinforcement learning control for a micro quadrotor flight,” *IEEE Control Systems Letters*, vol. 5, no. 2, pp. 505–510, 2020.
- [22] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, “Online learning: A comprehensive survey,” *Neurocomputing*, vol. 459, pp. 249–289, 2021.
- [23] A. Giusti, J. Guzzi, D. C. Cireşan, F.-L. He, J. P. Rodríguez, F. Fontana, M. Faessler, C. Forster, J. Schmidhuber, G. Di Caro, *et al.*, “A machine learning approach to visual perception of forest trails for mobile robots,” *IEEE Robotics and Automation Letters*, vol. 1, no. 2, pp. 661–667, 2015.
- [24] E. Kaufmann, A. Loquercio, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, “Deep drone racing: Learning agile flight in dynamic environments,” in *Conference on Robot Learning*, pp. 133–145, PMLR, 2018.
- [25] J. Janousek, P. Marcon, J. Klouda, J. Pokorný, P. Raichl, and A. Sirucková, “Deep neural network for precision landing and variable flight planning of autonomous uav,” in *2021 Photonics & Electromagnetics Research Symposium (PIERS)*, pp. 2243–2247, IEEE, 2021.
- [26] S. Vladov, Y. Shmelov, R. Yakovliev, A. Khebda, and O. Brusakova, “Modified neural network method for stabilizing multi-rotor unmanned aerial vehicles,” 2023.
- [27] D. K. Kim and T. Chen, “Deep neural network for real-time autonomous indoor navigation,” *arXiv preprint arXiv:1511.04668*, 2015.
- [28] Q. Li, J. Qian, Z. Zhu, X. Bao, M. K. Helwa, and A. P. Schoellig, “Deep neural networks for improved, impromptu trajectory tracking of quadrotors,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5183–5189, IEEE, 2017.
- [29] N. Smolyanskiy, A. Kamenev, J. Smith, and S. Birchfield, “Toward low-flying autonomous mav trail navigation using deep neural networks for environmental awareness,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4241–4247, IEEE, 2017.
- [30] S. Jung, S. Hwang, H. Shin, and D. H. Shim, “Perception, guidance, and navigation for indoor autonomous drone racing using deep learning,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 2539–2544, 2018.

- [31] A. Loquercio, A. I. Maqueda, C. R. Del-Blanco, and D. Scaramuzza, “Dronet: Learning to fly by driving,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 1088–1095, 2018.
- [32] D. Mantegazza, J. Guzzi, L. M. Gambardella, and A. Giusti, “Vision-based control of a quadrotor in user proximity: Mediated vs end-to-end learning approaches,” in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 6489–6495, IEEE, 2019.
- [33] J. A. Cardenas, U. E. Carrero, E. C. Camacho, and J. M. Calderon, “Intelligent position controller for unmanned aerial vehicles (uav) based on supervised deep learning,” *Machines*, vol. 11, no. 6, p. 606, 2023.
- [34] N. Imanberdiyev, C. Fu, E. Kayacan, and I.-M. Chen, “Autonomous navigation of uav by using real-time model-based reinforcement learning,” in *2016 14th international conference on control, automation, robotics and vision (ICARCV)*, pp. 1–6, IEEE, 2016.
- [35] R. Polvara, M. Patacchiola, S. Sharma, J. Wan, A. Manning, R. Sutton, and A. Cangelosi, “Autonomous quadrotor landing using deep reinforcement learning,” *arXiv preprint arXiv:1709.03339*, 2017.
- [36] S. Choi, S. Kim, and H. Jin Kim, “Inverse reinforcement learning control for trajectory tracking of a multirotor uav,” *International Journal of Control, Automation and Systems*, vol. 15, no. 4, pp. 1826–1834, 2017.
- [37] G. Kahn, T. Zhang, S. Levine, and P. Abbeel, “Plato: Policy learning using adaptive trajectory optimization,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3342–3349, IEEE, 2017.
- [38] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, “Control of a quadrotor with reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2096–2103, 2017.
- [39] S. Lee, T. Shim, S. Kim, J. Park, K. Hong, and H. Bang, “Vision-based autonomous landing of a multi-copter unmanned aerial vehicle using reinforcement learning,” in *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 108–114, IEEE, 2018.
- [40] M. B. Vankadari, K. Das, C. Shinde, and S. Kumar, “A reinforcement learning approach for autonomous control and landing of a quadrotor,” in *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 676–683, IEEE, 2018.

- [41] K. Kersandt, G. Muñoz, and C. Barrado, “Self-training by reinforcement learning for full-autonomous drones of the future,” in *2018 IEEE/AIAA 37th Digital Avionics Systems Conference (DASC)*, pp. 1–10, IEEE, 2018.
- [42] H. X. Pham, H. M. La, D. Feil-Seifer, and L. Van Nguyen, “Reinforcement learning for autonomous uav navigation using function approximation,” in *2018 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pp. 1–6, IEEE, 2018.
- [43] H. Liu, W. Zhao, F. L. Lewis, Z.-P. Jiang, and H. Modares, “Attitude synchronization for multiple quadrotors using reinforcement learning,” in *2019 Chinese Control Conference (CCC)*, pp. 2480–2483, IEEE, 2019.
- [44] N. O. Lambert, D. S. Drew, J. Yaconelli, S. Levine, R. Calandra, and K. S. Pister, “Low-level control of a quadrotor with deep model-based reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4224–4230, 2019.
- [45] A. Manukyan, M. A. Olivares-Mendez, M. Geist, and H. Voos, “Deep reinforcement learning-based continuous control for multicopter systems,” in *2019 6th International Conference on Control, Decision and Information Technologies (CoDIT)*, pp. 1876–1881, IEEE, 2019.
- [46] R. Srivastava, R. Lima, K. Das, and A. Maity, “Least square policy iteration for ibvs based dynamic target tracking,” in *2019 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 1089–1098, IEEE, 2019.
- [47] C. Wu, B. Ju, Y. Wu, X. Lin, N. Xiong, G. Xu, H. Li, and X. Liang, “Uav autonomous target search based on deep reinforcement learning in complex disaster scene,” *IEEE Access*, vol. 7, pp. 117227–117245, 2019.
- [48] C. Wang, J. Wang, Y. Shen, and X. Zhang, “Autonomous navigation of uavs in large-scale complex environments: A deep reinforcement learning approach,” *IEEE Transactions on Vehicular Technology*, vol. 68, no. 3, pp. 2124–2136, 2019.
- [49] Y. Zeng and X. Xu, “Path design for cellular-connected uav with reinforcement learning,” in *2019 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–6, IEEE, 2019.
- [50] B. Rubí, B. Morcego, and R. Pérez, “A deep reinforcement learning approach for path following on a quadrotor,” in *2020 European Control Conference (ECC)*, pp. 1092–1098, IEEE, 2020.

- [51] C.-H. Pi, K.-C. Hu, S. Cheng, and I.-C. Wu, “Low-level autonomous control and tracking of quadrotor using reinforcement learning,” *Control Engineering Practice*, vol. 95, p. 104222, 2020.
- [52] W. Zhao, H. Liu, and F. L. Lewis, “Robust formation control for cooperative underactuated quadrotors via reinforcement learning,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 10, pp. 4577–4587, 2020.
- [53] A. Guerra, F. Guidi, D. Dardari, and P. M. Djurić, “Reinforcement learning for uav autonomous navigation, mapping and target detection,” in *2020 IEEE/ION Position, Location and Navigation Symposium (PLANS)*, pp. 1004–1013, IEEE, 2020.
- [54] B. Li, Z. Gan, D. Chen, and D. Sergey Aleksandrovich, “Uav maneuvering target tracking in uncertain environments based on deep reinforcement learning and meta-learning,” *Remote Sensing*, vol. 12, no. 22, p. 3789, 2020.
- [55] S. Kulkarni, V. Chaphekar, M. M. U. Chowdhury, F. Erden, and I. Guvenc, “Uav aided search and rescue operation using reinforcement learning,” in *2020 SoutheastCon*, vol. 2, pp. 1–8, IEEE, 2020.
- [56] H. Hu and Q.-l. Wang, “Proximal policy optimization with an integral compensator for quadrotor control,” *Frontiers of Information Technology & Electronic Engineering*, vol. 21, no. 5, pp. 777–795, 2020.
- [57] J. E. Kooi and R. Babuška, “Inclined quadrotor landing using deep reinforcement learning,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2361–2368, IEEE, 2021.
- [58] B. Rubí, B. Morcego, and R. Pérez, “Deep reinforcement learning for quadrotor path following with adaptive velocity,” *Autonomous Robots*, vol. 45, no. 1, pp. 119–134, 2021.
- [59] L. Bhan, M. Quinones-Grueiro, and G. Biswas, “Fault tolerant control combining reinforcement learning and model-based control,” in *2021 5th International Conference on Control and Fault-Tolerant Systems (SysTol)*, pp. 31–36, IEEE, 2021.
- [60] X. Li, J. Zhang, and J. Han, “Trajectory planning of load transportation with multi-quadrotors based on reinforcement learning algorithm,” *Aerospace Science and Technology*, vol. 116, p. 106887, 2021.
- [61] Z. Jiang and G. Song, “A deep reinforcement learning strategy for uav autonomous landing on a platform,” *arXiv preprint arXiv:2209.02954*, 2022.

- [62] N. Abo Mosali, S. S. Shamsudin, S. A. Mostafa, O. Alfandi, R. Omar, N. Al-Fadhali, M. A. Mohammed, R. Malik, M. M. Jaber, and A. Saif, “An adaptive multi-level quantization-based reinforcement learning model for enhancing uav landing on moving targets,” *Sustainability*, vol. 14, no. 14, p. 8825, 2022.
- [63] F. Panetsos, G. C. Karras, and K. J. Kyriakopoulos, “A deep reinforcement learning motion control strategy of a multi-rotor uav for payload transportation with minimum swing,” in *2022 30th Mediterranean Conference on Control and Automation (MED)*, pp. 368–374, IEEE, 2022.
- [64] Z. Ye, K. Wang, Y. Chen, X. Jiang, and G. Song, “Multi-uav navigation for partially observable communication coverage by graph reinforcement learning,” *IEEE Transactions on Mobile Computing*, 2022.
- [65] X. Wang and X. Ye, “Consciousness-driven reinforcement learning: An on-line learning control framework,” *International Journal of Intelligent Systems*, vol. 37, no. 1, pp. 770–798, 2022.
- [66] M. Farsi and J. Liu, “Structured online learning for low-level control of quadro-tors,” in *2022 American Control Conference (ACC)*, pp. 1242–1247, IEEE, 2022.
- [67] K. Xia, Y. Huang, Y. Zou, and Z. Zuo, “Reinforcement learning control for moving target landing of vtol uavs with motion constraints,” *IEEE Transactions on Industrial Electronics*, 2023.
- [68] B. Ma, Z. Liu, Q. Dang, W. Zhao, J. Wang, Y. Cheng, and Z. Yuan, “Deep reinforcement learning of uav tracking control under wind disturbances environments,” *IEEE Transactions on Instrumentation and Measurement*, 2023.
- [69] G. G. d. Castro, G. S. Berger, A. Cantieri, M. Teixeira, J. Lima, A. I. Pereira, and M. F. Pinto, “Adaptive path planning for fusing rapidly exploring random trees and deep reinforcement learning in an agriculture dynamic environment uavs,” *Agriculture*, vol. 13, no. 2, p. 354, 2023.
- [70] A. Mitakidis, S. N. Aspragkathos, F. Panetsos, G. C. Karras, and K. J. Kyriakopoulos, “A deep reinforcement learning visual servoing control strategy for target tracking using a multirotor uav,” in *2023 9th International Conference on Automation, Robotics and Applications (ICARA)*, pp. 219–224, IEEE, 2023.
- [71] M. Shurrab, R. Mizouni, S. Singh, and H. Otok, “Reinforcement learning framework for uav-based target localization applications,” *Internet of Things*, vol. 23, p. 100867, 2023.

- [72] Z. Huang, Z. Yang, R. Krupani, B. Şenbaşlar, S. Batra, and G. S. Sukhatme, “Collision avoidance and navigation for a quadrotor swarm using end-to-end deep reinforcement learning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 300–306, IEEE, 2024.
- [73] X. Liu, Z. Yuan, Z. Gao, and W. Zhang, “Reinforcement learning-based fault-tolerant control for quadrotor uavs under actuator fault,” *IEEE Transactions on Industrial Informatics*, 2024.
- [74] X. Yi, H. Liu, Y. Wang, H. Duan, and K. P. Valavanis, “Safe reinforcement learning-based visual servoing control for quadrotors tracking unknown ground vehicles,” *IEEE Transactions on Intelligent Vehicles*, 2024.
- [75] P. Santana, L. Correia, R. Mendonça, N. Alves, and J. Barata, “Tracking natural trails with swarm-based visual saliency,” *Journal of Field Robotics*, vol. 30, no. 1, pp. 64–86, 2013.
- [76] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- [77] “Caffe | model zoo.” http://caffe.berkeleyvision.org/model_zoo.html. (Accessed on 10/11/2022).
- [78] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [79] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [80] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “Ssd: Single shot multibox detector,” in *European conference on computer vision*, pp. 21–37, Springer, 2016.
- [81] K. Li, W. Ni, and F. Dressler, “Lstm-characterized deep reinforcement learning for continuous flight control and resource allocation in uav-assisted sensor network,” *IEEE Internet of Things Journal*, vol. 9, no. 6, pp. 4179–4189, 2021.
- [82] E. Bilgin, *Mastering reinforcement learning with python: build next-generation, self-learning models using reinforcement learning techniques and best practices*. Packt Publishing Ltd, 2020.

- [83] M. Lapan, *Deep Reinforcement Learning Hands-On: Apply modern RL methods to practical problems of chatbots, robotics, discrete optimization, web automation, and more*. Packt Publishing Ltd, 2020.
- [84] M. Lapan, *Deep Reinforcement Learning Hands-On: Apply modern RL methods, with deep Q-networks, value iteration, policy gradients, TRPO, AlphaGo Zero and more*. Packt Publishing Ltd, 2018.
- [85] R. Polvara, M. Patacchiola, M. Hanheide, and G. Neumann, “Sim-to-real quadrotor landing via sequential deep q-networks and domain randomization,” *Robotics*, vol. 9, no. 1, p. 8, 2020.
- [86] M. Farsi and J. Liu, “Structured online learning-based control of continuous-time nonlinear systems,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 8142–8149, 2020.
- [87] C. Kanellakis and G. Nikolakopoulos, “Survey on computer vision for uavs: Current developments and trends,” *Journal of Intelligent & Robotic Systems*, vol. 87, pp. 141–168, 2017.
- [88] L. Wang, A. D. Ames, and M. Egerstedt, “Safety barrier certificates for collisions-free multirobot systems,” *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 661–674, 2017.
- [89] B. Riviere, W. Hönig, Y. Yue, and S.-J. Chung, “Glas: Global-to-local safe autonomy synthesis for multi-robot motion planning with end-to-end learning,” *IEEE robotics and automation letters*, vol. 5, no. 3, pp. 4249–4256, 2020.
- [90] B. L. Stevens, F. L. Lewis, and E. N. Johnson, *Aircraft control and simulation: dynamics, controls design, and autonomous systems*. John Wiley & Sons, 2015.
- [91] B. Xu, F. Gao, C. Yu, R. Zhang, Y. Wu, and Y. Wang, “Omnidrones: An efficient and flexible platform for reinforcement learning in drone control,” *IEEE Robotics and Automation Letters*, 2024.
- [92] Y. Yang, H. Modares, D. C. Wunsch, and Y. Yin, “Leader–follower output synchronization of linear heterogeneous systems with active leader using reinforcement learning,” *IEEE transactions on neural networks and learning systems*, vol. 29, no. 6, pp. 2139–2153, 2018.
- [93] A. Das and F. L. Lewis, “Distributed adaptive control for synchronization of unknown nonlinear networked systems,” *Automatica*, vol. 46, no. 12, pp. 2014–2021, 2010.

- [94] L. Wang, E. A. Theodorou, and M. Egerstedt, “Safe learning of quadrotor dynamics using barrier certificates,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2460–2465, IEEE, 2018.
- [95] A. Sarabakha and E. Kayacan, “Online deep learning for improved trajectory tracking of unmanned aerial vehicles using expert knowledge,” in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 7727–7733, IEEE, 2019.
- [96] S. He, W. Wang, H. Yang, Y. Cao, T. Jiang, and Q. Zhang, “State-aware rate adaptation for uavs by incorporating on-board sensors,” *IEEE Transactions on Vehicular Technology*, vol. 69, no. 1, pp. 488–496, 2019.
- [97] Y. Wang, J. Sun, H. He, and C. Sun, “Deterministic policy gradient with integral compensator for robust quadrotor control,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 50, no. 10, pp. 3713–3725, 2019.
- [98] S. Shin, Y. Kang, and Y.-G. Kim, “Evolution algorithm and online learning for racing drone,” in *NeurIPS 2019 Competition and Demonstration Track*, pp. 100–109, PMLR, 2020.
- [99] H. Shiri, J. Park, and M. Bennis, “Remote uav online path planning via neural network-based opportunistic control,” *IEEE Wireless Communications Letters*, vol. 9, no. 6, pp. 861–865, 2020.
- [100] V. Jaiton, K. Rothomphiwat, E. Ebeid, and P. Manoonpong, “Neural control and online learning for speed adaptation of unmanned aerial vehicles,” *Frontiers in neural circuits*, vol. 16, 2022.
- [101] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Pretraining neural-networks with neural-fly for rapid online learning,” in *ICRA2023 Workshop on Pretraining for Robotics (PT4R)*, 2023.
- [102] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *2011 IEEE international conference on robotics and automation*, pp. 2520–2525, IEEE, 2011.
- [103] K. Jia, S. Lin, Y. Du, C. Zou, and M. Lu, “Research on route tracking controller of quadrotor uav based on fuzzy logic and rbf neural network,” *IEEE Access*, 2023.
- [104] Y. Zhang, Y. Yang, W. Chen, and H. Yang, “Realtime brain-inspired adaptive learning control for nonlinear systems with configuration uncertainties,” *IEEE Transactions on Automation Science and Engineering*, 2023.

- [105] D. Srinivasan, *Innovations in Multi-Agent Systems and Application-1*, vol. 310. Springer, 2010.
- [106] M. Van Otterlo and M. Wiering, “Reinforcement learning and markov decision processes,” in *Reinforcement learning: State-of-the-art*, pp. 3–42, Springer, 2012.
- [107] T. Luukkonen, “Modelling and control of quadcopter,” *Independent Research Project in Applied Mathematics, Espoo*, vol. 22, no. 22, 2011.
- [108] M. Bangura, R. Mahony, *et al.*, “Nonlinear dynamic modeling for high performance control of a quadrotor,” 2012.
- [109] G. J. Leishman, *Principles of helicopter aerodynamics with CD extra*. Cambridge university press, 2006.
- [110] J.-M. Pfimlin, P. Binetti, P. Soueres, T. Hamel, and D. Trouchet, “Modeling and attitude control analysis of a ducted-fan micro aerial vehicle,” *Control Engineering Practice*, vol. 18, no. 3, pp. 209–218, 2010.
- [111] C. Powers, D. Mellinger, and V. Kumar, “Quadrotor kinematics and dynamics,” *Handbook of Unmanned Aerial Vehicles*, pp. 307–328, 2015.
- [112] E. Tal and S. Karaman, “Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness,” *IEEE Transactions on Control Systems Technology*, vol. 29, no. 3, pp. 1203–1218, 2020.
- [113] P. Anderson and A. Bose, “Stability simulation of wind turbine systems,” *IEEE transactions on power apparatus and systems*, no. 12, pp. 3791–3795, 1983.
- [114] M. Walid, N. Slaheddine, A. Mohamed, and B. Lamjed, “Modelling, identification and control of a quadrotor uav,” in *2018 15th International Multi-Conference on Systems, Signals & Devices (SSD)*, pp. 1017–1022, IEEE, 2018.
- [115] S. Islam, P. Liu, and A. El Saddik, “Nonlinear adaptive control for quadrotor flying vehicle,” *Nonlinear Dynamics*, vol. 78, pp. 117–133, 2014.
- [116] C. Knospe, “Pid control,” *IEEE Control Systems Magazine*, vol. 26, no. 1, pp. 30–31, 2006.
- [117] S. Martini, S. Sönmez, A. Rizzo, M. Stefanovic, M. J. Rutherford, and K. P. Valavanis, “Euler-lagrange modeling and control of quadrotor uav with aerodynamic compensation,” in *2022 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 369–377, IEEE, 2022.

- [118] S. Martini, M. Stefanovic, A. Rizzo, M. J. Rutherford, P. Livreri, and K. P. Valavanis, “A benchmark framework for testing, evaluation, and comparison of quadrotor linear and nonlinear controllers,” in *2023 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 471–478, IEEE, 2023.
- [119] Z. Zuo, “Trajectory tracking control design with command-filtered compensation for a quadrotor,” *IET Control Theory & Applications*, vol. 4, no. 11, pp. 2343–2355, 2010.
- [120] J. Alvarenga, N. I. Vitzilaios, K. P. Valavanis, and M. J. Rutherford, “Survey of unmanned helicopter model-based navigation and control techniques,” *Journal of Intelligent & Robotic Systems*, vol. 80, pp. 87–138, 2015.
- [121] P. Ioannou and B. Fidan, *Adaptive control tutorial*. SIAM, 2006.
- [122] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic policy gradient algorithms,” in *International conference on machine learning*, pp. 387–395, Pmlr, 2014.
- [123] L. Fan, J. Zhang, Y. He, Y. Liu, T. Hu, and H. Zhang, “Optimal scheduling of microgrid based on deep deterministic policy gradient and transfer learning,” *Energies*, vol. 14, no. 3, p. 584, 2021.